

Linux на встраиваемых устройствах

Лабораторный практикум

Лаборатория «СГУ — Базальт СПО»

2026

Содержание

1	Лабораторная №1. Базовые навыки работы с одноплатным компьютером	3
2	Лабораторная №2. Работа с платой по SSH	12
3	Лабораторная №3. Использование кросс-компилятора	18
4	Лабораторная №4. Создание основных компонентов образа: ядро Linux	23
5	Лабораторная №5. Создание основных компонентов образа: дерево устройств, корневая файловая система и загрузчик	29
6	Лабораторная №6. Подготовка носителя и запись образа	36
7	Лабораторная №7. Добавление поддержки интерфейса SPI	42
8	Лабораторная №8. Добавление поддержки интерфейса I2C	51
9	Лабораторная №9. Работа с логическим анализатором	59
10	Лабораторная №10. Основы программирования микроконтроллеров Arduino	68
11	Лабораторная №11. Интеграция Arduino и Lichee RV Dock по SPI с датчиком BME280	78
12	Лабораторная №12. Протокол MQTT	98
13	Лабораторная №13. Итоговое проектное задание по вариантам	113

1 Лабораторная №1. Базовые навыки работы с одноплатным компьютером

1.1 Цель работы

Научиться подключаться к одноплатному компьютеру через последовательный порт, входить в установленную на нём систему Linux и выполнять базовые операции в консоли.

1.2 Оборудование и исходные данные

- одноплатный компьютер Lichee RV Dock;
- microSD-карта с подготовленным образом ALT Linux;
- USB-UART-преобразователь;
- три соединительных провода;
- кабель USB Type-C и источник питания для платы;
- компьютер с Linux и установленной утилитой `tio`.

Внимание

В этой лабораторной используйте Lichee RV Dock и выданную преподавателем карту памяти. Не заменяйте плату или образ без согласования: расположение контактов и учётные данные могут отличаться.

1.3 Ключевые понятия

Определение

Одноплатный компьютер — компьютер, основные компоненты которого, включая процессор, оперативную память и интерфейсы ввода-вывода, размещены на одной печатной плате.

Одноплатные компьютеры имеют компактные размеры и низкое энергопотребление, поэтому широко применяются во встраиваемых системах, устройствах интернета вещей и учебных стендах. В лаборатории доступны Lichee RV Dock и MangoPi MQ Pro. Обе платы построены на базе системы на кристалле Allwinner D1 и имеют схожую периферию.

Определение

Система на кристалле (System-on-Chip, SoC) — микросхема, объединяющая процессорные ядра, контроллеры памяти и периферийные интерфейсы.

1.3.1 Архитектура набора команд

Упрощённо, архитектура — это набор правил и принципов, по которым процессор взаимодействует с программами и остальной системой. Важнейшая часть этих правил — **ISA (Instruction Set Architecture)**.

Определение

Архитектура набора команд (Instruction Set Architecture, ISA) — соглашение между программным обеспечением и процессором, которое определяет доступные машинные команды, регистры, типы данных и правила их выполнения.

ISA можно представить как язык, на котором программа «разговаривает» с процессором. Она определяет, какие инструкции умеет выполнять процессор, какие у него есть регистры, как устроено обращение к памяти и какие операции доступны программе.

ISA часто делят на два исторических подхода: **CISC и RISC**.

CISC (Complex Instruction Set Computer): этот подход представлен привычной архитектурой x86-64, на которой работает большинство ПК и серверов. Для неё характерен большой и сложный набор инструкций, в том числе инструкций переменной длины. Это усложняет декодирование команд, но позволяет сохранять совместимость с огромным количеством существующего ПО.

RISC (Reduced Instruction Set Computer): к этому подходу относятся Arm и RISC-V. Идея заключается в более регулярном наборе инструкций и упрощении их декодирования. Но современный RISC-процессор вовсе не обязательно прост внутри: его внутренняя реализация может быть сложной.

Ключевое различие Arm и RISC-V в том, что архитектура Arm является проприетарной и её реализации лицензируются, а у **RISC-V открытая ISA**, которую можно использовать без платы за саму спецификацию. На её основе можно создать как открытое, так и закрытое процессорное ядро.

Именно Arm сейчас является наиболее распространённой архитектурой среди SBC, а RISC-V постепенно набирает популярность и появляется во всё большем количестве устройств.

Определение

RISC-V — открытая модульная архитектура набора команд: базовые целочисленные команды дополняются расширениями, например для умножения, атомарных операций, чисел с плавающей точкой и векторных вычислений.

Определение

Allwinner D1 (sun20iw1p1) — система на кристалле с одним 64-разрядным ядром XuanTie C906 архитектуры RISC-V. Именно на работе с одноплатниками на базе этой системы будет основан данный курс.

1.4 Подготовка платы

Для запуска операционной системы плате нужен загрузочный носитель с записанным образом. В этой работе microSD-карта и образ ALT Linux предоставляются в готовом виде. Самостоятельная подготовка образа рассматривается в следующих лабораторных.

Основная работа с Lichee RV Dock выполняется через консоль. Графический режим в этой лабораторной не требуется.

Определение

UART (Universal Asynchronous Receiver-Transmitter) — последовательный интерфейс, передающий данные по отдельным линиям приёма RX и передачи TX без общей тактовой линии.

Определение

GPIO (General-Purpose Input/Output) — программно управляемые контакты общего назначения, часть которых может переключаться на выполнение функций UART, SPI, I2C и других аппаратных интерфейсов.

1.4.1 Подключение USB-UART-преобразователя

Контакты TX и RX у Lichee RV Dock подписаны на обратной стороне платы.

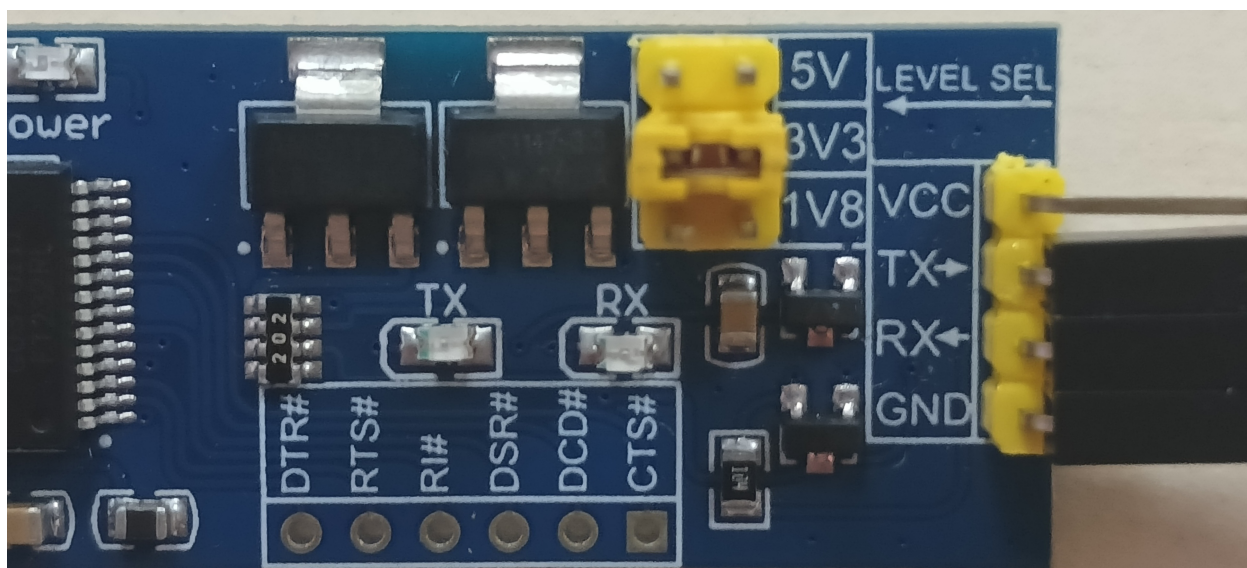


Рис. 1.1 – Джампер выбора напряжения

Опасность повреждения

Перед подключением установите джампер USB-UART-преобразователя в положение 3,3 В. Подача более высокого напряжения на сигнальные линии может повредить плату.

Соедините контакты в следующем порядке:

Lichee RV Dock	USB-UART-преобразователь
GND	GND
TX	RX
RX	TX

Линии TX и RX соединяются перекрёстно: передатчик одного устройства подключается к приёмнику другого.

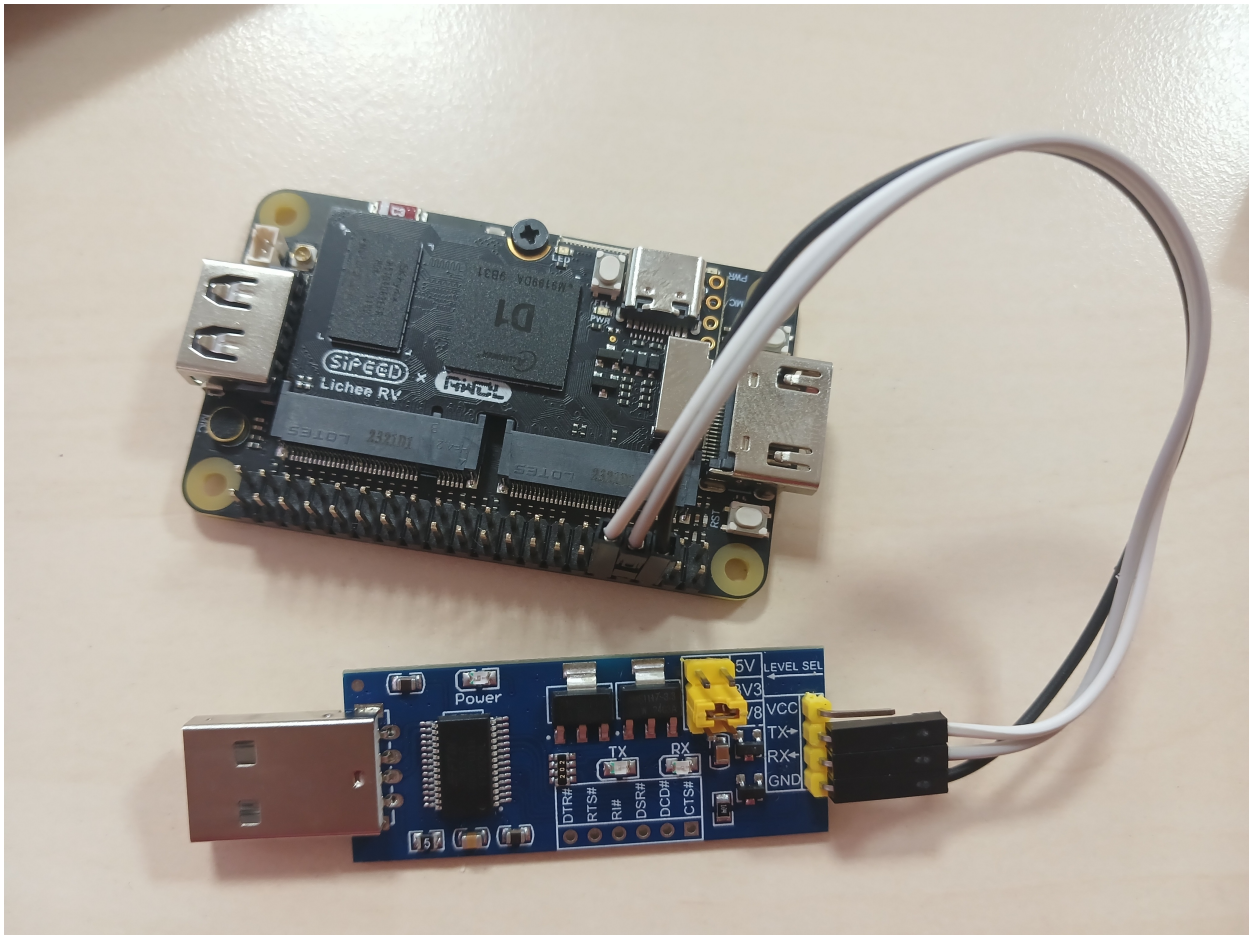


Рис. 1.2 - Подключение Lichee RV Dock к USB-UART-преобразователю

1.5 Порядок выполнения

1.5.1 Определение файла устройства

Подключите USB-UART-преобразователь к компьютеру. Для стационарного компьютера можно использовать USB-удлинитель из студенческого комплекта.

Выполните команду:

```
sudo dmesg | grep -i tty
```

Найдите в выводе имя нового последовательного устройства, например /dev/ttyUSB0.

```
[root@taranev ~]# dmesg | grep -i "tty"
[ 0.017205] ACPI: SSDT 0x000000008E82D000 002357 (v02 LENOVO TbTypeC 00000000 INTL 20200717)
[ 0.157096] printk: legacy console [tty0] enabled
[ 5.476955] systemd[1]: Reached target getty.target - Login Prompts.
[ 27.772101] Bluetooth: RFCOMM TTY layer initialized
[ 763.515842] usb 3-3.1: FTDI USB Serial Device converter now attached to ttyUSB0
[ 765.287605] ftdi_sio ttyUSB0: FTDI USB Serial Device converter now disconnected from ttyUSB0
[ 765.566621] usb 3-3.1: FTDI USB Serial Device converter now attached to ttyUSB0
[ 2422.627797] ftdi_sio ttyUSB0: FTDI USB Serial Device converter now disconnected from ttyUSB0
[ 3108.986077] usb 3-3.4: FTDI USB Serial Device converter now attached to ttyUSB0
[root@taranev ~]#
```

Рис. 1.3 - Определение устройства /dev/ttyUSB0 по выводу dmesg

Проверка

Проверка подключения. Выполните команду до и после подключения преобразователя. Новая строка в выводе должна содержать имя созданного файла устройства `/dev/ttyUSBX`, где `X` — номер адаптера.

Проверьте владельца и группу файла устройства:

```
stat -c '%A %U %G %n' /dev/ttyUSBX
```

На рабочих станциях ALT Linux последовательные устройства обычно принадлежат группе `uucp`. Если `tio` сообщает `Permission denied`, добавьте свою учётную запись в указанную командой `stat` группу:

```
sudo usermod -aG uucp "$USER"
```

После этого завершите сеанс пользователя и войдите снова: членство в новой группе применяется к новому сеансу. Проверьте результат командой `id -nG`. Постоянный запуск `tio` через `sudo` не рекомендуется, поскольку терминальной программе не нужны административные права.

1.5.2 Подключение к последовательной консоли

Подставьте найденный номер устройства вместо `X` и запустите `tio` со скоростью 115200 бод:

```
tio -b 115200 /dev/ttyUSBX
```

Установите подготовленную microSD-карту в плату. После проверки соединений подключите кабель USB Type-C к разъёму питания Lichee RV Dock и включите источник питания. В терминале должны появиться сообщения загрузчика и ядра Linux, а затем приглашение ко входу.

Проверка

Ожидаемый результат. Во время запуска платы в окне `tio` появляется текстовый журнал загрузки. Если вывод отсутствует, проверьте общий GND, перекрёстное подключение TX/RX, имя устройства и скорость 115200 бод.

Для выданного учебного образа используются следующие данные для входа:

```
login: root
password: altlinux
```

1.5.3 Изучение системы

После входа выполните команды из следующих разделов и зафиксируйте значимые результаты в отчёте.

1.5.3.1 Процессор и нагрузка

```
lscpu
cat /proc/cpuinfo
top
```

`lscpu` показывает архитектуру и основные характеристики процессора, `/proc/cpuinfo` содержит сведения, предоставленные ядром для отдельных процессоров, а `top` отображает текущую нагрузку. Для выхода из `top` нажмите `q`.

1.5.3.2 Оперативная память

```
free -h
cat /proc/meminfo
```

1.5.3.3 Накопители и файловые системы

```
lsblk
lsblk -f
df -h
```

`lsblk -f` дополнительно показывает типы файловых систем, метки и точки монтирования. В используемом минимальном образе ALT Linux команда `fdisk` может отсутствовать, поэтому она не требуется для выполнения первой лабораторной.

1.5.3.4 Сетевые интерфейсы

```
ip address
```

1.5.3.5 Файлы и каталоги

```
mkdir my_folder
cd my_folder
touch my_file.txt
echo "Hello, World!" > my_file.txt
echo "Lichee" >> my_file.txt
cat my_file.txt
```

Пример. Оператор `>` перезаписывает файл результатом команды, а оператор `>>` добавляет данные в его конец. Поэтому после выполнения примера `my_file.txt` содержит две строки.

1.5.3.6 Имя системы и время

```
hostnamectl
timedatectl
```

1.5.3.7 Текущий пользователь

```
whoami
id
pwd
```

Создание отдельной учётной записи и настройка удалённого входа выполняются в лабораторной №2. Команда `whoami` показывает имя текущего пользователя, `id` — его идентификаторы и группы, а `pwd` — текущий каталог.

1.6 Задание

1. Создайте учётную запись в домене лаборатории.
2. Проверьте возможность входа на лабораторный компьютер и использования `sudo`.
3. Подключите Lichee RV Dock к компьютеру через USB-UART-преобразователь.
4. Определите файл устройства преобразователя и подключитесь к консоли с помощью `tio`.
5. Войдите в ALT Linux с выданными учётными данными.
6. Получите сведения о процессоре, оперативной памяти, накопителях, файловых системах и сетевых интерфейсах.
7. Создайте каталог и файл, запишите в файл две строки и выведите его содержимое.
8. Определите имя системы и текущие настройки времени.
9. Определите текущего пользователя, его группы и текущий каталог.
10. Продемонстрируйте работающую последовательную консоль и результаты преподавателю.

1.7 Дополнительные задания

1.8 Контрольные вопросы

1. Чем открытая архитектура набора команд RISC-V отличается от x86-64?
2. Почему для первого подключения к одноплатному компьютеру используется UART, а не SSH?
3. Что такое GPIO и зачем при подключении периферии нужна схема расположения контактов?
4. Как определить, какой файл устройства в `/dev` соответствует USB-UART-преобразователю?
5. Что показывает команда `lscpu` и как интерпретировать поле Architecture на плате с RISC-V?

1.9 Требования к отчёту

Отчёт должен содержать:

- цель работы;
- схему или фотографию подключения USB-UART-преобразователя;
- имя обнаруженного файла устройства;
- основные сведения о процессоре, памяти, накопителях и сети;
- команды работы с файлами и полученный результат;
- ответы на контрольные вопросы;
- краткий вывод.

Подготовьте отчёт в согласованном с преподавателем формате и отправьте его до установленного срока.

2 Лабораторная №2. Работа с платой по SSH

2.1 Цель работы

Освоить удалённое взаимодействие с одноплатным компьютером: настроить сетевое подключение и SSH, подключиться к плате и передать файлы между системами.

2.2 Оборудование и предварительные требования

- одноплатный компьютер Lichee RV Dock с операционной системой Linux;
- рабочий компьютер с Linux и клиентами ssh и scp;
- доступная сеть Wi-Fi или Ethernet;
- последовательная консоль, настроенная в лабораторной №1;
- учётная запись с правами на настройку сети и службы SSH.

2.3 Ключевые понятия

Определение
SSH (Secure Shell) — криптографический сетевой протокол для защищённого удалённого входа, выполнения команд и туннелирования соединений.

Определение
SCP (Secure Copy Protocol) — средство защищённого копирования файлов между системами поверх SSH.

В отличие от UART, SSH работает через сеть и предоставляет удалённую командную оболочку после загрузки операционной системы и запуска сетевых служб. Последовательная консоль остаётся полезной для первичной настройки и диагностики проблем с сетью.

2.4 Порядок выполнения

2.4.1 Настройка сети

На плате определите имя сетевого интерфейса и просмотрите доступные сети Wi-Fi:


```
nmcli device status
nmcli device wifi list
```

Подключитесь к выбранной сети:

```
nmcli device wifi connect "название_сети" password "пароль_сети"
```

Проверьте состояние интерфейса, полученный адрес и маршрут по умолчанию:

```
ip -brief address
ip route
```

В строке `default via` найдите адрес шлюза и подставьте его в следующую команду:

```
ping -c 4 IP-адрес_шлюза
```

Затем отдельно проверьте разрешение DNS-имени и доступность внешнего узла:

```
getent hosts ya.ru
ping -c 4 ya.ru
```

Проверка

Проверка подключения. Проверки выполняются по порядку: состояние интерфейса, IP-адрес, маршрут, шлюз, DNS и внешний узел. Если IP-адрес и шлюз доступны, но `getent` не возвращает адрес `ya.ru`, неисправность относится к разрешению имён. Некоторые сети блокируют ICMP, поэтому не делайте вывод об отсутствии интернета только по одному неуспешному `ping`.

2.4.2 Настройка сервера SSH

Конфигурация сервера OpenSSH в ALT Linux находится в файле:

```
/etc/openssh/sshd_config
```

Убедитесь, что сервер SSH установлен и запущен. При необходимости измените параметр аутентификации по паролю:

```
PasswordAuthentication yes
```

После изменения конфигурации перезапустите службу:

```
sshd -t
systemctl restart sshd
systemctl status sshd --no-pager
ss -lntp 'sport = :22'
```

Команда `sshd -t` проверяет синтаксис конфигурации и при успехе не выводит текст. Не перезапускайте службу, если проверка завершилась с ошибкой. Команда `ss` должна показать, что `sshd` слушает TCP-порт 22.

Внимание

Аутентификация по паролю повышает риск подбора учётных данных. Используйте её только в изолированной учебной сети, задайте стойкий пароль и после освоения подключения перейдите на ключи SSH.

Внимание

Не включайте `PermitRootLogin yes` без явного указания преподавателя. Удалённый вход суперпользователя увеличивает последствия компрометации пароля или ключа.

2.4.3 Создание отдельного пользователя

После первичной настройки создайте согласованного с преподавателем пользователя, задайте ему пароль и выполняйте дальнейшую работу по SSH через эту учётную запись, а не через `root`:

```
useradd -m student  
passwd student
```

Вместо `student` используйте выбранное имя пользователя.

Проверьте, что домашний каталог создан и принадлежит новому пользователю:

```
getent passwd student  
ls -ld /home/student
```

В командах также замените `student` на фактическое имя.

2.4.4 Подключение по SSH

До первого подключения получите отпечаток ключа SSH-сервера через доверенную UART-консоль:

```
ssh-keygen -lf /etc/openssh/ssh_host_ed25519_key.pub
```

На рабочем компьютере выполните команду, подставив имя созданного пользователя и IP-адрес платы:

```
ssh пользователь@IP-адрес_платы
```

При первом подключении внимательно сверьте предложенный отпечаток ключа сервера с результатом, полученным через UART, и только затем подтвердите его сохранение.

Проверка

Ожидаемый результат. После ввода пароля появляется командная строка платы. Команды `hostnamectl` и `whoami` должны показать имя удалённой системы и созданного пользователя.

2.4.5 Настройка входа по ключу

На рабочем компьютере проверьте, существует ли открытая часть стандартного ключа:

```
test -f ~/.ssh/id_ed25519.pub
```

Если команда завершилась с ненулевым кодом, создайте ключ Ed25519. Задайте парольную фразу для защиты закрытого ключа:

```
ssh-keygen -t ed25519
```

Скопируйте на плату только открытый ключ:

```
ssh-copy-id -i ~/.ssh/id_ed25519.pub пользователь@IP-адрес_платы
```

Во время выполнения `ssh-copy-id` в последний раз потребуется пароль пользователя на плате. Закрытый файл `~/.ssh/id_ed25519` никому не передавайте.

Проверьте вход без запроса пароля пользователя платы:

```
ssh -o BatchMode=yes пользователь@IP-адрес_платы 'whoami; pwd'
```

Опция `BatchMode=yes` запрещает переход к парольной аутентификации. Успешный вывод имени пользователя и его домашнего каталога подтверждает, что применяется ключ. Парольная фраза локального закрытого ключа при этом может запрашиваться или обслуживаться `ssh-agent`.

Внимание

Не отключайте вход по паролю, пока не проверили ключ в отдельном терминале и не сохранили работающую UART-сессию. На общем учебном образе изменение политики аутентификации выполняется только по указанию преподавателя.

2.4.6 Передача файлов

Передайте файл с рабочего компьютера на плату:

```
scp файл пользователь@IP-адрес_платы:путь/на/плате/
```

Для копирования файла с платы на рабочий компьютер поменяйте местами источник и назначение:

```
scp пользователь@IP-адрес_платы:путь/к/файлу путь/на/рабочем/компьютере/
```

Проверка

Проверка передачи. Сравните размер и контрольную сумму исходного и полученного файлов командой `sha256sum`. Значения должны совпасть.

2.5 Задание

1. Запустите Lichee RV Dock с операционной системой Linux.
2. Настройте сетевое подключение платы по Wi-Fi или Ethernet.
3. Проверьте сервер SSH на плате и при необходимости настройте его.
4. Создайте отдельного пользователя с собственным паролем.
5. Подключитесь к плате по SSH с рабочего компьютера под созданной учётной записью.
6. Создайте или выберите ключ Ed25519, установите его командой `ssh-keygen -t ed25519 -C` и проверьте вход с `BatchMode=yes`.
7. Передайте файл между системами с помощью `scp` и проверьте его целостность.
8. Пропридемонстрируйте преподавателю сетевое подключение, сеанс SSH, вход по ключу и передачу файла.

2.6 Дополнительные задания

2.7 Контрольные вопросы

1. В чём принципиальное отличие SSH от UART-соединения?
2. Как безопасно проверить изменение `/etc/openssh/sshd_config` и убедиться, что сервер принимает соединения?
3. Для чего нужна утилита `scp` и какой транспорт она использует?
4. Как настроить Wi-Fi-соединение на одноплатном компьютере без графического интерфейса?
5. Почему рекомендуется отключать аутентификацию по паролю и переходить на ключи SSH?
6. Что делает команда `ssh-keygen -t ed25519 -C` и какие файлы ключа допустимо передавать на сервер?

2.8 Требования к отчёту

Отчёт должен содержать:

- цель работы;
- имя сетевого интерфейса и полученный платой IP-адрес;
- команды настройки и проверки сети;
- изменённые параметры сервера SSH;

- подтверждение входа под отдельным пользователем;
- отпечаток ключа сервера и подтверждение входа по ключу с BatchMode=yes;
- команды передачи файла и результаты проверки его целостности;
- ответы на контрольные вопросы;
- краткий вывод.

Подготовьте отчёт в согласованном с преподавателем формате и отправьте его до установленного срока.

3 Лабораторная №3.

Использование кросс-компилятора

3.1 Цель работы

Научиться собирать на компьютере с архитектурой x86-64 программы для RISC-V, проверять их с помощью QEMU и запускать на целевой плате.

3.2 Оборудование и предварительные требования

- рабочий компьютер с ALT Linux на архитектуре x86-64;
- одноплатный компьютер Lichee RV Dock с архитектурой RISC-V;
- настроенное подключение к плате и средство передачи файлов;
- права суперпользователя на рабочем компьютере;
- редактор исходного кода.

3.3 Ключевые понятия

Определение

Кросс-компиляция — сборка программы на одной аппаратной или программной платформе для выполнения на другой целевой платформе.

Обычный цикл разработки для целевой архитектуры включает написание кода, сборку кросс-компилятором, перенос программы на плату или запуск в эмуляторе, а затем проверку и отладку.

Определение

Тулчейн (toolchain) — набор компиляторов, компоновщиков, библиотек и вспомогательных утилит для разработки под определённую платформу.

Основной компонент тулчейна в этой работе выполняется на x86-64, но создаёт программы для RISC-V. В ALT Linux нужные инструменты доступны в пакете `gcc-riscv64-linux-gnu` из репозитория для разработчиков.

Определение

QEMU user-mode — режим эмуляции, в котором программа для гостевой архитектуры выполняется на ядре хостовой Linux с трансляцией инструкций и системных вызовов.

Механизм `binfmt_misc` позволяет ядру Linux распознать исполняемый файл RISC-V и автоматически передать его зарегистрированному интерпретатору QEMU. Это даёт возможность проверить консольную программу на рабочем компьютере до переноса на плату.

3.4 Порядок выполнения

3.4.1 Настройка репозитория пакетов

Просмотрите текущую конфигурацию репозитория:

```
apt-repo
```

Необходимые инструменты доступны в репозитории Sisyphus. Если он ещё не настроен, сохраните текущий вывод `apt-repo`, затем переключите источники пакетов:

Опасность повреждения

Команда `apt-repo rm all` удаляет все подключённые репозитории из конфигурации APT. Выполняйте её только на предназначенной для лабораторной системе и заранее сохраните текущую конфигурацию, чтобы восстановить используемую ветку.

```
apt-repo rm all
apt-repo set sisyphus
apt-get update
```

3.4.2 Установка инструментов

Установите кросс-компилятор и поддержку запуска программ RISC-V через QEMU:

```
apt-get install gcc-riscv64-linux-gnu qemu-user-static-binfmt-riscv
```

Для написания программы используйте любой доступный редактор или IDE. В консольной среде можно работать с `vim`, `neovim`, `nano` или `emacs`.

Проверьте доступность компилятора:

```
riscv64-linux-gnu-gcc --version
```

3.4.3 Подготовка программы

Напишите на C приложение, которое принимает числовой аргумент из командной строки и с помощью математической библиотеки вычисляет пять значений выбранных элементарных функций. Предусмотрите проверку количества аргументов и корректности введенного значения.

Имя исходного файла в следующих командах замените на фактическое.

3.4.4 Кросс-компиляция

Соберите статически скомпонованный исполняемый файл и подключите математическую библиотеку:

```
riscv64-linux-gnu-gcc -static -o math-app math-app.c -lm
```

Внимание

Для показанного ниже запуска через QEMU нужна статическая компоновка. Без `-static` программе потребуются загрузчик и динамические библиотеки RISC-V, которых по умолчанию нет на рабочем компьютере.

Проверьте архитектуру полученного файла:

```
file math-app
```

Проверка

Ожидаемый результат. Утилита `file` должна определить исполняемый файл как 64-разрядный RISC-V ELF и указать статическую компоновку.

3.4.5 Запуск через QEMU

При настроенном `binfmt_misc` программу можно запустить как обычный исполняемый файл:

```
chmod +x math-app  
./math-app аргумент
```

Если автоматический запуск не настроен, вызовите эмулятор явно:

```
qemu-riscv64-static ./math-app аргумент
```

Проверка

Проверка запуска. Программа должна вывести пять вычисленных значений и одинаково обрабатывать корректный аргумент при автоматическом и явном запуске через QEMU.

На рисунке показан пример работы приложения, запущенного через QEMU user-mode:

```
[taranev@taranev work_repos]$ ./test_cli 2
Math calculations for x = 2.0000:
=====
sqrt(x)      = 1.414214
x^2          = 4.000000
x^3          = 8.000000
sin(x)       = 0.909297
cos(x)       = -0.416147
```

Рис. 3.1 – Результат запуска приложения для RISC-V через QEMU user-static

3.4.6 Запуск на целевой плате

Передайте исполняемый файл на Lichee RV Dock с помощью scp, выдайте право на выполнение и запустите его с тем же аргументом. Сравните результат с запуском через QEMU.

3.5 Задание

1. Убедитесь в наличии или установите кросс-компилятор и подходящий редактор на рабочем компьютере.
2. Напишите приложение на C, которое принимает аргумент через интерфейс командной строки и с помощью библиотеки `libm` вычисляет пять значений выбранных элементарных математических функций.
3. Соберите приложение кросс-компилятором для RISC-V со статической компоновкой.
4. Проверьте архитектуру исполняемого файла.
5. Запустите программу через QEMU user-mode.
6. Передайте исполняемый файл на плату и выполните его.
7. Сравните результаты запуска на рабочем компьютере и плате.
8. Продемонстрируйте преподавателю исходный код, сборку и оба варианта запуска.

3.6 Контрольные вопросы

1. Что такое кросс-компиляция и в каких случаях она необходима?
2. Зачем нужен флаг `-static` при показанном запуске программы для RISC-V через QEMU?
3. Какую роль выполняют QEMU user-mode и `binfmt_misc`?

4. Чем различаются gcc и riscv64-linux-gnu-gcc?

3.7 Требования к отчёту

Отчёт должен содержать:

- цель работы;
- сведения об установленном кросс-компиляторе;
- исходный код приложения;
- команды и результаты кросс-компиляции;
- результат определения архитектуры файла утилитой file;
- результаты запуска через QEMU и на плате;
- сравнение полученных результатов;
- ответы на контрольные вопросы;
- краткий вывод.

Подготовьте отчёт в согласованном с преподавателем формате и отправьте его до установленного срока.

4 Лабораторная №4. Создание основных компонентов образа: ядро Linux

4.1 Цель работы

Изучить назначение ядра Linux, научиться изменять его конфигурацию и собрать ядро для Lichee RV Dock с заданными параметрами.

4.2 Оборудование и предварительные требования

- рабочий компьютер с ALT Linux;
- не менее 20 Гбайт свободного места для исходного кода и результатов сборки;
- доступ к репозиторию Sisyphus;
- установленный кросс-компилятор RISC-V из лабораторной №3;
- права суперпользователя для установки пакетов.

4.3 Ключевые понятия

Определение

Ядро Linux — центральная часть операционной системы, которая управляет аппаратными ресурсами и предоставляет системные интерфейсы пользовательским программам.

Само ядро не является полноценной операционной системой. Рабочую среду образуют ядро, системные библиотеки, утилиты и пользовательские приложения.

Основные задачи ядра:

- управление процессором, памятью и устройствами;
- обеспечение безопасности и разграничение прав;
- реализация сетевых протоколов;
- управление файловыми системами и операциями ввода-вывода.

Определение

Конфигурация ядра — набор параметров в файле `.config`, который определяет включаемые подсистемы, драйверы и режимы их сборки.

Сборка ядра из исходного кода включает компиляцию и компоновку ядра и выбранных модулей на основе этой конфигурации.

Определение

PREEMPT_RT — режим вытеснения ядра, уменьшающий задержки обработки событий и повышающий предсказуемость времени реакции системы.

В современных версиях Linux поддержка PREEMPT_RT входит в основное дерево исходного кода. Это параметр конфигурации ядра, а не отдельный загружаемый модуль.

4.4 Порядок выполнения

4.4.1 Установка зависимостей

Установите пакеты, необходимые для получения исходного кода, настройки и сборки ядра:

```
apt-get install wget build-essential bc flex bison ncurses-devel openssl  
↪ libssl-devel dtc
```

Исходный код ядра версии 6.16 доступен в пакете `kernel-source-6.16` из репозитория Sisyphus.

Опасность повреждения

Команда `apt-repo rm all` удаляет все подключённые репозитории из конфигурации APT. Выполняйте её только на предназначенной для лабораторной системе и заранее сохраните текущую конфигурацию, чтобы восстановить используемую ветку.

```
apt-repo rm all  
apt-repo set sisyphus  
apt-get update  
apt-get install kernel-source-6.16
```

Архив исходного кода будет установлен в `/usr/src/kernel/sources/kernel-source-6.16.tar`. Распакуйте его в отдельный каталог и перейдите в корень полученного дерева исходного кода:

```
mkdir -p ~/kernel-build  
tar -xf /usr/src/kernel/sources/kernel-source-6.16.tar -C ~/kernel-build
```

Определите имя созданного архивом каталога и перейдите в него. Все дальнейшие команды выполняются из корня этого дерева исходного кода.

Внимание

Все последующие команды make выполняйте из корня дерева исходного кода ядра. В другом каталоге система сборки не найдёт необходимые правила и конфигурацию.

4.4.2 Создание базовой конфигурации

Создайте стандартную конфигурацию для архитектуры RISC-V:

```
make ARCH=riscv CROSS_COMPILE=riscv64-linux-gnu- defconfig  
cp .config .config.defconfig
```

Параметры команды имеют следующее назначение:

- ARCH=riscv выбирает целевую архитектуру и соответствующий каталог arch/riscv/;
- CROSS_COMPILE=riscv64-linux-gnu- задаёт префикс инструментов кросс-компиляции;
- defconfig создаёт базовый файл .config из arch/riscv/configs/defconfig.

Базовая конфигурация включает основные возможности RISC-V, но не гарантирует наличие всех параметров, необходимых конкретной плате.

4.4.3 Изменение конфигурации

Откройте интерактивный интерфейс настройки для выбранной архитектуры:

```
make ARCH=riscv CROSS_COMPILE=riscv64-linux-gnu- menuconfig
```

В menuconfig используйте клавиши со стрелками для навигации, Enter для перехода, Y для включения параметра в ядро, N для выключения и M для сборки в виде модуля. Клавиша / открывает поиск параметров и показывает их расположение и зависимости.

Найдите параметр PREEMPT_RT через поиск:

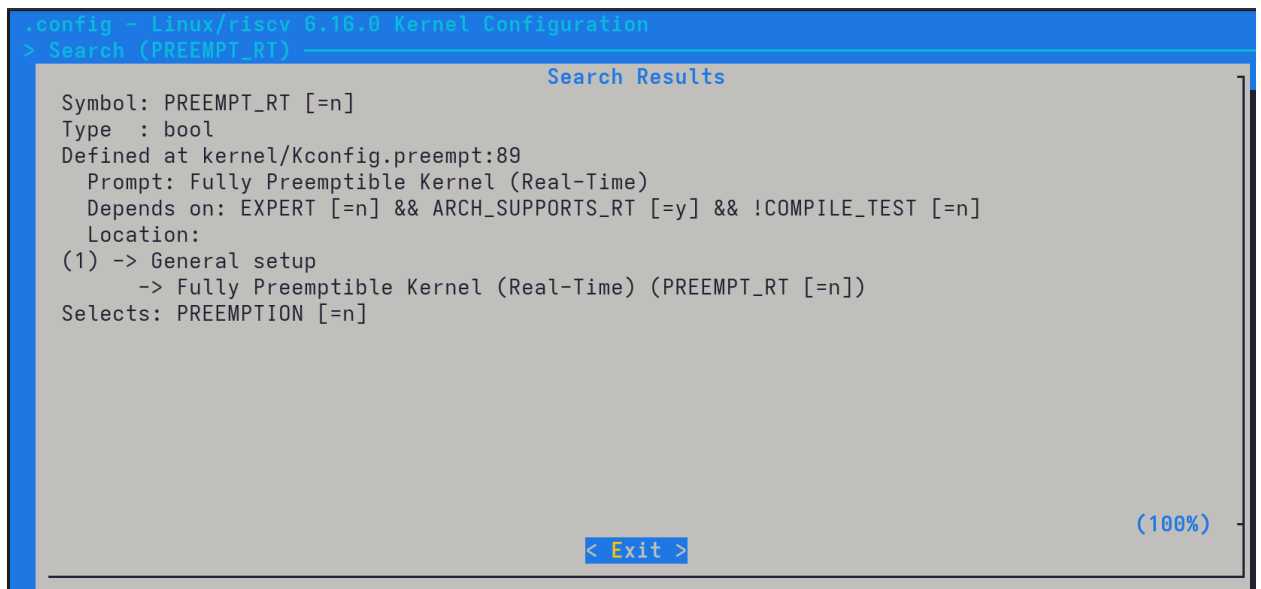


Рис. 4.1 – Поиск параметра PREEMPT_RT в menuconfig

Результат поиска показывает путь к параметру, его текущее значение и зависимости. Число слева от найденного пути позволяет быстро перейти к доступной части дерева настроек.

Для показанной конфигурации включите EXPERT, убедитесь, что ARCH_SUPPORTS_RT доступен, а COMPILE_TEST отключён. Затем выберите режим полного вытеснения реального времени PREEMPT_RT.

Кроме того, выберите поддержку семейства SoC Allwinner D1 и Wi-Fi-модуля RTW88_8723DS, как указано в задании. Сохраните изменения в .config.

Проверка

Проверка конфигурации. После выхода из menuconfig найдите выбранные параметры в .config. Включённые в ядро параметры имеют вид CONFIG_ИМЯ=y, а модульные — CONFIG_ИМЯ=m.

Зафиксируйте отличия от базовой конфигурации:

```
diff -u .config.defconfig .config > kernel-config.diff
```

Команда diff возвращает ненулевой статус, если различия найдены; в данном случае это ожидаемо.

4.4.4 Сборка ядра

Запустите параллельную сборку:

```
make ARCH=riscv CROSS_COMPILE=riscv64-linux-gnu- -j$(nproc)"
```

Проверка

Ожидаемый результат. После успешной сборки файл ядра находится в `arch/riscv/boot/Image`, его сжатый вариант — в `arch/riscv/boot/Image.gz`, а DTB платы — в `arch/riscv/boot/dts/allwinner/sun20i-d1-lichee-rv-dock.dtb`.

Сохраните для следующих лабораторных работ:

- `arch/riscv/boot/Image`;
- файл `.config` из корня дерева сборки;
- `arch/riscv/boot/dts/allwinner/sun20i-d1-lichee-rv-dock.dtb`;
- файл `kernel-config.diff`.

Внимание

Не удаляйте дерево сборки и перечисленные результаты после завершения работы. Ядро, конфигурация и DTB потребуются при подготовке загрузочного носителя.

4.5 Задание

1. Установите или проверьте зависимости для сборки ядра.
2. Получите и распакуйте исходный код ядра.
3. Создайте стандартную конфигурацию для архитектуры RISC-V и сохраните её копию.
4. Включите режим `PREEMPT_RT` по примеру из подготовительного материала.
5. В разделе выбора SoC включите поддержку семейства Allwinner D1.
6. Включите поддержку Wi-Fi-модуля `RTW88_8723DS`.
7. Добавьте один согласованный с преподавателем параметр, не дублирующий выбранные параметры той же подгруппы.
8. Зафиксируйте различия между изменённой и стандартной конфигурациями.
9. Соберите ядро с полученной конфигурацией.
10. Сохраните конфигурацию, ядро, DTB и файл различий для дальнейших лабораторных работ.
11. Продемонстрируйте преподавателю конфигурацию и результаты успешной сборки.

4.6 Контрольные вопросы

1. Какие основные функции выполняет ядро Linux в системе?
2. Для чего используется `make menuconfig` при сборке ядра?

3. Почему при сборке ядра необходимо указывать ARCH=riscv и CROSS_COMPILE=riscv64-linux-gnu-?
4. Что такое PREEMPT_RT и в каких сценариях он необходим?
5. Как выбрать поддержку целевого SoC Allwinner D1 в menuconfig?

4.7 Требования к отчёту

Отчёт должен содержать:

- цель работы;
- сведения о версии исходного кода и установленных инструментах;
- команды создания и изменения конфигурации;
- перечень включённых параметров и обоснование дополнительного параметра;
- файл или фрагмент различий конфигураций;
- команду и результат сборки;
- пути и сведения о полученных Image и DTB;
- ответы на контрольные вопросы;
- краткий вывод.

Подготовьте отчёт в согласованном с преподавателем формате и отправьте его до установленного срока.

5 Лабораторная №5. Создание основных компонентов образа: дерево устройств, корневая файловая система и загрузчик

5.1 Цель работы

Изучить назначение дерева устройств, OpenSBI, U-Boot, конфигурации Extlinux и корневой файловой системы, а затем подготовить эти компоненты для последующей записи образа на носитель.

5.2 Оборудование и предварительные требования

- рабочий компьютер с ALT Linux и доступом в интернет;
- установленный кросс-тулчейн RISC-V;
- утилиты `git`, `make` и зависимости для сборки U-Boot и OpenSBI;
- ядро, `.config` и DTB, полученные в лабораторной №4;
- не менее 10 Гбайт свободного места для исходного кода и загруженных компонентов.

5.3 Ключевые понятия

5.3.1 Дерево устройств

Определение

Device Tree (DT) — иерархическая структура данных, которая описывает оборудование платформы для ядра операционной системы.

Device Tree позволяет одному ядру работать на разных платах без перекомпиляции при наличии подходящего описания оборудования. Дополнительные сведения приведены в статье на altlinux.org.

Определение

DTS (Device Tree Source) — человекочитаемый исходный файл описания дерева устройств с расширением `.dts` или включаемый фрагмент с расширением `.dtsi`.

Определение

DTB (Device Tree Blob) — бинарное представление дерева устройств, полученное компиляцией DTS и передаваемое ядру при загрузке.

Основным источником DTS служат исходный код Linux и репозитории разработчиков оборудования. В дереве ядра описания обычно находятся в каталоге `arch/архитектура/boot/dts/производитель/`.

Для Lichee RV Dock исходный файл расположен по пути:

```
arch/riscv/boot/dts/allwinner/sun20i-d1-lichee-rv-dock.dts
```

После сборки ядра соответствующий DTB находится здесь:

```
arch/riscv/boot/dts/allwinner/sun20i-d1-lichee-rv-dock.dtb
```

Проверка

Практический ориентир. Если готового DTS для платы нет, отправной точкой может служить описание наиболее похожего устройства. Его адаптируют по документации SoC, электрической схеме и распиновке платы.

5.3.2 Цепочка загрузки

Определение

Загрузчик (bootloader) — программа, которая после включения системы подготавливает оборудование и загружает следующий программный компонент или ядро операционной системы.

Для одноплатных компьютеров широко применяется Das U-Boot, обычно называемый U-Boot.

Определение

SPL (Secondary Program Loader) — компактная первая стадия U-Boot, которая выполняет раннюю инициализацию, в том числе настраивает оперативную память для загрузки следующих компонентов.

Определение

OpenSBI — реализация интерфейса Supervisor Binary Interface для RISC-V, которая работает в привилегированном M-mode и предоставляет среде S-mode стандартные низкоуровневые сервисы.

Для Lichee RV Dock цепочку загрузки можно представить так:

1. После подачи питания встроенный BootROM процессора ищет загрузочный код на поддерживаемом носителе и считывает SPL из предусмотренной для него области.

2. SPL выполняет раннюю инициализацию, включая настройку DRAM, и загружает последующие компоненты комбинированного образа U-Boot.
3. OpenSBI запускается в M-mode, остаётся резидентным и передаёт управление U-Boot proper в S-mode.
4. U-Boot proper инициализирует необходимую периферию, читает конфигурацию загрузки и загружает ядро Linux и DTB.
5. U-Boot передаёт ядру управление, а ядро обращается к сервисам OpenSBI через вызовы SBI, например при перезагрузке системы.

BootROM не читает `extlinux.conf` и не ищет ядро в файловой системе: это выполняется на более позднем этапе U-Boot.

5.3.3 Корневая файловая система

Определение

Rootfs (root filesystem) — корневая файловая система, содержащая пользовательское пространство Linux: программы, библиотеки, конфигурацию и каталоги, необходимые после запуска ядра.

После инициализации ядро монтирует rootfs и запускает первый процесс, обычно систему инициализации. В rootfs находятся:

- исполняемые программы в `/bin`, `/sbin` и `/usr/bin`;
- библиотеки в `/lib` и `/usr/lib`;
- конфигурационные файлы в `/etc`;
- файлы устройств в `/dev`;
- временные данные и точки монтирования;
- домашние каталоги пользователей в `/home`.

Без rootfs ядро не сможет запустить пользовательские приложения и системные службы. Подходящий архив можно выбрать на странице регулярных сборок ALT Linux для riscv64. Для этой работы предлагается сборка с Xfce, если преподаватель не указал другой вариант.

5.3.4 Конфигурация Extlinux

Определение

extlinux.conf — текстовый файл конфигурации, который сообщает U-Boot пути к ядру и DTB, а также параметры командной строки ядра.

Простейшая конфигурация для подготовленных компонентов выглядит так:

```
label Linux
kernel /Image
```

```
fdt /sun20i-d1-lichee-rv-dock.dtb
append root=/dev/mmcblk0p2 rootwait console=ttyS0,115200
```

Параметры имеют следующее назначение:

- `kernel /Image` задаёт путь к ядру на загрузочном разделе;
- `fdt /sun20i-d1-lichee-rv-dock.dtb` задаёт путь к DTB;
- `root=/dev/mmcblk0p2` указывает раздел с корневой файловой системой;
- `rootwait` предписывает ядру дожидаться появления корневого устройства;
- `console=ttyS0,115200` включает вывод консоли ядра через UART со скоростью 115200 бод.

Внимание

Значения `kernel`, `fdt` и `root` должны соответствовать фактическим путям и разметке будущего носителя. Ошибка в любом из них остановит загрузку системы.

5.4 Порядок выполнения

5.4.1 Получение DTB

Возьмите DTB Lichee RV Dock из результатов лабораторной №4 и убедитесь, что имя платы определено верно:

```
file arch/riscv/boot/dts/allwinner/sun20i-d1-lichee-rv-dock.dtb
```

Проверка

Ожидаемый результат. Утилита `file` должна определить файл как Flattened Device Tree blob, а не как пустой файл или обычный текст.

5.4.2 Сборка OpenSBI

На сайте сообщества `linux-sunxi` приведён вариант сборки компонентов для Lichee RV. Получите исходный код OpenSBI и соберите динамическую прошивку:

```
git clone https://github.com/riscv-software-src/opensbi
cd opensbi
make CROSS_COMPILE=riscv64-linux-gnu- PLATFORM=generic FW_PIC=y
cd ..
```

Результат сборки, используемый U-Boot, находится по пути:

```
opensbi/build/platform/generic/firmware/fw_dynamic.bin
```

5.4.3 Сборка U-Boot

Получите ветку U-Boot для Allwinner D1, создайте конфигурацию Lichee RV Dock и запустите сборку, указав путь к OpenSBI:

```
git clone https://github.com/smaeul/u-boot -b d1-wip
cd u-boot
make CROSS_COMPILE=riscv64-linux-gnu- lichee_rv_dock_defconfig
make CROSS_COMPILE=riscv64-linux-gnu-
  ↪ OPENSBI=../opensbi/build/platform/generic/firmware/fw_dynamic.bin
cd ..
```

После успешной сборки в каталоге u-boot появится комбинированный образ u-boot-sunxi-with-spl.bin.

Проверка

Проверка сборки. Убедитесь, что u-boot-sunxi-with-spl.bin существует и имеет ненулевой размер. Сохраните журнал сборки для отчёта.

5.4.4 Получение готового U-Boot из репозитория

Вместо самостоятельной сборки можно использовать готовый U-Boot с OpenSBI из репозитория ALT Linux. Для сравнения со самостоятельно собранным вариантом получите пакет u-boot-sunxi-riscv.

Если репозиторий sisyphus_riscv64 не настроен, добавьте следующие строки в /etc/apt/sources.list:

Опасность повреждения

Полная замена /etc/apt/sources.list уничтожит текущий список источников пакетов. Сначала сохраните резервную копию файла и после получения пакета восстановите исходную конфигурацию.

```
rpm [sisyphus-riscv64]
  ↪ http://ftp.altlinux.org/pub/distributions/ALTlinux/ports/riscv64
  ↪ Sisyphus/riscv64 classic
rpm [sisyphus-riscv64]
  ↪ http://ftp.altlinux.org/pub/distributions/ALTlinux/ports/riscv64
  ↪ Sisyphus/noarch classic
```

Обновите индекс и установите пакет:

```
apt-get update
apt-get install u-boot-sunxi-riscv
```

Файлы загрузчиков для устройств на базе Allwinner D1 будут установлены в /usr/share/u-boot. Найдите вариант для Lichee RV Dock и сравните его размер и контрольную сумму с самостоятельно собранным образом:

```
sha256sum u-boot/u-boot-sunxi-with-spl.bin
```

Совпадение контрольных сумм не требуется: версии исходного кода и параметры сборки могут различаться.

5.4.5 Подготовка `extlinux.conf`

Создайте конфигурацию на основе приведённого примера. Укажите согласованную с преподавателем метку, по которой можно определить автора будущего образа, и проверьте пути к ядру, DTB и `rootfs`.

Проверка

Проверка конфигурации. В файле должны присутствовать директивы `label`, `kernel`, `fdt` и `append`; значения путей должны совпадать с именами подготовленных файлов и будущей разметкой носителя.

5.4.6 Получение `rootfs`

Скачайте указанный преподавателем архив `rootfs` или предложенную регулярную сборку ALT Linux для `riscv64`. Не распаковывайте архив без необходимости на этом этапе, но проверьте его тип, размер и контрольную сумму.

Внимание

Сохраните ядро, DTB, загрузчик, `extlinux.conf` и архив `rootfs`. Все эти компоненты потребуются при создании образа в следующей лабораторной работе.

5.5 Задание

1. Получите DTB Lichee RV Dock из результатов сборки ядра и проверьте тип файла.
2. Соберите OpenSBI и U-Boot для Lichee RV Dock.
3. Получите аналогичный готовый загрузчик из репозитория Sisyphus-riscv64.
4. Сравните размеры и контрольные суммы самостоятельно собранного и пакетного загрузчиков.
5. Скачайте с указанного преподавателем ресурса архив `rootfs`.
6. Подготовьте `extlinux.conf`, указав согласованную метку автора будущего образа.
7. Сохраните ядро, DTB, оба варианта загрузчика, `extlinux.conf` и архив `rootfs`.
8. Продемонстрируйте преподавателю подготовленные компоненты.

5.6 Контрольные вопросы

1. Что такое Device Tree и какую роль оно выполняет при загрузке Linux?
2. Чем файл .dts отличается от .dtb?
3. Для чего нужны SPL и U-Boot proper в цепочке загрузки?
4. Какую роль выполняет OpenSBI при загрузке RISC-V?
5. Какая информация содержится в файле extlinux.conf?

5.7 Требования к отчёту

Отчёт должен содержать:

- цель работы;
- схему цепочки загрузки Lichee RV Dock;
- путь и результат проверки DTB;
- команды и результаты сборки OpenSBI и U-Boot;
- сведения о пакетном U-Boot и сравнение двух вариантов;
- содержимое подготовленного extlinux.conf;
- имя, размер и контрольную сумму архива rootfs;
- перечень сохранённых компонентов;
- ответы на контрольные вопросы;
- краткий вывод.

Подготовьте отчёт в согласованном с преподавателем формате и отправьте его до установленного срока.

6 Лабораторная №6. Подготовка носителя и запись образа

6.1 Цель работы

Освоить подготовку загрузочной SD-карты для одноплатного компьютера и записать на неё готовые компоненты Linux: U-Boot, ядро, Device Tree, конфигурацию загрузчика и корневую файловую систему.

6.2 Оборудование и исходные данные

- одноплатный компьютер Lichee RV Dock;
- SD-карта и кардридер;
- компьютер с Linux;
- файл `u-boot-sunxi-with-spl.bin`;
- ядро `kernels/Image_6.16_wifi_rt`;
- файл Device Tree `dtb/sun20i-d1-lichee-rv-dock.dtb`;
- конфигурация загрузчика `extlinux.conf`, подготовленная в лабораторной №5;
- архив корневой файловой системы `rootfs.tar.xz`;
- USB-UART-преобразователь с логическими уровнями 3,3 В.

Опасность повреждения

Команды `dd`, `fdisk` и `mkfs` безвозвратно уничтожают данные на выбранном устройстве. Перед каждым запуском проверьте модель и размер карты и убедитесь, что `/dev/sdX` обозначает именно SD-карту, а не системный диск.

6.3 Ключевые понятия

Определение

Таблица разделов — структура на носителе, которая описывает расположение и размеры его разделов.

В этой работе на SD-карте создаются два раздела. Раздел `B00T` с файловой системой `VFAT` хранит ядро, `DTB` и конфигурацию загрузчика. Раздел `R00TFS` с файловой системой `ext4` хранит корневую файловую систему Linux.

Определение

U-Boot — загрузчик, который подготавливает оборудование и запускает ядро Linux с выбранными Device Tree и параметрами командной строки.

U-Boot записывается не в файловую систему, а непосредственно на устройство со смещением 8 КиБ. Это смещение предусмотрено схемой загрузки Allwinner: BootROM ищет SPL/U-Boot в определённой области носителя. Первый раздел начинается с сектора 2048, поэтому загрузчик размещается перед ним и не пересекается с файловыми системами.

6.4 Порядок выполнения

6.4.1 Определение устройства SD-карты

Подключите кардридер и определите имя устройства:

```
lsblk -o NAME,SIZE,MODEL,FSTYPE,LABEL,MOUNTPOINTS
```

В дальнейших командах заменяйте `/dev/sdX` фактическим именем устройства, например `/dev/sdb`. Разделы этого устройства будут называться `/dev/sdX1` и `/dev/sdX2`.

Проверка

Проверка устройства. Сравните вывод `lsblk` до и после подключения кардридера. Новое устройство должно иметь размер, соответствующий SD-карте.

Размонтируйте все автоматически подключённые разделы карты:

```
sudo umount /dev/sdX?*
```

Внимание

Если оболочка сообщает, что раздел не смонтирован, это не является ошибкой подготовки. Если устройство занято, закройте файловый менеджер и терминалы, использующие каталоги карты, после чего повторите размонтирование.

6.4.2 Очистка начала карты

Запишите нули в первые 10 МиБ, чтобы удалить прежнюю таблицу разделов и служебные данные файловых систем:

```
sudo dd if=/dev/zero of=/dev/sdX bs=1M count=10 status=progress conv=fsync
```

6.4.3 Создание разделов

Запустите fdisk:

```
sudo fdisk /dev/sdX
```

Последовательно введите команды:

```
o          # создать новую таблицу разделов DOS (MBR)
n          # создать раздел
p          # основной раздел
1          # номер первого раздела
2048       # начальный сектор
+100M     # размер раздела B00T
n          # создать раздел
p          # основной раздел
2          # номер второго раздела
<Enter>    # начать сразу после первого раздела
<Enter>    # занять всё оставшееся пространство
w          # записать изменения и выйти
```

Для второго раздела примите предложенный последний сектор, чтобы использовать оставшееся пространство карты независимо от её фактической ёмкости.

6.4.4 Создание файловых систем

Создайте VFAT с меткой B00T и ext4 с меткой R00TFS:

```
sudo mkfs.vfat -n B00T /dev/sdX1
sudo mkfs.ext4 -L R00TFS /dev/sdX2
```

VFAT используется для B00T как простая файловая система, поддерживаемая U-Boot. ext4 подходит для R00TFS, поскольку поддерживает права доступа, символические ссылки и другие свойства файлов Linux.

Проверьте результат:

```
sudo fdisk -l /dev/sdX
lsblk -o NAME,SIZE,FSTYPE,LABEL,MOUNTPOINTS /dev/sdX
```

Проверка

Ожидаемый результат. На карте существуют два раздела: первый размером около 100 МиБ с меткой B00T, второй занимает оставшееся пространство и имеет метку R00TFS.

6.4.5 Запись U-Boot

На схеме показано итоговое размещение компонентов для карты объёмом 32 ГБ. Рисунок выполнен не в масштабе.

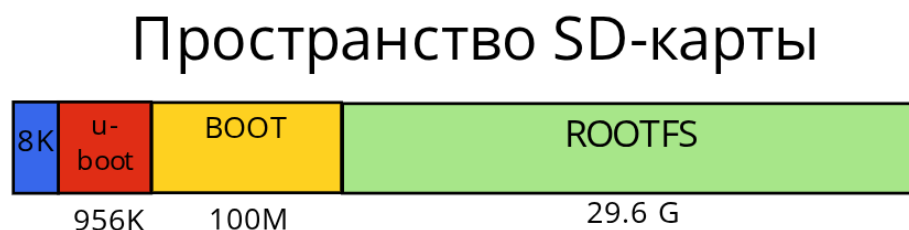


Рис. 6.1 – Схема разбивки SD-карты на разделы BOOT и ROOTFS

Запишите U-Boot со смещением 8 КиБ. В параметре of указывается всё устройство без номера раздела:

```
sudo dd if="/путь/до/загрузчика/lichee rv dock/u-boot-sunxi-with-spl.bin"  
↪ of=/dev/sdX bs=1K seek=8 conv=fsync,notrunc status=progress
```

Опасность повреждения

Ещё раз проверьте /dev/sdX перед запуском. Ошибка в имени устройства повредит данные на другом диске. Разделы карты должны оставаться размонтированными.

Параметры `bs=1K seek=8` пропускают 8 КиБ от начала карты. Таблица MBR остаётся в первом секторе, а U-Boot должен полностью помещаться в области до первого раздела, начинающегося с сектора 2048.

6.4.6 Заполнение раздела BOOT

Переподключите карту или попросите систему перечитать таблицу разделов, затем смонтируйте разделы штатными средствами окружения. Определите их каталоги командой:

```
lsblk -o NAME,LABEL,MOUNTPOINTS /dev/sdX
```

В следующих командах замените /путь/до/BOOT фактическим каталогом монтирования раздела BOOT:

```
cp kernels/Image_6.16_wifi_rt /путь/до/BOOT/Image  
cp dts_and_dtb/sun20i-d1-lichee-rv-dock.dtb /путь/до/BOOT/  
mkdir -p /путь/до/BOOT/extlinux  
cp extlinux.conf /путь/до/BOOT/extlinux/extlinux.conf
```

Внимание

U-Boot ищет конфигурацию по пути `extlinux/extlinux.conf` внутри раздела `B00T`. Файл в корне раздела или каталог с другим именем не будут соответствовать используемой схеме загрузки.

6.4.7 Заполнение раздела `R00TFS`

Замените `/путь/до/R00TFS` фактическим каталогом монтирования раздела `R00TFS` и распакуйте архив с сохранением прав доступа:

```
sudo tar -xJpf /путь/до/rootfs.tar.xz -C /путь/до/R00TFS/
```

Перед извлечением карты сбросьте кэш записи и размонтируйте оба раздела:

```
sync  
sudo umount /dev/sdX1 /dev/sdX2
```

Внимание

Не извлекайте кардридер до завершения `sync` и размонтирования: отложенная запись может оставить файлы или файловые системы в повреждённом состоянии.

6.4.8 Проверка загрузки

Установите SD-карту в Lichee RV Dock и наблюдайте загрузку через UART.

Опасность повреждения

Подключайте UART только при выключенном питании платы. Используйте уровень 3,3 В, соедините GND платы и преобразователя и не подавайте 5 В на сигнальные линии.

Проверка

Ожидаемый результат. U-Boot читает `extlinux/extlinux.conf`, загружает ядро и DTB из раздела `B00T`, после чего Linux монтирует раздел `R00TFS`.

6.5 Задание

1. Определите устройство SD-карты и подтвердите его модель и размер.
2. Создайте разделы `B00T` и `R00TFS` по указанной схеме.
3. Создайте файловые системы VFAT и ext4 с соответствующими метками.
4. Запишите U-Boot со смещением 8 КиБ.
5. Разместите ядро, DTB и `extlinux.conf` в разделе `B00T`.

6. Распакуйте корневую файловую систему в раздел R00TFS.
7. Корректно размонтируйте карту и загрузите Lichee RV Dock.
8. Продемонстрируйте преподавателю журнал успешной загрузки через UART.

6.6 Контрольные вопросы

1. Почему для загрузочной SD-карты создаются два раздела?
2. Зачем U-Boot записывается со смещением 8 КиБ?
3. Какая файловая система используется для раздела B00T и почему?
4. Какие компоненты необходимы для успешной загрузки системы?

6.7 Требования к отчёту

Отчёт должен содержать:

- цель работы;
- модель, размер и имя устройства SD-карты;
- полученную таблицу разделов и метки файловых систем;
- команды записи U-Boot и размещения компонентов образа;
- структуру раздела B00T;
- фрагмент журнала успешной загрузки через UART;
- ответы на контрольные вопросы;
- краткий вывод.

Подготовьте отчёт в согласованном с преподавателем формате и отправьте его до установленного срока.

7 Лабораторная №7. Добавление поддержки интерфейса SPI

7.1 Цель работы

Изучить принципы работы SPI, настроить контроллер Allwinner D1 в Device Tree, включить необходимые драйверы ядра Linux и проверить обмен с SPI-устройством из пользовательского пространства.

7.2 Оборудование и предварительные требования

- одноплатный компьютер Lichee RV Dock с подготовленной SD-картой;
- компьютер со средой сборки ядра Linux и исходными файлами Device Tree;
- результаты лабораторных №4–6;
- USB-UART-преобразователь с логическими уровнями 3,3 В;
- перемычка для теста MOSI-MISO;
- Python 3 и модуль spidev на плате.

Опасность повреждения

Все внешние соединения с GPIO выполняйте только при выключенном питании платы. Выводы Allwinner D1 работают с логическими уровнями 3,3 В; подача 5 В может повредить плату.

7.3 Ключевые понятия

Определение

SPI (Serial Peripheral Interface) — синхронный последовательный интерфейс, в котором контроллер формирует тактовый сигнал и выбирает периферийное устройство отдельной линией CS.

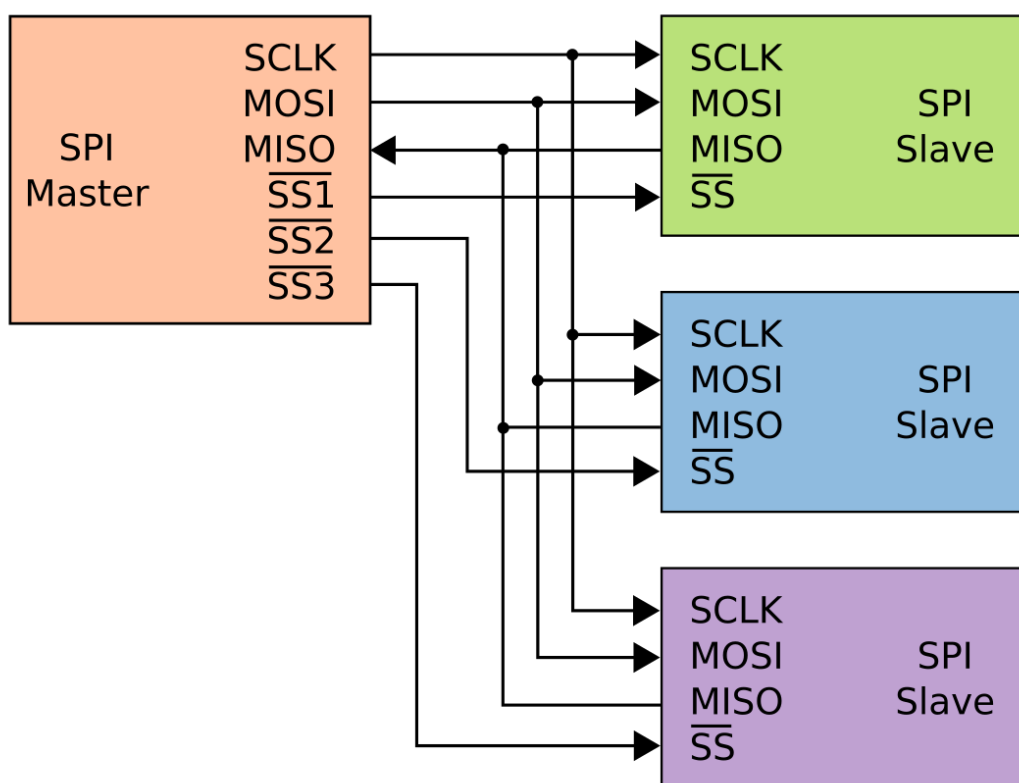


Рис. 7.1 – Топология SPI с общими линиями данных и отдельными сигналами выбора периферийных устройств

SPI использует четыре основных сигнала:

- SCK — тактовый сигнал;
- MOSI — данные от контроллера к периферийному устройству;
- MISO — данные от периферийного устройства к контроллеру;
- CS — выбор конкретного периферийного устройства.

Линии MOSI, MISO и SCK могут быть общими для нескольких устройств, а линия CS должна быть отдельной для каждого из них. Специализированный вывод контроллера SPI или обычный GPIO может формировать CS.

Определение

Полнодуплексный обмен — одновременная передача данных по MOSI и приём данных по MISO на каждом такте SCK.

Каждая операция SPI принимает столько же байтов, сколько передаёт, даже если принятые байты не содержат полезной информации.

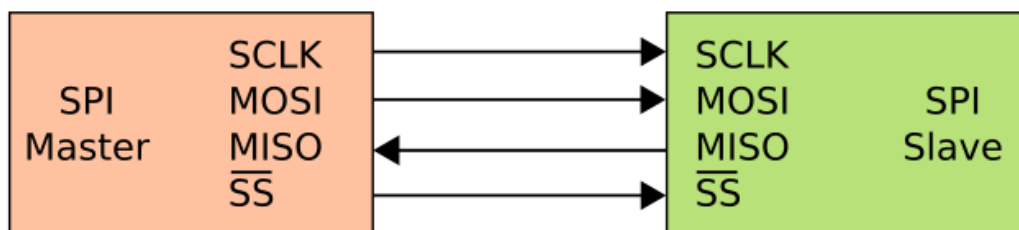


Рис. 7.2 – Подключение одного SPI-устройства к Lichee RV Dock

7.4 Порядок выполнения

7.4.1 Анализ аппаратной конфигурации

На Lichee RV Dock сигналы SPI1 выведены на контакты PD10–PD15 как альтернативные функции GPIO.

Sipeed Lichee RV DOCK
pinout diagram

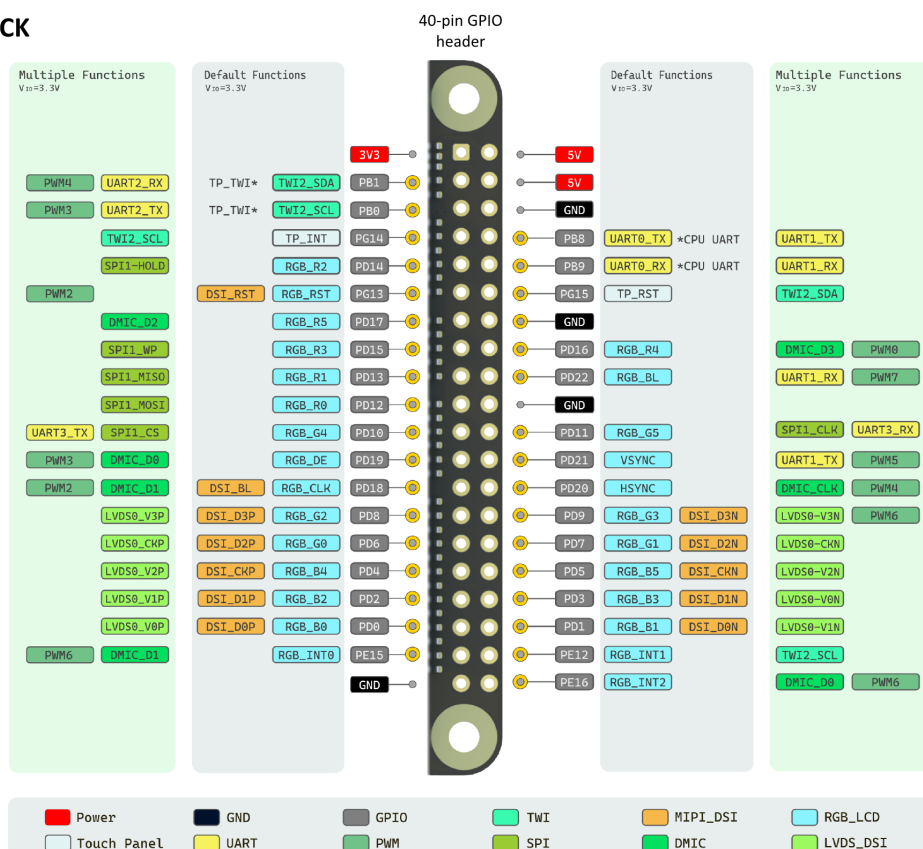


Рис. 7.3 – Распиновка Lichee RV Dock: сигналы SPI1 на выводах PD10–PD15

По умолчанию эти выводы могут быть назначены RGB-интерфейсу дисплея. Перед активацией SPI проверьте исходный Device Tree и исключите конфликт функций.

Внимание

Один физический вывод нельзя одновременно использовать для RGB и SPI. Если узел дисплея занимает PD10–PD15, отключите конфликтующую функцию в Device Tree до включения SPI1.

7.4.2 Анализ описания SPI в Device Tree

Исходные файлы платы находятся в каталоге `arch/riscv/boot/dts/allwinner/`. Файл `sun20i-d1-lichee-rv-dock.dts` включает описание базовой платы, а оно, в свою очередь, включает общие файлы `.dtsi` с контроллерами SoC.

Определение

Device Tree Source Include (DTSI) — включаемый исходный файл дерева устройств с общими описаниями аппаратных блоков и выводов.

В общем файле объявлены группы выводов двух контроллеров SPI:

```
/omit-if-no-ref/
spi0_pins: spi0-pins {
    pins = "PC2", "PC3", "PC4", "PC5";
    function = "spi0";
};

/omit-if-no-ref/
spi1_pb_pins: spi1-pb-pins {
    pins = "PB0", "PB8", "PB9", "PB10", "PB11", "PB12";
    function = "spi1";
};

/omit-if-no-ref/
spi1_pd_pins: spi1-pd-pins {
    pins = "PD10", "PD11", "PD12", "PD13", "PD14", "PD15";
    function = "spi1";
};
```

Директива `/omit-if-no-ref/` исключает неиспользуемый узел из итогового DTB. Контроллеры объявлены отдельно и изначально отключены:

```
spi0: spi@4025000 {
    compatible = "allwinner,sun20i-d1-spi",
                 "allwinner,sun50i-r329-spi";
    reg = <0x4025000 0x1000>;
    interrupts = <31 IRQ_TYPE_LEVEL_HIGH>;
    clocks = <&ccu CLK_BUS_SPI0>, <&ccu CLK_SPI0>;
    clock-names = "ahb", "mod";
    resets = <&ccu RST_BUS_SPI0>;
    dmas = <&dma 22>, <&dma 22>;
    dma-names = "rx", "tx";
```

```

        num-cs = <1>;
        status = "disabled";
        #address-cells = <1>;
        #size-cells = <0>;
};

spi1: spi@4026000 {
    compatible = "allwinner,sun20i-d1-spi-dbi",
        "allwinner,sun50i-r329-spi-dbi",
        "allwinner,sun50i-r329-spi";
    reg = <0x4026000 0x1000>;
    interrupts = <32 IRQ_TYPE_LEVEL_HIGH>;
    clocks = <&ccu CLK_BUS_SPI1>, <&ccu CLK_SPI1>;
    clock-names = "ahb", "mod";
    resets = <&ccu RST_BUS_SPI1>;
    dmas = <&dma 23>, <&dma 23>;
    dma-names = "rx", "tx";
    num-cs = <1>;
    status = "disabled";
    #address-cells = <1>;
    #size-cells = <0>;
};

```

Свойство `compatible` связывает аппаратный узел с поддерживающим его драйвером. При обнаружении активного совместимого узла драйвер выполняет инициализацию устройства.

7.4.3 Активация SPI1 и SPIDEV

Для включения SPI1 на группе PD10–PD15 добавьте в DTS платы:

```

&spi1 {
    pinctrl-0 = <&spi1_pd_pins>;
    pinctrl-names = "default";
    status = "okay";
};

```

Для доступа из пользовательского пространства требуется драйвер SPIDEV и дочерний узел устройства. В этой работе используется совместимый идентификатор, присутствующий в таблице драйвера:

```

&spi1 {
    pinctrl-0 = <&spi1_pd_pins>;
    pinctrl-names = "default";
    status = "okay";

    spidev0: spidev@0 {
        compatible = "rohm,dh2228fv";
        reg = <0>;
    };
};

```

```
        spi-max-frequency = <1000000>;  
    };  
};
```

Внимание

Идентификатор `rohm,dh2228fv` применяется здесь как учебный способ привязать `SPIDEV`. Для реального периферийного устройства используйте его штатный `compatible` и драйвер, если они существуют.

7.4.4 Настройка ядра

Используйте конфигурацию, подготовленную в лабораторной №4. Проверьте и включите параметры:

```
CONFIG_SPI=y  
CONFIG_SPI_MASTER=y  
CONFIG_SPI_SUN6I=y  
CONFIG_SPI_SPIDEV=y  
CONFIG_DMA_SUN6I=y
```

Драйвер `SPI_SUN6I` исторически назван по первому поддерживаемому семейству, но применяется и к совместимым контроллерам новых SoC, включая `sun20i`. Драйвер `DMA_SUN6I` нужен контроллеру для доступа к DMA-каналам в рассматриваемой конфигурации.

Внимание

Рабочая конфигурация требует согласованных изменений Device Tree и ядра. При отсутствии одного из нужных драйверов устройство `/dev/spidev*` может не появиться, а контроллер может завершить инициализацию с ошибкой.

Соберите ядро и DTB, обновите компоненты на SD-карте и загрузите плату согласно предыдущим лабораторным.

7.4.5 Проверка устройства

После загрузки проверьте наличие символьного устройства:

```
ls -l /dev/spidev*  
dmesg | grep -i spi
```

Проверка

Ожидаемый результат. Для шины 1 и устройства с `reg = <0>` появляется `/dev/spidev1.0`. Фактическое имя подтвердите по выводу системы.

7.4.6 Полнодуплексный тест

При полностью отключённом питании соедините выводы MOSI и MISO, затем загрузите плату и запустите тест:

```
import time

import spidev

def spi_sequential_test(bus=1, device=0, speed=1_000_000):
    spi = spidev.SpiDev()
    spi.open(bus, device)
    spi.max_speed_hz = speed
    spi.mode = 0
    spi.lsbfirst = False

    try:
        print("Последовательный тест SPI запущен")
        for value in range(256):
            response = spi.xfer2([value])
            print(
                f"Отправлено: {value:3d} (0x{value:02X}) | "
                f"Получено: {response[0]:3d} (0x{response[0]:02X}) | "
                f"Двоично: {response[0]:08b}"
            )
            time.sleep(0.01)
    finally:
        spi.close()

if __name__ == "__main__":
    spi_sequential_test(bus=1, device=0, speed=1_000_000)
```

Опасность повреждения

Устанавливайте и снимайте перемычку MOSI-MISO только при выключенном питании. Не соединяйте сигнальные выводы с питанием или землёй и не подавайте на них 5 В.

Проверка

Ожидаемый результат. При исправной перемычке каждый принятый байт совпадает с переданным. Несовпадение требует проверки имени `/dev/spidev*`, распиновки, режима и соединения MOSI-MISO.

```
[root@localhost my_spi_device]# python3 spi.py
SPI последовательный тест запущен...
Отправляю данные: 0, 1, 2, 3, ...
-----
Отправлено: 0 (0x00) | Получено: 0 (0x00) | Бинарно: 00000000
Отправлено: 1 (0x01) | Получено: 1 (0x01) | Бинарно: 00000001
Отправлено: 2 (0x02) | Получено: 2 (0x02) | Бинарно: 00000010
Отправлено: 3 (0x03) | Получено: 3 (0x03) | Бинарно: 00000011
Отправлено: 4 (0x04) | Получено: 4 (0x04) | Бинарно: 00000100
Отправлено: 5 (0x05) | Получено: 5 (0x05) | Бинарно: 00000101
Отправлено: 6 (0x06) | Получено: 6 (0x06) | Бинарно: 00000110
Отправлено: 7 (0x07) | Получено: 7 (0x07) | Бинарно: 00000111
Отправлено: 8 (0x08) | Получено: 8 (0x08) | Бинарно: 00001000
Отправлено: 9 (0x09) | Получено: 9 (0x09) | Бинарно: 00001001
Отправлено: 10 (0x0A) | Получено: 10 (0x0A) | Бинарно: 00001010
```

Рис. 7.4 – Результат полнодуплексного теста SPI с переключкой между MOSI и MISO

7.5 Задание

1. Проанализируйте Device Tree платы Lichee RV Dock и найдите описание SPI1 и группы `spi1_pd_pins`.
2. Убедитесь, что выводы PD10–PD15 не заняты конфликтующим узлом.
3. Активируйте SPI1 и добавьте дочерний узел для доступа через SPIDEV.
4. Включите `CONFIG_SPI`, `CONFIG_SPI_MASTER`, `CONFIG_SPI_SUN6I`, `CONFIG_SPI_SPIDEV` и `CONFIG_DMA_SUN6I`.
5. Соберите ядро и DTB, обновите загрузочную карту и загрузите систему.
6. Подтвердите наличие `/dev/spidev*`.
7. Проведите тест с переключкой MOSI-MISO и зафиксируйте результат.
8. Продемонстрируйте работу преподавателю.

7.6 Контрольные вопросы

1. Какие четыре сигнала используются в SPI и за что отвечает каждый?
2. Чем топология SPI отличается от шинной топологии I2C?
3. Почему узел SPI в Device Tree по умолчанию имеет `status = "disabled"`?
4. Зачем нужен SPIDEV и какое устройство появляется в `/dev` после его активации?
5. Какой идентификатор `compatible` используется в этой работе для привязки SPIDEV?

6. Почему для рассматриваемой конфигурации Allwinner D1 требуется CONFIG_DMA_SUN6I?

7.7 Требования к отчёту

Отчёт должен содержать:

- цель работы;
- фрагменты Device Tree до и после изменения;
- включённые параметры конфигурации ядра;
- вывод проверки /dev/spidev* и значимые сообщения dmesg;
- схему или фотографию перемычки MOSI-MISO;
- результат полнодуплексного теста;
- ответы на контрольные вопросы;
- краткий вывод.

Подготовьте отчёт в согласованном с преподавателем формате и отправьте его до установленного срока.

8 Лабораторная №8. Добавление поддержки интерфейса I2C

8.1 Цель работы

Изучить принципы работы I2C, активировать контроллер TWI2 микросхемы Allwinner D1 в Device Tree, включить необходимые драйверы ядра Linux и вывести данные на дисплей SSD1306 из пользовательского пространства.

8.2 Оборудование и предварительные требования

- одноплатный компьютер Lichee RV Dock с подготовленной SD-картой;
- компьютер со средой сборки ядра Linux и исходными файлами Device Tree;
- результаты лабораторных №4–7;
- OLED-дисплей SSD1306 с интерфейсом I2C;
- четыре соединительных провода;
- Python 3, i2c-tools и модуль luma-oled на плате.

Опасность повреждения

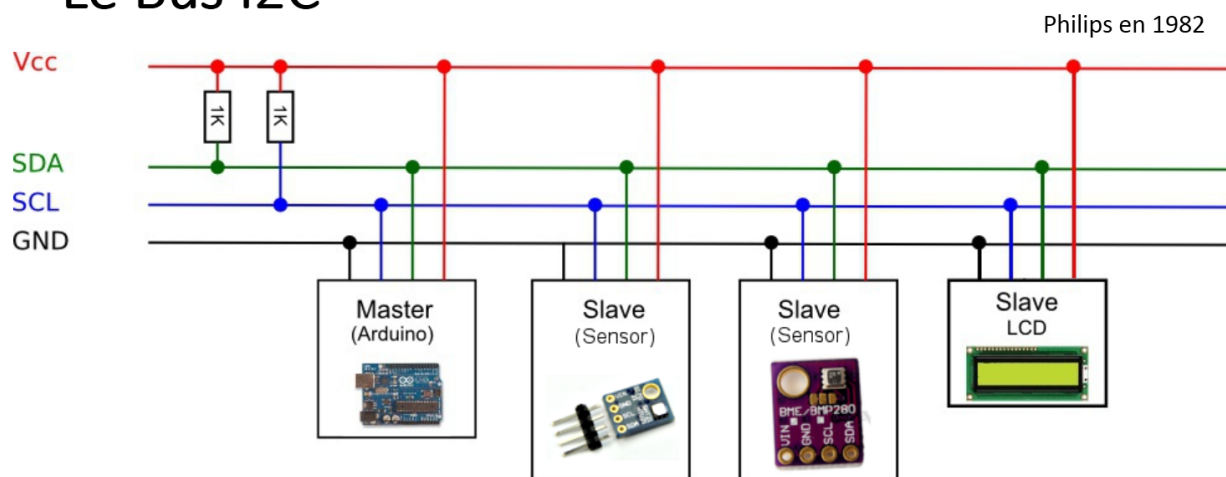
Подключайте дисплей только при полностью выключенном питании. Питайте его от 3,3 В и проверьте, что подтягивающие резисторы линий SDA и SCL не подключены к 5 В.

8.3 Ключевые понятия

Определение

I2C (Inter-Integrated Circuit) — синхронная двухпроводная шина, на которой устройства используют общие линии SDA и SCL и различаются адресами.

Le Bus I2C



- 2 fils de connexion (SCL : horloge, SDA: donnée)
- 127 adresses ou esclave possibles
- Vitesse 100kb/s à 400kb/s (Arduino)
- Transmission synchrone

14/21

Рис. 8.1 – Многоточечная топология шины I2C с линиями SDA и SCL

Контроллер шины формирует тактовый сигнал и инициирует обмен с выбранным периферийным устройством. Интерфейс использует две линии:

- SDA (Serial Data) — двунаправленная линия данных;
- SCL (Serial Clock) — линия тактирования, формируемого контроллером.

Определение

TWI (Two-Wire Interface) — используемое в документации Allwinner название аппаратного интерфейса, совместимого с I2C.

Обозначения TWI2 в документации платы и I2C2 в Linux и Device Tree относятся к одному контроллеру. На Lichee RV Dock подтверждены следующие сигналы:

Сигнал	Вывод Allwinner D1
TWI2_SCL	PE12
TWI2_SDA	PE13

Sipeed Lichee RV DOCK pinout diagram

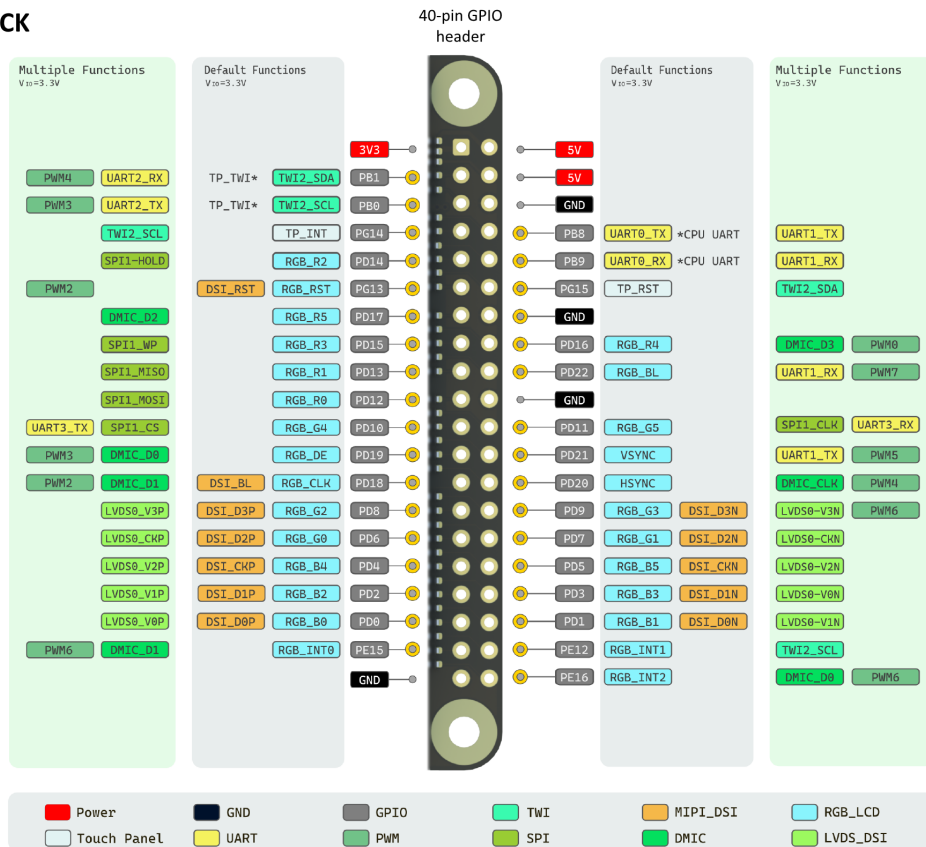


Рис. 8.2 – Распиновка Lichee RV Dock: линии TWI2 SDA и TWI2 SCL

Определение

Открытый сток — схема выхода, в которой устройство может притянуть линию к низкому уровню, а высокий уровень создаётся внешним подтягивающим резистором.

SDA и SCL требуют подтяжек к допустимому напряжению питания. Несколько модулей могут иметь собственные подтягивающие резисторы, поэтому перед подключением необходимо проверить схему конкретного дисплейного модуля.

8.4 Порядок выполнения

8.4.1 Активация I2C2 в Device Tree

По аналогии с лабораторной №7 добавьте или проверьте группу выводов PE12 и PE13 в общем файле DTSI:

```
/omit-if-no-ref/
i2c2_pe_pins: i2c2-pe-pins {
    pins = "PE12", "PE13";
    function = "i2c2";
};
```

В исходном DTS платы активируйте контроллер и назначьте ему эту

группу:

```
&i2c2 {  
    pinctrl-0 = <&i2c2_pe_pins>;  
    pinctrl-names = "default";  
    status = "okay";  
};
```

Внимание

Перед добавлением группы найдите существующие объявления в используемой версии исходного дерева. Не создавайте второй узел с той же меткой и убедитесь, что PE12 и PE13 не назначены другому активному устройству.

Соберите DTB после изменения. Ошибок и предупреждений о повторяющихся метках или некорректных свойствах быть не должно.

8.4.2 Настройка ядра

Включите или проверьте следующие параметры конфигурации:

```
CONFIG_I2C=y  
CONFIG_I2C_MUX=y  
CONFIG_I2C_MV64XXX=y  
CONFIG_I2C_CHARDEV=y
```

CONFIG_I2C_MV64XXX включает драйвер контроллера, совместимого с аппаратным блоком Allwinner D1. CONFIG_I2C_CHARDEV предоставляет символьные устройства /dev/i2c-* для программ пользовательского пространства. Поддержка мультиплексоров CONFIG_I2C_MUX сохраняется в конфигурации лаборатории для совместимости с используемой сборкой.

Соберите ядро и DTB, обновите компоненты на SD-карте и загрузите плату.

8.4.3 Проверка I2C-адаптера

Установите диагностические утилиты и библиотеку дисплея:

```
sudo apt-get install i2c-tools python3-module-luma-oled
```

Определите фактическую нумерацию адаптеров:

```
i2cdetect -l  
ls -l /dev/i2c-*  
dmesg | grep -i i2c
```

Проверка

Ожидаемый результат. Активированный контроллер i2c2 обычно представлен устройством /dev/i2c-2. Если номер отличается, используйте фактический номер и в имени устройства, и в параметре port программы.

Внимание

Не сканируйте неизвестную шину без проверки документации подключённых устройств: некоторые адреса и команды могут изменить их состояние. Для выданного SSD1306 проверяйте только разрешённый преподавателем диапазон или известный адрес.

8.4.4 Подключение SSD1306

При выключенном питании соедините дисплей с платой:

SSD1306	Lichee RV Dock
VCC	3.3V
GND	GND
SCL	TWI2_SCL (PE12)
SDA	TWI2_SDA (PE13)

Опасность повреждения

Сверьте маркировку контактов конкретного модуля до включения питания. Перепутанные VCC и GND либо подтяжка SDA/SCL к 5 В могут повредить дисплей или плату.

8.4.5 Вывод имени на дисплей

Программа ожидает имя и фамилию в аргументах командной строки и использует типичный адрес SSD1306 0x3C. Если проверенный адрес или номер адаптера отличаются, скорректируйте address и port.

```
#!/usr/bin/env python3

import sys
import time

from PIL import ImageFont
from luma.core.interface.serial import i2c
from luma.core.render import canvas
from luma.oled.device import ssd1306

def display_large_centered(device, first_name, last_name):
```

```

first_name = first_name.lower()
last_name = last_name.lower()

try:
    font = ImageFont.truetype(
        "/usr/share/fonts/truetype/dejavu/DejaVuSans.ttf", 16
    )
except OSError:
    font = ImageFont.load_default()

with canvas(device) as draw:
    name_box = draw.textbbox((0, 0), first_name, font=font)
    surname_box = draw.textbbox((0, 0), last_name, font=font)
    name_width = name_box[2] - name_box[0]
    surname_width = surname_box[2] - surname_box[0]

    draw.text(((128 - name_width) // 2, 15), first_name, fill="white",
        ↪ font=font)
    draw.text(
        ((128 - surname_width) // 2, 38),
        last_name,
        fill="white",
        font=font,
    )

def main():
    if len(sys.argv) != 3:
        print(f"Использование: {sys.argv[0]} ИМЯ ФАМИЛИЯ")
        return 1

    serial = i2c(port=2, address=0x3C)
    device = ssd1306(serial)
    device.clear()
    display_large_centered(device, sys.argv[1], sys.argv[2])
    print(f"Выведено на экран: {sys.argv[1].lower()} {sys.argv[2].lower()}")
    time.sleep(10)
    return 0

if __name__ == "__main__":
    raise SystemExit(main())

```

Запустите сохранённую программу, передав собственные данные:

```
python3 display_name.py Иван Петров
```

Проверка

Ожидаемый результат. На дисплее в течение 10 секунд отображаются переданные имя и фамилия, а программа завершается без ошибки ввода-вывода.

Внимание

Резервный шрифт `ImageFont.load_default()` может не содержать кириллицу. Если символы отображаются неверно, установите шрифт с поддержкой кириллицы и укажите путь к нему, не меняя логику работы с I2C.

8.5 Задание

1. Найдите описание `i2c2` в исходном дереве устройств.
2. Добавьте или проверьте группу `i2c2_re_pins` для PE12 и PE13.
3. Активируйте `i2c2` в DTS платы и исключите конфликт выводов.
4. Включите требуемые параметры конфигурации ядра.
5. Соберите ядро и DTB, обновите загрузочную карту и загрузите систему.
6. Определите фактический номер I2C-адаптера и подтвердите наличие `/dev/i2c-*`.
7. Подключите SSD1306 и выведите на него собственные имя и фамилию.
8. Продемонстрируйте результат преподавателю.

8.6 Контрольные вопросы

1. Почему интерфейс I2C на Lichee RV Dock обозначен как TWI?
2. Как организована адресация устройств на шине I2C?
3. Чем различаются линии SDA и SCL по назначению?
4. Какие параметры ядра включаются для поддержки I2C на Allwinner D1?
5. Для чего нужен `CONFIG_I2C_CHARDEV` и какое устройство появляется в `/dev`?
6. Как безопасно определить I2C-адрес подключённого устройства, если он неизвестен?

8.7 Требования к отчёту

Отчёт должен содержать:

- цель работы;
- подтверждённое соответствие `TWI2_SCL/TWI2_SDA` выводам SoC;

- фрагменты Device Tree и параметры конфигурации ядра;
- вывод `i2cdetect -l` и фактическое имя `/dev/i2c-*`;
- схему или фотографию подключения SSD1306;
- исходный код и фотографию результата вывода имени и фамилии;
- ответы на контрольные вопросы;
- краткий вывод.

Подготовьте отчёт в согласованном с преподавателем формате и отправьте его до установленного срока.

9 Лабораторная №9. Работа с логическим анализатором

9.1 Цель работы

Изучить принципы работы логического анализатора, освоить управление GPIO через API `libgpiod 2.x` и получить практические навыки измерения цифровых сигналов и декодирования UART в Sigrok/PulseView.

9.2 Оборудование и предварительные требования

- одноплатный компьютер Lichee RV Dock с ALT Linux;
- USB-логический анализатор, поддерживаемый драйвером `fx2lafw`;
- соединительные провода;
- компьютер с Sigrok и PulseView;
- компилятор GCC, библиотека `libgpiod 2.x` и заголовочные файлы;
- доступ к UART TX и выводу PE16 платы.

Опасность повреждения

Собирайте и изменяйте схему только при выключенном питании. Соедините GND платы и анализатора, подключайте к GPIO только входной канал анализатора и не подавайте 5 В на выводы Lichee RV Dock.

9.3 Ключевые понятия

Определение

Логический анализатор — прибор, который дискретизирует логические уровни нескольких цифровых линий и записывает их изменения во времени.

Логический анализатор удобен для длительного многоканального захвата и декодирования цифровых протоколов. Осциллограф показывает фактическую форму и амплитуду сигнала, поэтому лучше подходит для анализа фронтов, выбросов, шумов и нарушений электрических уровней.

В ALT Linux доступны пакеты проекта Sigrok:

- `sigrok-cli` — утилита командной строки;
- `pulseview` — графический интерфейс.

Определение

Sigrok — проект программного обеспечения для работы с измерительными приборами и декодирования протоколов.

PulseView является графическим приложением проекта Sigrok для настройки захвата, отображения сигналов и работы с декодерами.

Определение

GPIO-линия — отдельный управляемый цифровой сигнал, идентифицируемый GPIO-чипом и смещением линии внутри него.

9.4 Порядок выполнения

9.4.1 Установка программного обеспечения

Установите программы для анализатора:

```
sudo apt-get install sigrok-cli sigrok-firmware-fx2lafw pulseview
```

Подключите анализатор к компьютеру и проверьте обнаружение:

```
sigrok-cli --scan
```

Проверка

Ожидаемый результат. В списке устройств присутствует анализатор с драйвером fx2lafw. Если устройство не найдено, проверьте USB-подключение, пакет прошивки и системный журнал.

9.4.2 Измерение цифрового импульса

При выключенном питании соедините GND анализатора с GND платы, а один входной канал CH0 или CH1 — с исследуемым выводом GPIO. После проверки схемы включите питание и запустите PulseView:

```
pulseview
```

Выберите анализатор fx2lafw, оставьте нужный канал, задайте частоту дискретизации и запустите захват. Для тестового сигнала с длительностью импульса 10 мс измерьте длительности высокого и низкого уровней, период и частоту.

Проверка

Проверка настройки. На один период должно приходиться достаточно отсчётов для устойчивого измерения. Если фронты отображаются нестабильно или пропускаются, увеличьте частоту дискретизации.

9.4.3 Декодирование UART

UART-консоль Lichee RV Dock передаёт загрузочные сообщения через TX. При выключенном питании подключите:

- GND анализатора к GND платы;
- CH0 анализатора к TX платы.

В PulseView выполните следующие действия:

1. Оставьте подключённый канал.
2. Установите частоту дискретизации 1 МГц или выше.
3. Добавьте декодер UART.
4. Назначьте вход декодера каналу, подключённому к TX.
5. Установите параметры формата 115200 8N1.

```
Baud rate: 115200
Data bits: 8
Parity: none
Stop bits: 1
Bit order: lsb-first
Data format: ascii
Invert RX: no
Invert TX: no
Sample point (%): 50
```

Проверка

Ожидаемый результат. Декодер отображает читаемые фрагменты загрузочного журнала. Если символы искажены, сначала проверьте общий GND, выбранный канал, скорость 115200 бод и отсутствие инверсии.

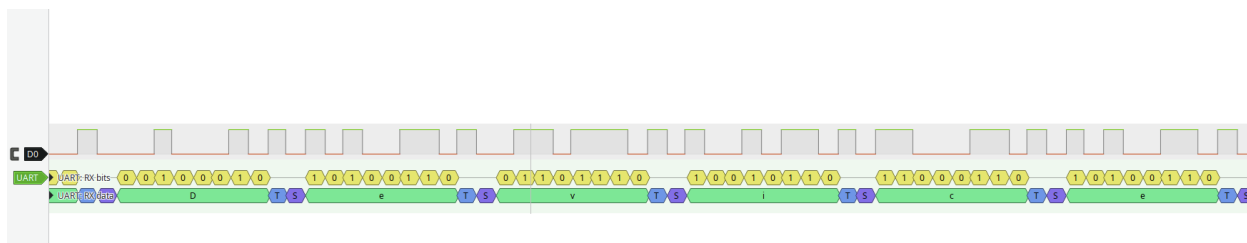


Рис. 9.1 – Декодирование загрузочного вывода UART 115200 8N1 в PulseView

9.4.4 Работа с libgpiod 2.x

Определение

libgpiod — библиотека пользовательского пространства для работы с символьным интерфейсом GPIO ядра Linux.

В ALT Linux пакет libgpiod2 содержит библиотеку времени выполнения, а libgpiod-devel — заголовки и файлы для компоновки программ:

```
sudo apt-get install libgpiod2 libgpiod-devel
```

Проверьте установленную версию и доступные GPIO-чипы:

```
gpiodetect --version
gpiodetect
gpioinfo
```

Внимание

Синтаксис утилит и C API libgpiod 1.x и 2.x различается. Команды и код этой работы рассчитаны на libgpiod 2.x.

Основные объекты API 2.x:

- `struct gpiod_chip` представляет открытое символьное устройство `/dev/gpiochipN`;
- `struct gpiod_line_settings` хранит направление и начальное значение линии;
- `struct gpiod_line_config` связывает настройки со смещениями линий;
- `struct gpiod_request_config` хранит параметры запроса, включая имя потребителя;
- `struct gpiod_line_request` представляет полученный исключительный доступ к линиям.

Основные функции, используемые в каркасе программы:

```
#include <gpiod.h>

struct gpiod_chip *gpiod_chip_open(const char *path);
void gpiod_chip_close(struct gpiod_chip *chip);

struct gpiod_line_settings *gpiod_line_settings_new(void);
int gpiod_line_settings_set_direction(
    struct gpiod_line_settings *settings,
    enum gpiod_line_direction direction
);
int gpiod_line_settings_set_output_value(
```

```

    struct gpiod_line_settings *settings,
    enum gpiod_line_value value
);
void gpiod_line_settings_free(struct gpiod_line_settings *settings);

struct gpiod_line_config *gpiod_line_config_new(void);
int gpiod_line_config_add_line_settings(
    struct gpiod_line_config *config,
    const unsigned int *offsets,
    size_t num_offsets,
    struct gpiod_line_settings *settings
);
void gpiod_line_config_free(struct gpiod_line_config *config);

struct gpiod_request_config *gpiod_request_config_new(void);
void gpiod_request_config_set_consumer(
    struct gpiod_request_config *config,
    const char *consumer
);
void gpiod_request_config_free(struct gpiod_request_config *config);

struct gpiod_line_request *gpiod_chip_request_lines(
    struct gpiod_chip *chip,
    struct gpiod_request_config *req_config,
    struct gpiod_line_config *line_config
);
int gpiod_line_request_set_value(
    struct gpiod_line_request *request,
    unsigned int offset,
    enum gpiod_line_value value
);
void gpiod_line_request_release(struct gpiod_line_request *request);

```

В API 2.x для логических состояний применяются GPIOD_LINE_VALUE_INACTIVE и GPIOD_LINE_VALUE_ACTIVE, а не необозначенные целые значения.

9.4.5 Определение линии PE16

В используемой конфигурации PE16 соответствует смещению 144 в /dev/gpiochip0. Перед работой подтвердите это соответствие и убедитесь, что линия свободна:

```

gpioinfo -c 0 144
gpioget -c 0 144
gpiomon -c 0 144

```

Для ручной установки выхода утилитой gpioset используется синтаксис 2.x:

```

gpioset -c 0 144=1

```

Внимание

Номер `/dev/gpiochip0` и смещение 144 необходимо проверять на фактически загруженной системе. Device Tree и версия ядра могут изменить представление GPIO; занятая линия не должна запрашиваться программой.

9.4.6 Каркас генератора меандра

Приведённый каркас открывает GPIO-чип, настраивает PE16 как выход и освобождает ресурсы после Ctrl+C. Самостоятельно дополните цикл формирования высокого и низкого уровней и обработкой длительностей из аргументов командной строки.

```
#include <gpiod.h>
#include <signal.h>
#include <stdio.h>

#define GPIO_CHIP "/dev/gpiochip0"
#define GPIO_OFFSET 144

static volatile sig_atomic_t keep_running = 1;

static void signal_handler(int signum)
{
    (void)signum;
    keep_running = 0;
}

int main(void)
{
    struct gpiod_chip *chip;
    struct gpiod_line_request *request;
    struct gpiod_line_settings *settings;
    struct gpiod_line_config *line_config;
    struct gpiod_request_config *request_config;
    unsigned int offset = GPIO_OFFSET;

    signal(SIGINT, signal_handler);
    signal(SIGTERM, signal_handler);

    chip = gpiod_chip_open(GPIO_CHIP);
    if (!chip) {
        perror("gpiod_chip_open");
        return 1;
    }

    settings = gpiod_line_settings_new();
    line_config = gpiod_line_config_new();
```

```

request_config = gpiod_request_config_new();
if (!settings || !line_config || !request_config) {
    fprintf(stderr, "Не удалось создать конфигурацию GPIO\n");
    gpiod_request_config_free(request_config);
    gpiod_line_config_free(line_config);
    gpiod_line_settings_free(settings);
    gpiod_chip_close(chip);
    return 1;
}

if (gpiod_line_settings_set_direction(
    settings, GPIOD_LINE_DIRECTION_OUTPUT) < 0 ||
    gpiod_line_settings_set_output_value(
    settings, GPIOD_LINE_VALUE_INACTIVE) < 0 ||
    gpiod_line_config_add_line_settings(
    line_config, &offset, 1, settings) < 0) {
    fprintf(stderr, "Не удалось настроить линию GPIO\n");
    gpiod_request_config_free(request_config);
    gpiod_line_config_free(line_config);
    gpiod_line_settings_free(settings);
    gpiod_chip_close(chip);
    return 1;
}

gpiod_request_config_set_consumer(request_config, "wavegen");
request = gpiod_chip_request_lines(chip, request_config, line_config);
if (!request) {
    perror("gpiod_chip_request_lines");
    gpiod_request_config_free(request_config);
    gpiod_line_config_free(line_config);
    gpiod_line_settings_free(settings);
    gpiod_chip_close(chip);
    return 1;
}

printf("Генератор запущен на линии %u; Ctrl+C для остановки\n", offset);
while (keep_running) {
    /* Реализуйте генерацию меандра здесь. */
}

gpiod_line_request_set_value(
    request, offset, GPIOD_LINE_VALUE_INACTIVE
);
gpiod_line_request_release(request);
gpiod_request_config_free(request_config);
gpiod_line_config_free(line_config);
gpiod_line_settings_free(settings);
gpiod_chip_close(chip);
return 0;

```

```
}
```

Скомпилируйте программу с библиотекой `libgpiod`:

```
gcc -Wall -Wextra -Wpedantic generator.c -o generator -lgpiod
```

Проверка

Проверка сборки. GCC должен завершиться без ошибок и предупреждений, а команда `ldd ./generator` должна показывать зависимость от `libgpiod.so.2`.

9.4.7 Реализация и измерение генератора

Программа должна принимать длительности высокого и низкого уровней в микросекундах, непрерывно формировать сигнал и корректно освобождать линию после `Ctrl+C`. Пример запуска для периода 500 мкс:

```
./generator 250 250
```

При выключенном питании подключите входной канал анализатора к PE16, а GND — к GND платы. После включения выполните захват в PulseView и измерьте длительности высокого и низкого уровней, период и частоту.

Опасность повреждения

Не соединяйте PE16, настроенный как выход, с другим активным выходом. Конфликт логических уровней может повредить оборудование.

Внимание

Пользовательский процесс Linux не обеспечивает жёсткий реальный масштаб времени. Планировщик, системные вызовы и функция ожидания вносят задержку и джиттер, поэтому измеренные интервалы могут отличаться от заданных, особенно при малых длительностях.

Проверка

Ожидаемый результат. Анализатор показывает периодический прямоугольный сигнал. После `Ctrl+C` программа завершается, а `gpioinfo` больше не показывает потребителя `wavegen` для выбранной линии.

9.5 Задание

1. Установите Sigrok, прошивку `fx2lafw` и PulseView.
2. Подключите анализатор и подтвердите его обнаружение.
3. Измерьте параметры тестового импульса длительностью 10 мс.

4. Подключитесь к ТХ платы и декодируйте UART 115200 8N1.
5. Установите `libgpiod 2.x` и пакет заголовочных файлов.
6. Подтвердите соответствие PE16 фактическому GPIO-чипу и смещению линии.
7. Дополните каркас генератора обработкой двух аргументов и формированием меандра.
8. Обеспечьте корректное завершение по `Ctrl+C` и освобождение GPIO-линии.
9. Соберите и запустите генератор с длительностями 250 и 250 мкс.
10. Измерьте высокий и низкий уровни, период и частоту в PulseView и сохраните захват.
11. Продемонстрируйте работу преподавателю.

9.6 Контрольные вопросы

1. Для чего нужен логический анализатор при отладке цифровых сигналов?
2. Какие параметры необходимо настроить в PulseView для декодирования UART?
3. Что такое GPIO-линия в терминах `libgpiod 2.x` и как к ней обратиться из программы?
4. Чем управление GPIO через `libgpiod` отличается от прямой записи в регистры микроконтроллера?

9.7 Требования к отчёту

Отчёт должен содержать:

- цель работы;
- модель анализатора и вывод `sigrok-cli --scan`;
- настройки и результат декодирования UART;
- версию `libgpiod` и вывод, подтверждающий выбранные GPIO-чип и смещение;
- исходный код и команду сборки генератора;
- параметры запуска и сохранённый захват PulseView;
- таблицу заданных и измеренных длительностей, периода и частоты;
- описание завершения программы и освобождения линии;
- ответы на контрольные вопросы;
- краткий вывод.

Подготовьте отчёт в согласованном с преподавателем формате и отправьте его до установленного срока.

10 Лабораторная №10. Основы программирования микроконтроллеров Arduino

10.1 Цель работы

Освоить базовые принципы программирования микроконтроллеров на примере платформы Arduino, научиться управлять светодиодами и обрабатывать внешние прерывания, а также организовать совместную работу Arduino и одноплатного компьютера.

10.2 Оборудование и исходные данные

- Arduino Uno и USB-кабель;
- светодиодная мезонинная плата;
- Lichee RV Dock с программой генерации меандра на GPIO 144 из лабораторной №9;
- преобразователь логических уровней $5 \leftrightarrow 3,3$ В на макетной плате;
- соединительные провода;
- компьютер с Arduino IDE и PulseView;
- логический анализатор.

10.3 Ключевые понятия

10.3.1 Платформа Arduino

Определение

Arduino — открытая аппаратно-программная платформа для прототипирования электронных устройств на базе микроконтроллерных плат и среды разработки.

- **микроконтроллер** — центральный вычислительный компонент платы, для Arduino Uno это ATmega328P;
- **программная среда** — Arduino IDE с библиотеками для работы с периферией;
- **стандартизированные разъёмы** — контакты для подключения периферии;
- **открытая архитектура** — возможность изучения, модификации и создания совместимых плат.

10.3.2 Программирование микроконтроллера и разработка под ОС

Программирование микроконтроллеров, таких как Arduino, принципиально отличается от разработки под операционные системы. В Arduino отсутствует операционная система — программа работает напрямую с аппаратным обеспечением, что обеспечивает прямой доступ к регистрам периферийных устройств. Это позволяет управлять оборудованием на самом низком уровне, но требует от разработчика глубокого понимания архитектуры микроконтроллера.

Одним из важных преимуществ программирования микроконтроллеров является детерминированное выполнение операций — время выполнения каждой инструкции известно точно, что критически важно для систем реального времени. Однако это достигается за счёт ограниченных ресурсов: микроконтроллеры имеют мало памяти (обычно килобайты) и работают на сравнительно низких частотах процессора (мегагерцы вместо гигагерц).

Определение

Прошивка (firmware) — программа, записанная в память микроконтроллера и запускаемая при включении или сбросе платы. Приложение под Linux хранится как исполняемый файл и запускается или останавливается средствами операционной системы.

В отличие от этого, разработка под операционную систему, как на Lichee RV Dock, происходит на другом уровне абстракции. Здесь программа работает поверх ядра Linux, которое предоставляет абстракцию оборудования через драйверы и системные вызовы. Это упрощает разработку, но добавляет накладные расходы. Операционная система обеспечивает многозадачность — несколько программ могут работать одновременно, разделяя ресурсы процессора. Системы с ОС обычно имеют значительно больше ресурсов: больше памяти, более высокую производительность процессора и развитую файловую систему. Программа в таком контексте представляет собой исполняемый файл, который можно запускать и останавливать по мере необходимости.

10.3.3 Архитектура микроконтроллера ATmega328P

Arduino Uno основана на микроконтроллере ATmega328P: - **Тактовая частота:** 16 МГц - **Флэш-память:** 32 КБ (для программы) - **ОЗУ:** 2 КБ (для данных) - **Энергонезависимая память:** 1 КБ EEPROM - **Периферия:** таймеры, АЦП, UART, SPI, I2C, GPIO

10.3.4 Загрузка прошивки в Arduino

Процесс программирования Arduino состоит из нескольких этапов. Сначала разработчик пишет код в среде Arduino IDE, используя упрощённый диалект C/C++ с готовыми библиотеками для работы с периферией. Затем среда разработки компилирует код в машинные инструкции для архитектуры AVR, оптимизируя его под ограниченные ресурсы микроконтроллера.

После компиляции происходит загрузка прошивки через USB-порт с использованием встроенного программатора. Как только прошивка успешно загружена, программа начинает выполняться сразу же — микроконтроллер переходит к выполнению кода без необходимости перезагрузки или дополнительных действий.

Определение

Загрузчик (bootloader) — служебная программа в памяти микроконтроллера, которая принимает новую прошивку и передаёт управление основной программе.

При включении Arduino загрузчик первым делом проверяет, не поступила ли команда на загрузку новой прошивки. Если такой команды нет, он просто передаёт управление основной программе. Если же пользователь пытается загрузить новый скетч, загрузчик активирует режим программирования: он принимает данные через последовательный порт (UART), проверяет их целостность, и записывает во флэш-память микроконтроллера. После успешной записи загрузчик выполняет верификацию — сравнивает записанные данные с полученными, и только затем запускает новую программу. Эта система позволяет программировать Arduino без использования внешних программаторов, что значительно упрощает процесс разработки и отладки.

10.3.5 Основные концепции программирования Arduino

Определение

Скетч (sketch) — исходная программа для Arduino. Её структура включает функции `setup()` и `loop()`.

```
void setup() {  
    // Выполняется один раз при старте  
    // Инициализация пинов, настройка периферии  
}  
  
void loop() {  
    // Выполняется бесконечно в цикле  
    // Основная логика программы
```

}

Определение

Цифровые входы и выходы (GPIO) — контакты общего назначения, режим которых задаётся функцией `pinMode()`.

Базовые режимы:

- **OUTPUT** — пин работает как выход: микроконтроллер управляет напряжением на пине (HIGH — 5 В или LOW — 0 В) для управления светодиодами, реле и другими устройствами
- **INPUT** — пин работает как вход: микроконтроллер считывает внешний сигнал (HIGH или LOW) от кнопок, датчиков и других схем

Помимо базовых, существуют дополнительные режимы, которые настраивают внутренние резисторы микроконтроллера:

- **INPUT_PULLUP** — вход с подтяжкой к питанию. Внутренний резистор (около 20–50 кОм) подтягивает пин к высокому уровню (HIGH). Когда внешнее устройство (например, кнопка) соединяет пин с землёй, читается LOW. Этот режим удобен для кнопок и контактов — не нужен внешний резистор, а по умолчанию пин всегда находится в известном (высоком) состоянии

Определение

INPUT_PULLDOWN — режим входа с внутренней подтяжкой к земле, доступный не на всех микроконтроллерах. ATmega328P на Arduino Uno его не поддерживает: используйте внешний резистор или INPUT_PULLUP с инвертированной логикой.

Определение

Прерывание (interrupt) — механизм, при котором процессор временно приостанавливает основной код и выполняет обработчик в ответ на аппаратное или программное событие.

После завершения обработчика процессор продолжает выполнение прерванного кода.

Различают два основных типа прерываний. **Аппаратные прерывания** возникают от внешних устройств и периферии микроконтроллера — сигналов на цифровых пинах, таймеров, UART, SPI, I2C и других коммуникационных интерфейсов. **Программные прерывания** инициируются самим кодом через специальные инструкции процессора — они используются для вызова функций операционной системы или приоритетного переключения задач.

Режимы срабатывания аппаратных прерываний: При настройке внешнего прерывания на пине можно выбрать один из четырёх режимов: - **LOW** — прерывание срабатывает постоянно, пока на пине низкий уровень - **CHANGE** — прерывание срабатывает при любом изменении сигнала (с HIGH на LOW и наоборот) - **RISING** — прерывание срабатывает только при переходе с LOW на HIGH (восходящий фронт) - **FALLING** — прерывание срабатывает только при переходе с HIGH на LOW (нисходящий фронт) На Arduino Uno внешние прерывания доступны на пинах 2 и 3.

Внимание

Обработчик прерывания (ISR) должен завершаться как можно быстрее. Не вызывайте в нём `delay()`, `Serial.print()` и другие блокирующие или зависящие от прерываний функции. Переменные, совместно используемые ISR и основным кодом, объявляйте как `volatile`; для многобайтовых данных дополнительно обеспечивайте атомарный доступ.

10.4 Порядок выполнения

10.4.1 Подготовка оборудования

1. **Arduino Uno** — подключите к компьютеру через USB
2. **Светодиодная мезонинная плата** — вставляется в соответствии с распиновкой Arduino
3. **Соединительные перемычки** — для соединения оборудования
4. **Lichee RV Dock** — с программой генерации меандра на выводе GPIO 144 из лабораторной №9
5. **Преобразователь логических уровней**, размещённый на макетной плате

10.4.2 Установка и настройка Arduino IDE

1. Установите Arduino IDE из репозитория с помощью пакета *arduino*
2. Выберите правильную плату: **Инструменты** → **Плата** → **Arduino Uno**
3. Выберите правильный порт: **Инструменты** → **Порт** (например, COM3 или /dev/ttyUSB0)

10.4.3 Первая программа: «Моргающий светодиод»

Создайте новый скетч и напишите простейшую программу:

```
// Подключение светодиода к пину 13

void setup() {
  // Настройка пина как выхода
```

```

    pinMode(LED_BUILTIN, OUTPUT);
}

void loop() {
    // Включить светодиод
    digitalWrite(LED_BUILTIN, HIGH);
    delay(1000); // Ждать 1 секунду

    // Выключить светодиод
    digitalWrite(LED_BUILTIN, LOW);
    delay(1000); // Ждать 1 секунду
}

```

`pinMode()` настраивает контакт как вход или выход, `digitalWrite()` устанавливает высокий или низкий логический уровень, а `delay()` приостанавливает выполнение на указанное количество миллисекунд.

Проверка

Проверка программы. После загрузки скетча встроенный светодиод должен включаться и выключаться с интервалом одна секунда.

10.4.4 Работа с прерываниями

Прерывания позволяют Arduino реагировать на внешние события без постоянной проверки в основном цикле.

Пины 2 и 3 на Arduino Uno могут быть использованы для работы с аппаратными прерываниями.

Программа с прерыванием:

```

// Пин для прерывания (порт 2 поддерживает прерывание INT0)
#define INTERRUPT_PIN 2

// Пин для светодиода
#define LED_PIN 5

// Счётчик прерываний
volatile int interruptCounter = 0;

void setup() {
    // Настройка пина светодиода
    pinMode(LED_PIN, OUTPUT);

    // Настройка пина прерывания как входа
    pinMode(INTERRUPT_PIN, INPUT);

    // Настройка прерывания:
    // - Пин: INTERRUPT_PIN (2)
    // - Функция обработки: handleInterrupt

```

```

// - Режим: FALLING (срабатывает при переходе с HIGH на LOW)
// Функция digitalPinToInterrupt преобразует номер пина в номер прерывания на
↳ нем

// Таким образом пользовательская функция handleInterrupt привязывается к
↳ прерыванию
// на пине INTERRUPT_PIN в режиме FALLING
attachInterrupt(digitalPinToInterrupt(INTERRUPT_PIN), handleInterrupt,
↳ FALLING);

// Инициализация последовательного порта для отладки
Serial.begin(115200);
Serial.println("Arduino готов к работе");
Serial.println("Ожидание прерываний от Lichee...");
}

void loop() {
    // Основной цикл может выполнять другие задачи
    digitalWrite(LED_PIN, HIGH);
    delay(500);
    digitalWrite(LED_PIN, LOW);
    delay(500);

    // Вывод счётчика прерываний
    static int lastCount = 0;
    if (interruptCounter != lastCount) {
        Serial.print("Получено прерываний: ");
        Serial.println(interruptCounter);
        lastCount = interruptCounter;
    }
}

// Функция обработки прерывания
void handleInterrupt() {
    interruptCounter++; // Увеличиваем счётчик
}

```

10.5 Задание

Ознакомьтесь с ключевыми понятиями, протестируйте элементарные программы и выполните следующие подзадачи.

10.5.1 Программа «Светофор»

Создайте программу, имитирующую бегущий огонёк на мезонинной плате путём мигания светодиодами, соединёнными с пинами 3, 5, 6, 9 и 10. Огонёк должен бежать от пина 3 к пину 10 и обратно, минуя пины, которые не соединены со светодиодами.

10.5.2 Программа «Повторитель меандра»

На Lichee RV Dock запустите программу для генерации меандра из лабораторной №9.

Опасность повреждения

Между Arduino Uno с логическими уровнями 5 В и Lichee RV Dock с уровнями 3,3 В обязателен преобразователь уровней и общий GND. Подайте Lichee 3,3 В на LV, Arduino 5 В на HV; соедините сигнал GPIO 144 Lichee с LV1, а соответствующий HV1 — с пином 2 Arduino. Не соединяйте выводы питания 3,3 В и 5 В друг с другом.

Соедините через логический преобразователь уровней пины в следующем соответствии:

- Lichee GPIO 144 (3,3 В) → LV1; HV1 → Arduino pin 2
- Lichee 3.3V → LV преобразователя
- Arduino 5V → HV преобразователя
- GND Lichee, Arduino и преобразователя → общий GND

Преобразователь логических уровней согласует сигналы устройств с разными логическими напряжениями. В этой схеме он преобразует сигнал Lichee с уровня 3,3 В в уровень, корректный для входа Arduino; двунаправленные каналы также защищают сторону Lichee при обмене в обратном направлении. На рисунке ниже приведён пример подключения Arduino к Lichee через макетную плату с преобразователем уровней:

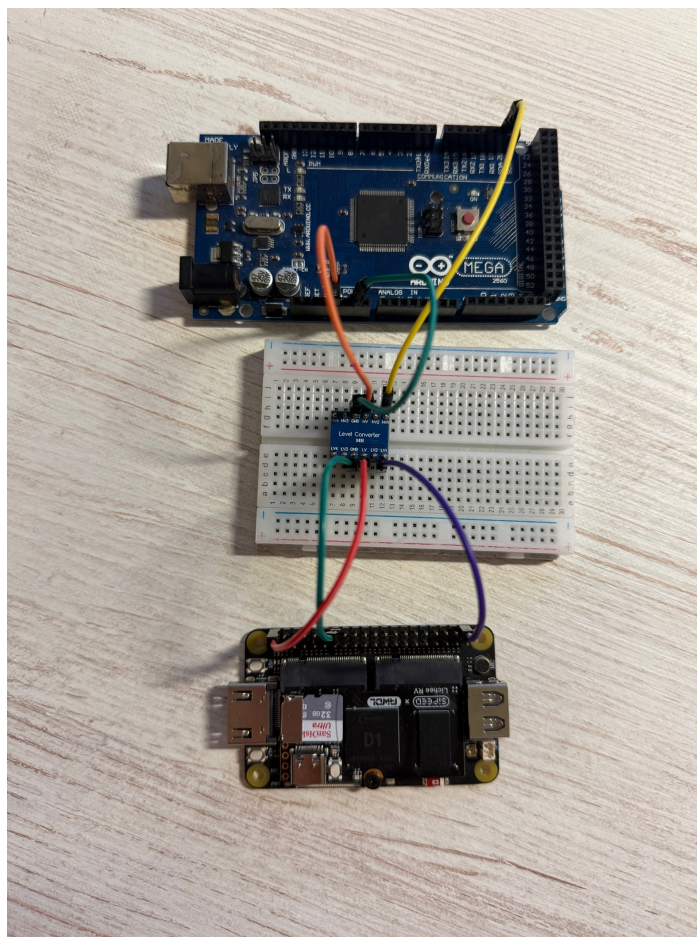


Рис. 10.1 – Пример подключения Lichee RV Dock к Arduino через преобразователь логических уровней

Затем напишите программу, которая на пине 4 повторяет генерируемый на Lichee меандр, работая с прерыванием на пине №2 в режиме CHANGE.

Анализ сигналов логическим анализатором:

Используя навык работы с логическим анализатором из лабораторной №9, проанализируйте сигналы:

1. **Подключите логический анализатор** к пину GPIO 144 на Lichee и пину 4 на Arduino
2. **Запустите PulseView**
3. **Наблюдайте прерывания** — сигналы от Lichee RV Dock
4. Убедитесь в корректности работы программы для Arduino
5. **Измерьте временные параметры** — длительность импульсов, интервалы и временную разницу между фронтами сигналов

Проверка

Проверка результата. На логическом анализаторе сигнал пина 4 Arduino должен повторять сигнал GPIO 144 Lichee. Измерьте задержку между соответствующими фронтами.

10.6 Контрольные вопросы

1. В чём ключевое отличие программирования микроконтроллера без ОС от разработки под операционную систему?
2. Какую роль выполняет загрузчик в Arduino?
3. Какие режимы срабатывания аппаратных прерываний доступны на ATmega328P и чем они отличаются?
4. Почему переменные, используемые и в обработчике прерывания, и в основном цикле, должны объявляться с ключевым словом `volatile`?
5. Зачем нужен преобразователь логических уровней при соединении Arduino с уровнями 5 В и Lichee RV Dock с уровнями 3,3 В?
6. Почему в обработчике прерывания не рекомендуется использовать функции `delay()` и `Serial.print()`?

10.7 Требования к отчёту

Отчёт должен содержать:

- цель работы;
- исходные тексты разработанных скетчей;
- схему или фотографию подключения Arduino и Lichee RV Dock;
- результаты измерения сигналов логическим анализатором;
- ответы на контрольные вопросы;
- краткий вывод.

Продемонстрируйте работу преподавателю и отправьте отчёт в согласованном формате до установленного срока.

11 Лабораторная №11.

Интеграция Arduino и Lichee RV Dock по SPI с датчиком BME280

11.1 Цель работы

Освоить совместную работу одноплатного компьютера Lichee RV Dock и микроконтроллера Arduino Uno в единой распределённой системе. Научиться считывать данные с датчика BME280 по I2C на Arduino, передавать их на Lichee через SPI-интерфейс и отображать на OLED-экране по I2C — тем самым объединяя знания и навыки, полученные в лабораторных №7 (SPI), №8 (I2C, OLED) и №10 (Arduino).

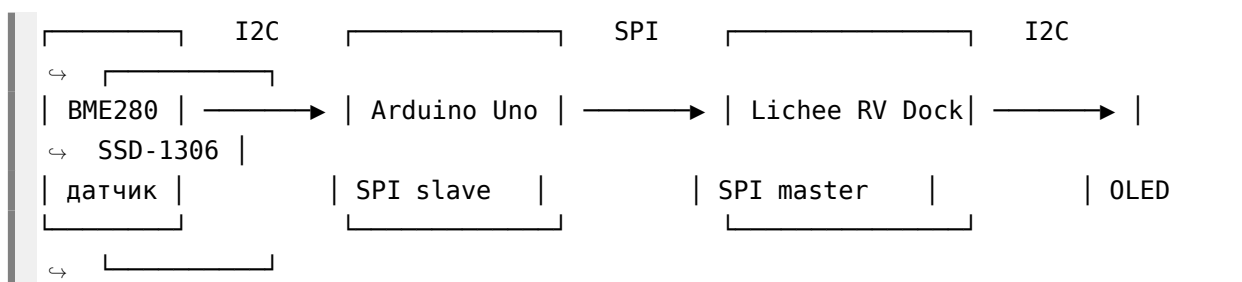
11.2 Оборудование и исходные данные

- Lichee RV Dock с настроенными SPI1 и I2C TWI2;
- Arduino Uno и USB-кабель;
- датчик BME280;
- OLED-дисплей SSD-1306;
- преобразователь логических уровней 5↔3,3 В;
- соединительные провода и макетная плата;
- компьютер с Arduino IDE, PulseView и логическим анализатором.

11.3 Ключевые понятия

11.3.1 Архитектура системы

Общая схема обмена данными:



Архитектура объединяет три интерфейса и два вычислительных устройства:

1. **Arduino → BME280 (I2C)** — чтение температуры, влажности и давления с датчика
2. **Arduino → Lichee (SPI)** — передача собранных данных на одноплатник; Arduino работает как SPI-ведомый (slave)
3. **Lichee → OLED (I2C)** — отображение полученных данных на экране SSD-1306 (лабораторная №8)

Передача по SPI выполняется серией из 12 однобайтовых транзакций — по одной на каждый байт трёх чисел формата float. Такой подход выбран потому, что AVR-микроконтроллер ATmega328P в режиме SPI-slave имеет однобайтовый буфер передачи: при непрерывном тактировании нескольких байт (burst) shift register не перезагружается из буфера SPDR автоматически. Использование 12 отдельных транзакций (SS↓→1 байт→SS↑) гарантирует перезагрузку shift register из SPDR в начале каждой из них.

Внимание

Каждый байт передавайте отдельной SPI-транзакцией: опустить SS, передать ровно один байт, затем поднять SS. Не объединяйте 12 байт в одну транзакцию с постоянно активным SS, иначе описанный протокол Arduino-slave сформирует неверный ответ.

11.3.2 Датчик BME280 и библиотека GyverBME280

Определение

BME280 — цифровой датчик температуры, влажности и атмосферного давления производства Bosch Sensortec с интерфейсами I2C и SPI. В этой работе используется I2C.

Характеристики: - Диапазон температур: от -40 до $+85^{\circ}\text{C}$, точность $\pm 0.5^{\circ}\text{C}$ - Диапазон влажности: от 0 до 100%, точность $\pm 3\%$ - Диапазон давления: от 300 до 1100 гПа, точность ± 1 гПа - Напряжение питания: 1.8–3.6 В - I2C-адрес по умолчанию: **0x76** (вывод SDO замкнут на GND)

Опасность повреждения

В этой работе питайте BME280 только от 3.3V. Не подавайте 5 В на VCC датчика или линии I2C.

Подключение BME280 к Arduino:

BME280		Arduino Uno
-----		-----
VCC	→	3.3V
GND	→	GND
SDA	→	A4 (SDA)
SCL	→	A5 (SCL)

Библиотека GyverBME280

В отличие от громоздкой Adafruit BME280 (требуется Adafruit Sensor, Adafruit BusIO и пр.), GyverBME280 — лёгкая библиотека, занимающая минимум памяти и не требующая дополнительных зависимостей. Установка через менеджер библиотек Arduino IDE: «GyverBME280».

Проверка

Проверка библиотеки. Если библиотека не находится или её API отличается, убедитесь, что в менеджере Arduino IDE установлена именно GyverBME280, и сверьтесь с примером из установленной версии.

Основные методы:

```
#include <GyverBME280.h>
GyverBME280 bme;

bme.begin();                // инициализация (I2C-адрес 0x76 по умолчанию)
bme.readTemperature();      // температура в °C (float)
bme.readHumidity();         // влажность в % (float)
bme.readPressure();         // давление в Па (float), для гПа разделить на 100
```

Простой пример чтения датчика:

```
#include <GyverBME280.h>
GyverBME280 bme;

void setup() {
  Serial.begin(115200);
  Wire.begin();
  if (bme.begin()) {
    Serial.println("BME280 OK");
  } else {
    Serial.println("ERROR: BME280 not found!");
  }
}

void loop() {
  Serial.print("T: "); Serial.print(bme.readTemperature(), 1);
  Serial.print(" C  H: "); Serial.print(bme.readHumidity(), 1);
  Serial.print(" %  P: "); Serial.println(bme.readPressure() / 100.0, 1);
  delay(1000);
}
```

11.3.3 SPI-обмен на Arduino в режиме ведомого устройства

11.3.3.1 Аппаратная организация SPI

Arduino Uno построена на микроконтроллере ATmega328P, который имеет аппаратный модуль SPI. Контакты:

Функция	Пин Arduino Uno	В slave режиме
SS (Slave Select)	10	вход (LOW активирует слейв)
MOSI (Master Out Slave In)	11	вход
MISO (Master In Slave Out)	12	выход
SCK (Serial Clock)	13	вход

В режиме слейва Arduino не генерирует тактовый сигнал — SCK приходит от мастера (Lichee). Сигнал SS, опущенный в LOW, активирует слейв и сообщает ему о начале транзакции. По фронтам SCK происходит одновременный сдвиг битов: мастер выставляет бит на MOSI, слейв — на MISO.

11.3.3.2 Регистры SPI

Работа модуля SPI определяется тремя регистрами:

Определение

SPCR (SPI Control Register) — регистр управления аппаратным модулем SPI.

Бит	Имя	Назначение
7	SPIE	SPI Interrupt Enable — разрешение прерывания по завершению передачи байта
6	SPE	SPI Enable — включение модуля SPI
5	DORD	Data Order: 0=MSB first, 1=LSB first
4	MSTR	Master/Slave Select: 0=Slave, 1=Master
3	CPOL	Clock Polarity: 0=SCK в покое LOW
2	CPHA	Clock Phase: 0=захват по переднему фронту
1-0	SPR	Clock Rate (только для мастера)

Для включения слейва с прерываниями (SPI mode 0):

```
SPCR = _BV(SPE) | _BV(SPIE); // 0b11000000: SPI вкл, прерывания вкл, mode 0
```

Определение

SPSR (SPI Status Register) — регистр состояния SPI. Бит SPIF устанавливается аппаратно по завершении передачи байта; для его сброса

читают SPSR, а затем обращаются к SPDR.

Определение

SPDR (SPI Data Register) — регистр данных SPI. Записанный в него байт передаётся в следующей транзакции, а чтение возвращает последний принятый байт.

11.3.3.3 Прерывание SPI_STC_vect

Когда байт полностью передан (8 тактов SCK), аппаратно устанавливается флаг SPIF, и если бит SPIE в SPCR равен 1, генерируется прерывание. Обработчик помечается макросом ISR(SPI_STC_vect).

Ключевой момент для слейва: **до начала следующей однобайтовой транзакции** (пока мастер не опустил SS в LOW) необходимо записать в SPDR байт, который должен быть отправлен мастеру. Если мастер поднимет SS, а затем опустит снова — shift register загрузит содержимое SPDR заново.

Почему в работе используются отдельные транзакции?

При непрерывной передаче следующий ответный байт должен попасть в SPDR до начала его тактирования. Если обработчик прерывания не успеет подготовить данные, мастер получит устаревший или повторный байт. В этой работе 12 отдельных однобайтовых транзакций с поднятием SS между ними дают обработчику гарантированное время на подготовку следующего значения.

11.3.3.4 Базовый пример SPI-slave

Минимальный скетч, отвечающий мастеру фиксированным байтом:

```
#include <SPI.h>

volatile byte g_response = 0xA5;

ISR(SPI_STC_vect) {
    (void)SPDR;          // чтение принятого байта (сброс SPIF)
    SPDR = g_response;    // подготовка ответа на следующую транзакцию
}

void setup() {
    Serial.begin(115200);
    pinMode(SS, INPUT_PULLUP);
    pinMode(SCK, INPUT);
    pinMode(MOSI, INPUT);
    pinMode(MISO, OUTPUT);
    SPCR = _BV(SPE) | _BV(SPIE);
    sei();
    SPDR = g_response;    // предзагрузка первого байта
```

```

Serial.println("SPI slave ready");
}

void loop() {
    // прерывание делает всю работу
}

```

Пояснение: - `pinMode(SS, INPUT_PULLUP)` — встроенная подтяжка к HIGH, чтобы SS не плавало и не активировало слейв ложно - `SPCR = _BV(SPE) | _BV(SPIE)` — включение SPI в режиме слейва с прерываниями (MSTR=0 по умолчанию) - `sei()` — глобальное разрешение прерываний - `SPDR = g_response` — предзагрузка первого ответного байта до того, как мастер начнёт первую транзакцию. Без этого первый байт будет случайным - В ISR: чтение SPDR (обязательно для сброса SPIF!), затем запись следующего ответного байта

11.3.4 SPI на Lichee RV Dock

Краткое напоминание: для работы SPI на Lichee необходимо:

1. Ядро с поддержкой SPI_SUN6I, DMA_SUN6I, SPI_SPIDEV
2. Device Tree с активированным `&spi1` на пинах PD10–PD15
3. Устройство `/dev/spidev1.0` (проверить: `ls -la /dev/spidev*`)

Подробная процедура настройки описана в лабораторной №7.

Проверка

Проверка устройства. Номер SPI-шины зависит от Device Tree и сборки ядра. Выполните `ls -la /dev/spidev*` и укажите найденные номера шины и устройства вместо значений из примера.

Пины SPI1 на Lichee RV Dock:

Сигнал	Пин Lichee	GPIO
CS	—	PD10
SCK	—	PD11
MOSI	—	PD12
MISO	—	PD13

Python-библиотека spidev:

```

import spidev

spi = spidev.SpiDev()
spi.open(bus=1, device=0)      # /dev/spidev1.0
spi.max_speed_hz = 1000000    # 1 МГц

```

```
spi.mode = 0                                # CPOL=0, CPHA=0

rx = spi.xfer2([0x00])[0]                  # отправить один байт, получить ответ
spi.close()
```

xfer2() удерживает CS активным в пределах одного вызова и освобождает после его завершения. Поэтому 12 отдельных вызовов xfer2([0x00]) формируют 12 однобайтовых транзакций.

11.3.5 I2C OLED SSD-1306 на Lichee (лабораторная №8)

Дисплей SSD-1306 — монохромный OLED 128×64 пикселя с I2C-интерфейсом. Подробная процедура настройки I2C на Lichee описана в лабораторной №8.

Подключение:

Опасность повреждения

OLED на I2C Lichee в этой работе питайте от 3.3V. Использовать 5 В можно только при подтверждённом для конкретного модуля согласовании питания и уровней линий SDA/SCL.

SSD1306		Lichee RV Dock
-----		-----
VCC	→	3.3V
GND	→	GND
SCL	→	TWI2_SCL (PE12)
SDA	→	TWI2_SDA (PE13)

I2C-адрес: 0x3C (стандартный для SSD-1306).

Проверка: `ls -la /dev/i2c-*`

Python-библиотека luma-oled:

```
from luma.core.interface.serial import i2c
from luma.oled.device import ssd1306
from luma.core.render import canvas
from PIL import ImageFont

serial = i2c(port=2, address=0x3C)
oled = ssd1306(serial)
font = ImageFont.truetype("/usr/share/fonts/truetype/dejavu/DejaVuSans.ttf",
↪ 14)

with canvas(oled) as draw:
    draw.text((0, 0), "Hello!", fill="white", font=font)
```

11.3.6 Согласование логических уровней

Arduino Uno работает от 5 В, Lichee RV Dock — от 3.3 В. Прямое соединение линий данных недопустимо: 5 В на входе Lichee может вывести GPIO-пины Allwinner D1 из строя.

Для согласования используется **логический преобразователь уровней** на макетной плате. Принцип работы: двунаправленный параллельный преобразователь на MOSFET-транзисторах, у которого каждая пара каналов имеет сторону HV (high voltage, 5 В) и LV (low voltage, 3.3 В).

Питание преобразователя:

Вывод преобразователя	Подключить к
HV	Arduino 5V
LV	Lichee 3.3V
GND	Общий GND (Arduino + Lichee)

Направления сигналов:

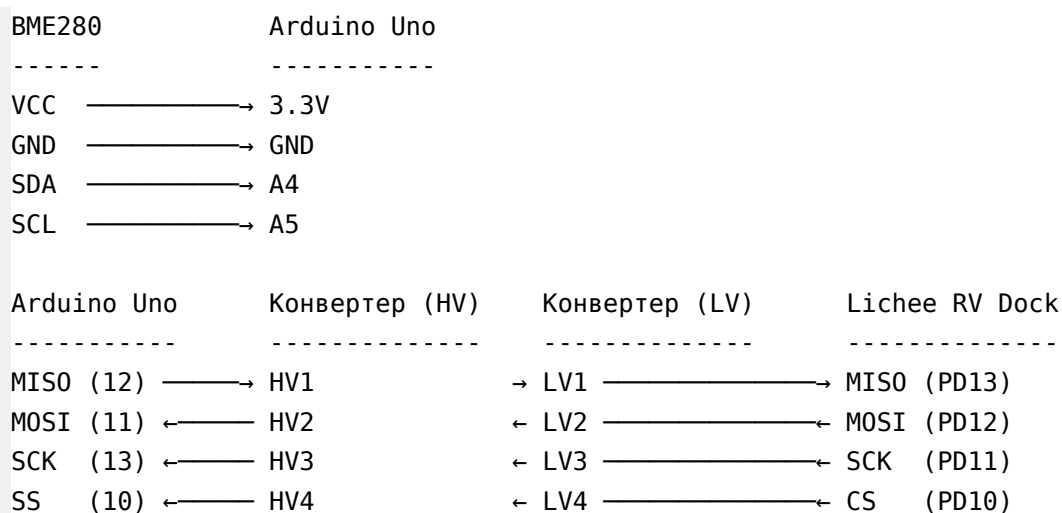
Сигнал	Направление	Подключение Arduino	Подключение Lichee
MOSI	Lichee → Arduino	MOSI (11) ↔ HV2	MOSI (PD12) ↔ LV2
SCK	Lichee → Arduino	SCK (13) ↔ HV3	SCK (PD11) ↔ LV3
SS/CS	Lichee → Arduino	SS (10) ↔ HV4	CS (PD10) ↔ LV4
MISO	Arduino → Lichee	MISO (12) ↔ HV1	MISO (PD13) ↔ LV1

Обратите внимание: для сигналов, идущих от Lichee к Arduino (MOSI, SCK, SS), задействована сторона LV как вход, HV как выход. Для сигнала MISO (Arduino → Lichee) — наоборот: HV как вход, LV как выход.

Полная схема соединений:

Опасность повреждения

До включения полной схемы установите преобразователь 5↔3,3 В между всеми линиями SPI Arduino и Lichee: HV подключите к Arduino 5 В, LV — к Lichee 3,3 В. GND Arduino, Lichee, преобразователя и периферии объедините в общий GND.



Питание конвертера:
 Arduino 5V —→ HV
 Lichee 3.3V —→ LV
 Arduino GND —→ GND ← Lichee GND

SSD-1306 OLED Lichee RV Dock

 VCC —→ 3.3V
 GND —→ GND
 SDA —→ TWI2_SDA (PE13)
 SCL —→ TWI2_SCL (PE12)

Все GND (Arduino, Lichee, конвертер и периферия) должны быть соединены в общий GND — без этого сигналы не будут иметь общего уровня отсчёта и SPI не заработает.

Частая ошибка: путать MOSI и MISO при соединении. Правило простое — одноимённые пины соединяются друг с другом: MOSI↔MOSI, MISO↔MISO. НЕ перекрещивать!

11.3.7 Отладка логическим анализатором

Для проверки корректности SPI-обмена рекомендуется использовать логический анализатор (fx2lafw) и программу PulseView (лабораторная №9).

Опасность повреждения

Подключайте логический анализатор со стороны Lichee, после преобразователя уровней, где амплитуда сигналов равна 3,3 В. Обязательно соедините GND анализатора с общим GND схемы; не подавайте на его входы 5 В без подтверждения допустимого диапазона.

Проверка

Проверка обмена. Сначала добейтесь обмена тестовым скриптом, затем подтвердите в PulseView режим SPI и границы однобайтовых транзакций по линиям CS, SCK, MOSI и MISO.

Подключение анализатора (рекомендуется к пинам Lichee — все сигналы 3.3 В, безопасно для fx2lafw):

Канал анализатора	Пин Lichee	Сигнал
D0	PD10	CS
D1	PD11	SCK
D2	PD12	MOSI
D3	PD13	MISO
GND	GND	общий GND

Настройка PulseView:

1. Частота дискретизации: не менее **4 МГц** для SPI на 1 МГц.
2. Лимит семплов: 1-2М
3. Добавить декодер: Add protocol decoder → SPI
4. Привязать каналы: **CS#** = D0, **SCK** = D1, **MOSI** = D2, **MISO** = D3
5. Параметры декодера:
 - CS# polarity: **Active low**
 - CPOL: **0** (SCK в покое LOW)
 - CPHA: **0** (захват по переднему/rising фронту)
 - Bit order: **MSB first**
 - Wordsize: **8 bits**

Что должно быть видно на экране:

При работающей системе каждые 2 секунды появляется серия из 12 SPI-транзакций. В каждой транзакции CS переходит в LOW, проходят 8 тактов SCK, затем CS возвращается в HIGH. При скорости 1 МГц передача восьми битов занимает около 8 мкс без учёта накладных расходов. На MOSI передаются нулевые байты, а на MISO — байты данных датчика.

11.4 Порядок выполнения

11.4.1 Подготовка оборудования

Убедитесь, что:

- **Lichee RV Dock:** настроен SPI (лабораторная №7), настроен I2C (лабораторная №8)
- Присутствуют файлы устройств:

```
ls -la /dev/spidev*
ls -la /dev/i2c-*
```
- Установлены Python-библиотеки:

```
apt-get install python3-spidev python3-module-luma-oled
```
- **Arduino Uno:** подключена к компьютеру, Arduino IDE установлена (apt-get install arduino)
- В менеджере библиотек Arduino IDE установлена **GyverBME280**

11.4.2 Проверка датчика BME280

Соберите часть схемы: подключите BME280 к Arduino (VCC→3.3V, GND→GND, SDA→A4, SCL→A5).

Загрузите тестовый скетч в Arduino:

```
#include <GyverBME280.h>
GyverBME280 bme;

void setup() {
  Serial.begin(115200);
  Wire.begin();

  if (bme.begin()) {
    Serial.println("BME280 OK (I2C 0x76)");
  } else {
    Serial.println("ERROR: BME280 not found!");
    Serial.println("Check SDA(A4), SCL(A5), 3.3V, GND.");
    while (1);
  }
}

void loop() {
  float t = bme.readTemperature();
  float h = bme.readHumidity();
  float p = bme.readPressure() / 100.0;

  Serial.print("T: "); Serial.print(t, 1);
  Serial.print(" C  H: "); Serial.print(h, 1);
  Serial.print(" %  P: "); Serial.println(p, 1);

  delay(1000);
}
```

Откройте **Инструменты** → **Монитор порта** со скоростью 115200 бод.

Проверка

Ожидаемый результат. В мониторе порта появляются показания температуры, влажности и давления. При сообщении об ошибке проверьте SDA, SCL, питание 3,3 В и GND.

11.4.3 Проверка SPI-соединения

На этом шаге мы убедимся, что SPI-обмен между Arduino и Lichee работает. Arduino будет отвечать фиксированным байтом, Python на Lichee — читать его.

11.4.3.1 Скетч для Arduino

```
#include <SPI.h>
```

```

volatile bool g_done = false;

ISR(SPI_STC_vect) {
    (void)SPDR;          // читаем принятый байт
    SPDR = 0xA5;         // сразу готовим ответ на следующую транзакцию
    g_done = true;
}

void setup() {
    Serial.begin(115200);
    delay(500);
    pinMode(LED_BUILTIN, OUTPUT);

    pinMode(SS, INPUT_PULLUP);
    pinMode(SCK, INPUT);
    pinMode(MOSI, INPUT);
    pinMode(MISO, OUTPUT);

    SPCR = _BV(SPE) | _BV(SPIE);
    sei();

    SPDR = 0xA5;         // предзагрузка первого байта

    Serial.println("SPI slave ready. Sending 0xA5.");
}

void loop() {
    if (g_done) {
        g_done = false;

        static bool led = false;
        led = !led;
        digitalWrite(LED_BUILTIN, led);
    }
}

```

Загрузите скетч. Светодиод L на плате Arduino должен начать мигать при каждом SPI-запросе.

11.4.3.2 Тестовый Python-скрипт на Lichee

Создайте файл test_spi.py:

```

#!/usr/bin/env python3
import spidev, time

spi = spidev.SpiDev()
spi.open(1, 0)          # /dev/spidev1.0
spi.max_speed_hz = 1000000
spi.mode = 0

```

```

print("SPI test: Ctrl+C to stop")
try:
    while True:
        rx = spi.xfer2([0x00])[0]
        print(f"RX: {rx:02X} (0x{rx:02X})")
        time.sleep(0.5)
except KeyboardInterrupt:
    spi.close()
    print("Done.")

```

Запустите `python3 test_spi.py`.

Проверка

Ожидаемый результат. В консоли повторяется RX: A5. Если выводится RX: FF, проверьте MISO, общий GND, направления каналов преобразователя уровней и его питание.

11.4.3.3 Отладка логическим анализатором

Подключите логический анализатор к пинам Lichee (PD10=D0, PD11=D1, PD12=D2, PD13=D3, GND). Запустите захват в PulseView с частотой не менее 4 МГц одновременно с тестовым скриптом. Вы должны увидеть:

- CS (D0): короткие импульсы LOW (активный уровень)
- SCK (D1): 8 тактов на каждый CS-импульс
- MOSI (D2): все биты = 0
- MISO (D3): биты = 10100101 (0xA5)

11.4.4 Сборка полной схемы

Соберите схему полностью, руководствуясь таблицей соединений из подготовительного материала:

1. Подключите BME280 к Arduino (I2C)
2. Подключите Arduino к конвертеру уровней (SPI)
3. Подключите конвертер к Lichee (SPI)
4. Подайте питание на конвертер: HV = Arduino 5V, LV = Lichee 3.3V
5. Соедините GND всех устройств в общий GND
6. Подключите OLED SSD-1306 к 3.3V и I2C Lichee

На рисунке ниже приведен пример такого соединения:

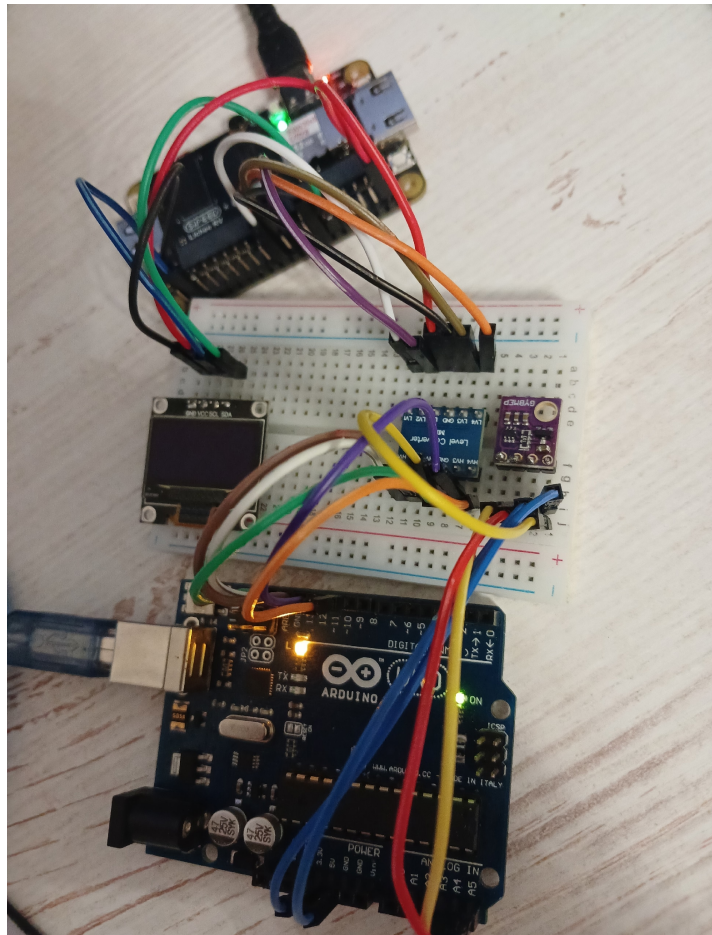


Рис. 11.1 – Пример полной схемы подключения

11.4.5 Итоговые программы

11.4.5.1 Итоговый скетч для Arduino

Следующий листинг должен совпадать с каноническим примером `labs/examples/arduino_i2c_spi/arduino_spi_slave/arduino_spi_slave.ino`:

```
#include <GyverBME280.h>
#include <SPI.h>
GyverBME280 bme;
volatile float      g_temp      = 0.0;
volatile float      g_hum       = 0.0;
volatile float      g_press     = 0.0;
volatile bool       g_ok        = false;
volatile byte       g_buf[12];
volatile byte       g_idx       = 0;
volatile bool       g_done      = false;
volatile unsigned int g_cnt      = 0;

void buf_load() {
    float t = g_temp;
    float h = g_hum;
    float p = g_press;
    noInterrupts();
```

```

    if (g_ok) {
        memcpy((void*)(g_buf + 0), &t, 4);
        memcpy((void*)(g_buf + 4), &h, 4);
        memcpy((void*)(g_buf + 8), &p, 4);
    }
    g_idx = 0;
    SPDR = g_buf[0];
    g_done = false;
    interrupts();
}
ISR(SPI_STC_vect) {
    (void)SPDR;
    g_idx++;
    if (g_idx < 12) {
        SPDR = g_buf[g_idx];
    } else {
        g_idx = 0;
        g_done = true;
    }
    g_cnt++;
}
void setup() {
    Serial.begin(115200);
    delay(500);
    pinMode(LED_BUILTIN, OUTPUT);
    Serial.println(F("Arduino SPI Slave + BME280"));
    Serial.println(F("====="));
    Wire.begin();
    if (bme.begin()) {
        g_ok = true;
        Serial.println(F("BME280 OK (I2C 0x76)"));
    } else {
        g_ok = false;
        Serial.println(F("ERROR: BME280 not found!"));
        Serial.println(F("Check SDA(A4), SCL(A5), 3.3V, GND."));
    }
    pinMode(SS, INPUT_PULLUP);
    pinMode(SCK, INPUT);
    pinMode(MOSI, INPUT);
    pinMode(MISO, OUTPUT);
    SPCR = _BV(SPE) | _BV(SPIE);
    sei();
    buf_load();
    Serial.println(F("SPI slave ready. Waiting for Lichee master..."));
    Serial.println(F("====="));
}
void loop() {
    static uint32_t last = 0;
    uint32_t now = millis();

```



```

    if (now - last >= 1000) {
        if (g_ok) {
//            noInterrupts();
            g_temp = bme.readTemperature();
            g_hum = bme.readHumidity();
            g_press = bme.readPressure() / 100.0;
//            interrupts();
        }
        Serial.print(F("T:"));
        Serial.print(g_temp, 1);
        Serial.print(F(" H:"));
        Serial.print(g_hum, 1);
        Serial.print(F(" P:"));
        Serial.print(g_press, 1);
        Serial.print(F(" cnt:"));
        Serial.println(g_cnt);
        last = now;
    }
    if (g_done) {
        buf_load();
        digitalWrite(LED_BUILTIN, !digitalRead(LED_BUILTIN));
    }
}

```

setup() инициализирует датчик и SPI, а loop() раз в секунду читает BME280. Обработчик SPI продвигает индекс буфера; после отправки 12 байт buf_load() атомарно обновляет буфер и предзагружает первый байт следующего набора. Светодиод L переключается после передачи полного набора данных.

11.4.5.2 Итоговый Python-скрипт для Lichee

Следующий листинг должен совпадать с каноническим примером labs/examples/lichee_integration/lichee_spi_oled.py:

```

#!/usr/bin/env python3

import spidev
import struct
import time
from luma.core.interface.serial import i2c
from luma.oled.device import ssd1306
from luma.core.render import canvas
from PIL import ImageFont

SPI_BUS = 1
SPI_DEVICE = 0
SPI_SPEED = 1000000

```

```

I2C_PORT    = 2
I2C_ADDR    = 0x3C

UPDATE_INTERVAL = 2.0

try:
    font = ImageFont.truetype(
        "/usr/share/fonts/truetype/dejavu/DejaVuSans.ttf", 14)
except Exception:
    font = ImageFont.load_default()

def main():
    spi = spidev.SpiDev()
    spi.open(SPI_BUS, SPI_DEVICE)
    spi.max_speed_hz = SPI_SPEED
    spi.mode = 0
    print(f"SPI: /dev/spidev{SPI_BUS}.{SPI_DEVICE} @ {SPI_SPEED} Hz")

    serial = i2c(port=I2C_PORT, address=I2C_ADDR)
    oled = ssd1306(serial)
    oled.clear()
    print(f"OLED: I2C-{I2C_PORT}, addr 0x{I2C_ADDR:02X}")

    print("\n" + "=" * 44)
    print("SYSTEM RUNNING")
    print(" BME280 -> Arduino(I2C) -> Lichee(SPI) -> OLED(I2C)")
    print(" Press Ctrl+C to stop")
    print("=" * 44 + "\n")

    try:
        while True:
            rx = [spi.xfer2([0x00])[0] for _ in range(12)]
            data = bytes(rx)
            temp, hum, press = struct.unpack("<fff", data)

            print(f"T: {temp:6.1f} C    "
                  f"H: {hum:5.1f} %    "
                  f"P: {press:7.1f} hPa")

            with canvas(oled) as draw:
                draw.text((0, 0), f"Temp: {temp:.1f} C",
                           fill="white", font=font)
                draw.text((0, 20), f"Hum: {hum:.1f} %",
                           fill="white", font=font)
                draw.text((0, 40), f"Press: {press:.1f} hPa",
                           fill="white", font=font)

            time.sleep(UPDATE_INTERVAL)

```

```

except KeyboardInterrupt:
    print("\nStopped by user.")
finally:
    oled.clear()
    spi.close()
    print("SPI closed, OLED cleared.")

if __name__ == "__main__":
    main()

```

Скрипт открывает SPI (bus=1, device=0, 1 МГц, режим 0) и I2C OLED (port=2, addr=0x3C). В бесконечном цикле он выполняет 12 однобайтовых SPI-транзакций, распаковывает данные как три числа float с порядком байтов от младшего к старшему (<fff) и выводит значения в консоль и на OLED. Интервал обновления составляет 2 секунды. При нажатии Ctrl+C скрипт очищает экран и закрывает SPI.

11.4.6 Запуск и проверка

1. **Загрузите скетч** `arduino_spi_slave.ino` на Arduino (плата: Arduino Uno, порт: `/dev/ttyUSB0` или аналогичный)
2. **Проверьте Serial-монитор** (115200 бод) — должны появиться сообщения об инициализации BME280 и SPI, а затем показания датчика и счётчик ISR (cnt:)
3. **Запустите Python-скрипт** на Lichee:

```
python3 lichee_spi_oled.py
```

4. **Убедитесь в корректной работе:**

- В консоли Lichee: строки T: ... C H: ... % P: ... hPa с реальными данными
- На OLED-экране: три строки с температурой, влажностью и давлением
- На Arduino: светодиод L мигает, счётчик ISR растёт

5. **Проверьте логическим анализатором (опционально):** 12 CS-импульсов с 8 тактами SCK каждый, MOSI=0, MISO меняется

Проверка

Ожидаемый результат. В консоли Lichee и на OLED отображаются реалистичные показания BME280, светодиод L на Arduino переключается после обмена, а счётчик ISR растёт.

11.5 Задание

Ознакомьтесь с ключевыми понятиями и выполните следующие подзадачи.

1. **Проверить работу датчика.** Подключите BME280 к Arduino по I2C. Напишите скетч, читающий температуру, влажность и давление, и убедитесь в корректности показаний через Serial-монитор.
2. **Проверить SPI-соединение.** Соберите SPI-цепь (Arduino → конвертер уровней → Lichee). Напишите скетч, отвечающий фиксированным байтом 0xA5, и Python-скрипт на Lichee, читающий этот байт. Добейтесь уверенного приёма. Проверьте сигналы логическим анализатором в PulseView (опционально).
3. **Собрать полную схему.** Руководствуясь таблицей соединений, соберите схему целиком: BME280→Arduino, Arduino→Конвертер→Lichee (SPI), OLED→Lichee (I2C). Обеспечьте общий GND.
4. **Протестировать финальную программу.** Загрузите итоговый скетч в Arduino и запустите Python-скрипт на Lichee. На OLED-экране должны отображаться температура, влажность и давление с датчика BME280.
5. **Ответить на контрольные вопросы (письменно, в отчёте).**
6. **Ответить на вопросы преподавателя.**
7. **Продемонстрировать работу преподавателю.**

11.6 Контрольные вопросы

1. Почему для подключения BME280 к Arduino используется I2C, а не SPI?
2. Какую роль выполняет сигнал SS в SPI и почему на Arduino он настроен как INPUT_PULLUP?
3. Зачем в скетче выполняется предзагрузка `SPDR = g_buf[0]` до начала SPI-транзакций?
4. Почему для передачи 12 байт используется 12 отдельных однобайтовых транзакций, а не одна 12-байтовая? Что произойдёт при пакетной передаче?
5. Для чего нужен логический преобразователь уровней? Какие сигналы проходят через него в направлении HV→LV, а какие — LV→HV?
6. Как `struct.unpack("<fff", data)` интерпретирует 12 байт, полученных по SPI? Что означает префикс `<`?

7. Какие регистры ATmega328P управляют работой SPI в режиме ведомого устройства? Какие биты необходимо установить?
8. Как отдельные вызовы `spi.xfer2()` формируют однобайтовые SPI-транзакции?
9. Как проверить корректность SPI-обмена с помощью логического анализатора? Какие сигналы должны быть видны на каждом канале?
10. Почему важно соединить GND Arduino, Lichee и конвертера в общий GND? Что произойдёт, если этого не сделать?

11.7 Требования к отчёту

Отчёт должен содержать:

- цель работы;
- схему или фотографию полной аппаратной конфигурации;
- результаты отдельных проверок BME280 и SPI;
- исходные тексты итоговых программ или ссылки на использованные канонические примеры;
- снимок OLED с показаниями датчика;
- результаты проверки логическим анализатором, если она выполнялась;
- ответы на контрольные вопросы;
- краткий вывод.

Отправьте отчёт в согласованном формате до установленного срока.

12 Лабораторная №12. Протокол MQTT

12.1 Цель работы

Изучить протокол MQTT — лёгкий протокол обмена сообщениями для IoT-систем. Освоить архитектуру «издатель-подписчик» (Pub/Sub), научиться настраивать MQTT-брокер Mosquitto на одноплатном компьютере и создавать клиентские приложения на Python и C для публикации и приёма данных.

12.2 Оборудование и исходные данные

- Lichee RV Dock с ALT Linux и сетевым подключением;
- компьютер с ALT Linux в той же учебной сети;
- Mosquitto и клиентские утилиты;
- Python 3 с библиотекой Paho MQTT;
- компилятор GCC и libmosquitto-devel для дополнительной части.

12.3 Ключевые понятия

12.3.1 Протокол MQTT

Определение
MQTT (Message Queuing Telemetry Transport) — лёгкий протокол обмена сообщениями по модели «издатель-подписчик», применяемый в интернете вещей, автоматизации и телеметрии.

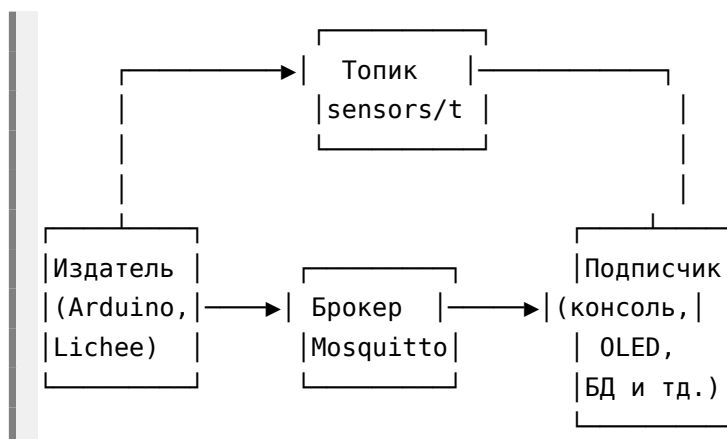
Перечислим основные особенности протокола:

- **Лёгкий протокол** — минимальный заголовок пакета (2 байта), подходит для устройств с ограниченной памятью и медленными каналами
- **Асинхронная модель** — издатель и подписчик не знают друг о друге, взаимодействуют только через брокер
- **Устойчивость к обрывам** — keep-alive, сохранённые сессии, сообщения о разрыве.

12.3.2 Архитектура «издатель-подписчик»

В отличие от клиент-серверной архитектуры, где отправитель напрямую обращается к получателю по адресу, MQTT использует модель

Pub/Sub:



Определение

Брокер (broker) — сервер, принимающий сообщения от издателей и доставляющий их подписчикам. В этой работе брокером служит Lichee RV Dock с Mosquitto.

Определение

Издатель (publisher) — клиент, отправляющий сообщения в топик брокера.

Определение

Подписчик (subscriber) — клиент, получающий сообщения из одного или нескольких топиков.

12.3.3 Топики

Определение

Топик (topic) — строковый адрес в иерархии MQTT, уровни которого разделены символом /.

Топик	Данные
home/livingroom/temperature	Температура в гостиной
home/kitchen/humidity	Влажность на кухне
lichee/stats/cpu	Загрузка ЦП одноплатника
sensors/bme280	Все данные с датчика BME280

Шаблонные подписки позволяют подписываться на группы топиков:

- **+** — заменяет ровно один уровень иерархии. Подписка home/+/temperature получит данные о температуре из всех комнат
- **#** — заменяет любое количество уровней (только в конце). Подписка home/# получит все сообщения из всех комнат и подтопиков дома

12.3.4 Качество обслуживания (QoS)

MQTT предоставляет три уровня гарантии доставки сообщений:

Уровень	Название	Описание	Пример использования
QoS 0	At most once	Сообщение отправляется один раз без подтверждения. Возможна потеря.	Телеметрия: температура раз в минуту
QoS 1	At least once	Брокер подтверждает получение (PUBACK). Возможны дубликаты.	Команды управления освещением
QoS 2	Exactly once	Четырёхэтапное рукопожатие. Гарантирует ровно одну доставку.	Транзакции, списание средств

В рамках данной лабораторной работы используется QoS 0 — самый простой и быстрый режим.

12.3.5 Retained-сообщения

Определение

Retained-сообщение — последнее сохранённое брокером сообщение топика, которое автоматически получают новые подписчики.

При публикации с флагом `retain = true` брокер сохраняет последнее сообщение в топике и автоматически отправляет его каждому новому подписчику. Это удобно для статусов, которые меняются редко: новый клиент немедленно получает актуальное значение, не дожидаясь следующей публикации.

12.3.6 Will-сообщения

Определение

Will-сообщение — сообщение, которое брокер публикует при нештатном отключении клиента.

Will-сообщение задаётся при подключении клиента к брокеру. Если клиент отключается нештатно из-за обрыва связи или тайм-аута keep-alive, брокер автоматически публикует это сообщение. Так система мониторинга узнаёт об аварийном отключении устройства.

12.3.7 MQTT-брокер Mosquitto

Определение

Mosquitto — открытая реализация MQTT-брокера, доступная в репозиториях ALT Linux.

Установка и запуск:

```
apt-get install mosquitto
systemctl start mosquitto
systemctl enable mosquitto
```

Проверка работоспособности командной строкой:

```
# В первом терминале запустите подписчика:
mosquitto_sub -h localhost -t "test/topic"

# Во втором терминале запустите издателя:
mosquitto_pub -h localhost -t "test/topic" -m "Hello MQTT!"
```

Проверка

Ожидаемый результат. В терминале подписчика появляется Hello MQTT!.

12.3.8 Python-клиент paho-mqtt

Определение

Eclipse Paho — набор клиентских библиотек MQTT для разных языков программирования.

Установите библиотеку для Python:

```
apt-get install python3-module-paho
```

Основные функции обратного вызова:

```
import paho.mqtt.client as mqtt

def on_connect(client, userdata, flags, rc):
    """Вызывается при подключении к брокеру. rc == 0 – успех."""
    print(f"Connected: {rc}")

def on_message(client, userdata, msg):
    """Вызывается при получении сообщения из подписанного топика."""
    print(f"{msg.topic}: {msg.payload.decode()}")

client = mqtt.Client()
client.on_connect = on_connect
client.on_message = on_message

client.connect("localhost", 1883)    # адрес брокера, порт
client.subscribe("topic/name")      # подписка на топик
client.loop_forever()               # бесконечный цикл обработки
```

Для публикации используется метод `publish()`:

```
client.publish("topic/name", "data_string")
```

12.3.9 С-клиент `libmosquitto`

Определение

libmosquitto — клиентская библиотека MQTT для языка C.

```
apt-get install libmosquitto libmosquitto-devel
```

12.4 Порядок выполнения

12.4.1 Установка и настройка брокера Mosquitto

Брокер будет запущен на Lichee RV Dock. На ПК установите только клиентские утилиты для тестирования.

1. **На Lichee:** установите Mosquitto и клиентские утилиты:

```
apt-get install mosquitto
```

2. **На ПК (ALT Linux):** установите Mosquitto — пакет включает клиентские утилиты `mosquitto_pub/mosquitto_sub` для тестирования:

```
apt-get install mosquitto
```

3. **На Lichee:** запустите брокер и добавьте в автозагрузку:

```
systemctl start mosquitto
systemctl enable mosquitto
```

4. **На Lichee:** настройте брокер для приёма внешних подключений. По умолчанию Mosquitto слушает только `localhost`. Добавьте в файл `/etc/mosquitto/mosquitto.conf` строки:

Внимание

Настройки `listener 1883 0.0.0.0` и `allow_anonymous true` открывают брокер на всех сетевых интерфейсах без аутентификации. Используйте их только в изолированной учебной сети; не подключайте такой стенд к публичной или недоверенной сети.

```
listener 1883 0.0.0.0
allow_anonymous true
```

В реальной сети необходимо включить аутентификацию, ограничить доступ firewall-правилами и по возможности использовать TLS.

Перезапустите брокер:

```
systemctl restart mosquitto
```

Проверьте, что порт 1883 слушается на всех интерфейсах:

```
ss -tlnp | grep 1883
```

Проверка

Ожидаемый результат. Вывод содержит `0.0.0.0:1883` или `*:1883`.

5. **Узнайте IP-адрес Lichee** (понадобится для подключения подписчика с ПК):

```
ip address | grep "inet " | grep -v 127.0.0.1
```

или:

```
hostname -i
```

Запишите этот IP-адрес — далее будем обозначать его `<IP_LICHEE>`.

Проверка

Проверка адреса. Команда `hostname -i` может вывести несколько адресов. Выберите адрес интерфейса в одной учебной сети с ПК и подтвердите его командами `ip address` и `ping <IP_LICHEE>`.

6. **Проверка связности с ПК:**

```
# С ПК проверьте доступность Lichee:
ping <IP_LICHEE>
```

```
# Проверьте, что порт 1883 на Lichee открыт:
telnet <IP_LICHEE> 1883
```

Если соединение устанавливается (в telnet появится Connected или escape-символ) — брокер доступен.

7. Протестируйте Pub/Sub между Lichee и ПК:

Терминал 1 — на ПК, подписчик:

```
mosquitto_sub -h <IP_LICHEE> -t "test/hello"
```

Терминал 2 — на Lichee (по SSH), издатель:

```
mosquitto_pub -h localhost -t "test/hello" -m "Hello from Lichee!"
```

Проверка

Ожидаемый результат. В терминале на ПК появляется Hello from Lichee!: брокер принимает внешнее подключение и доставляет сообщение подписчику.

12.4.2 Python-издатель системных метрик

Создайте скрипт `mqtt_publisher.py`, который читает загрузку CPU и использование RAM и публикует их в топик `lichee/stats`:

```
#!/usr/bin/env python3

import paho.mqtt.client as mqtt
import json
import time

TOPIC = "lichee/stats"
INTERVAL = 2.0

def get_cpu_usage():
    """Возвращает загрузку CPU в процентах (от 0 до 100)."""
    with open("/proc/stat") as f:
        fields = f.readline().split()
        user, nice, system, idle = map(int, fields[1:5])
        total = user + nice + system + idle

    # Расчёт по дельте между вызовами
    if not hasattr(get_cpu_usage, "prev"):
        get_cpu_usage.prev = (total, idle)
        return 0.0

    prev_total, prev_idle = get_cpu_usage.prev
    get_cpu_usage.prev = (total, idle)

    diff_total = total - prev_total
    diff_idle = idle - prev_idle
```

```

        return 100.0 * (1.0 - diff_idle / diff_total) if diff_total > 0 else 0.0

def get_ram_usage():
    """Возвращает использование RAM в процентах (от 0 до 100)."""
    total = free = 0
    with open("/proc/meminfo") as f:
        for line in f:
            if line.startswith("MemTotal:"):
                total = int(line.split()[1])
            elif line.startswith("MemAvailable:"):
                free = int(line.split()[1])
    return 100.0 * (total - free) / total if total > 0 else 0.0

def main():
    client = mqtt.Client()
    client.connect("localhost", 1883)
    client.loop_start()

    print(f"Publishing to '{TOPIC}' every {INTERVAL}s. Ctrl+C to stop.")
    try:
        while True:
            cpu = get_cpu_usage()
            ram = get_ram_usage()
            payload = json.dumps({"cpu": round(cpu, 1), "ram": round(ram, 1)})
            client.publish(TOPIC, payload)
            print(f"Published: {payload}")
            time.sleep(INTERVAL)
    except KeyboardInterrupt:
        print("Stopped.")
    finally:
        client.loop_stop()
        client.disconnect()

if __name__ == "__main__":
    main()

```

Объяснение: - `get_cpu_usage()` читает `/proc/stat`, вычисляя загрузку как `100% - доля_простоя между двумя вызовами`. Первый вызов возвращает 0 (базовый замер), последующие — реальное значение - `get_ram_usage()` читает `/proc/meminfo`, находит поля `MemTotal` и `MemAvailable`, вычисляет процент занятой памяти - Данные упаковываются в JSON и публикуются каждые 2 секунды - `loop_start()` запускает сетевой цикл `raho` в фоновом потоке, не блокируя основной

Запустите скрипт:

```
python3 mqtt_publisher.py
```

Оставьте работать — на следующем шаге мы подпишемся на этот топик.

12.4.3 Python-подписчик на ПК

Скрипт подписчика будет запущен на вашем ПК. Он подключается к брокеру на Lichee по IP-адресу и получает данные из топика `lichee/stats`.

Внимание

Перед запуском подписчика замените `<IP_LICHEE>` фактическим адресом Lichee и проверьте с ПК доступность порта 1883, например командой `telnet <IP_LICHEE> 1883`. Не запускайте скрипт с шаблонным адресом.

Создайте на ПК скрипт `mqtt_subscriber.py`:

```
#!/usr/bin/env python3

import paho.mqtt.client as mqtt
import json

# IP-адрес Lichee RV Dock из шага 1
BROKER = "<IP_LICHEE>"
TOPIC = "lichee/stats"

def on_connect(client, userdata, flags, rc):
    print(f"Connected to broker (rc={rc})")
    client.subscribe(TOPIC)
    print(f"Subscribed to '{TOPIC}'")

def on_message(client, userdata, msg):
    try:
        data = json.loads(msg.payload.decode())
        cpu = data.get("cpu", "?")
        ram = data.get("ram", "?")
        print(f"CPU: {cpu:5.1f}%   RAM: {ram:5.1f}%")
    except json.JSONDecodeError:
        print(f"Raw: {msg.payload.decode()}")

def main():
    client = mqtt.Client()
    client.on_connect = on_connect
    client.on_message = on_message
    client.connect(BROKER, 1883)
    print(f"Connected to MQTT broker at {BROKER}")
    print("Listening for messages...")
    client.loop_forever()

if __name__ == "__main__":
    main()
```

Объяснение: - BROKER — IP-адрес Lichee RV Dock в вашей локальной сети. Замените `<IP_LICHEE>` на реальный адрес, определённый на шаге

1 - `on_connect` вызывается при успешном подключении — здесь выполняется подписка на топик - `on_message` вызывается при получении сообщения: парсинг JSON, извлечение `cpu` и `ram`, форматированный вывод - `loop_forever()` блокирует поток и обрабатывает входящие MQTT-пакеты бесконечно

Установка `paho-mqtt` на ПК (если не установлена):

```
apt-get install python3-module-paho
```

Запуск:

Проверка

Проверка запуска. Сначала запустите брокер Mosquitto, затем издатель на Lichee и после него подписчик на ПК. Перед следующим этапом убедитесь, что предыдущий работает без ошибок.

1. Убедитесь, что издатель `mqtt_publisher.py` запущен на Lichee (шаг 2)
2. Запустите подписчик на ПК:

```
python3 mqtt_subscriber.py
```

В консоли должно появиться:

```
Connected to broker (rc=0)
Subscribed to 'lichee/stats'
CPU:  12.3%  RAM:  45.7%
CPU:  15.1%  RAM:  45.8%
...
```

12.4.4 С-клиент-издатель

MQTT-клиент можно написать не только на Python, но и на C с использованием библиотеки `libmosquitto`. Это даёт меньший размер бинарного файла и лучшую производительность.

Проверка

Дополнительная проверка. Сначала проверьте цепочку с Python-издателем, затем замените только издатель C-клиентом и сравните формат сообщений и устойчивость подключения.

12.4.4.1 Установка библиотек на одноплатнике

```
apt-get install libmosquitto libmosquitto-devel
```

12.4.4.2 Исходный код

Создайте файл `mqtt_c_publisher.c`:

```
#include <stdio.h>
#include <stdlib.h>
#include <string.h>
#include <unistd.h>
#include <mosquitto.h>

#define MQTT_HOST    "localhost"
#define MQTT_PORT    1883
#define MQTT_TOPIC    "lichee/stats"
#define KEEPALIVE    60

static float get_cpu_usage(void) {
    FILE *fp = fopen("/proc/stat", "r");
    if (!fp) return -1;

    unsigned long user, nice, system, idle;
    fscanf(fp, "cpu %lu %lu %lu %lu", &user, &nice, &system, &idle);
    fclose(fp);

    unsigned long total = user + nice + system + idle;
    static unsigned long prev_total = 0, prev_idle = 0;
    float usage = 0.0;

    if (prev_total > 0 && total > prev_total) {
        float diff_idle = idle - prev_idle;
        float diff_total = total - prev_total;
        usage = 100.0 * (1.0 - diff_idle / diff_total);
    }
    prev_total = total;
    prev_idle = idle;
    return usage;
}

static float get_ram_usage(void) {
    FILE *fp = fopen("/proc/meminfo", "r");
    if (!fp) return -1;

    unsigned long total = 0, available = 0;
    char line[128];
    while (fgets(line, sizeof(line), fp)) {
        if (strstr(line, "MemTotal:"))
            sscanf(line, "MemTotal: %lu kB", &total);
        if (strstr(line, "MemAvailable:"))
            sscanf(line, "MemAvailable: %lu kB", &available);
    }
    fclose(fp);
}
```



```

        return (total > 0) ? 100.0 * (total - available) / total : -1;
    }

int main(void) {
    mosquitto_lib_init();

    struct mosquitto *mosq = mosquitto_new(NULL, true, NULL);
    if (!mosq) {
        fprintf(stderr, "Error: mosquitto_new failed\n");
        return 1;
    }

    if (mosquitto_connect(mosq, MQTT_HOST, MQTT_PORT, KEEPALIVE)) {
        fprintf(stderr, "Error: cannot connect to broker\n");
        mosquitto_destroy(mosq);
        mosquitto_lib_cleanup();
        return 1;
    }

    printf("C MQTT publisher started.\n");
    printf("Publishing to '%s'. Press Ctrl+C to stop.\n", MQTT_TOPIC);

    char payload[128];
    while (1) {
        float cpu = get_cpu_usage();
        float ram = get_ram_usage();

        if (cpu < 0 || ram < 0) {
            fprintf(stderr, "Error reading system stats\n");
            sleep(1);
            continue;
        }

        snprintf(payload, sizeof(payload),
                 "{\"cpu\":%.1f,\"ram\":%.1f}", cpu, ram);

        int ret = mosquitto_publish(mosq, NULL, MQTT_TOPIC,
                                   strlen(payload), payload, 0, false);
        if (ret != MOSQ_ERR_SUCCESS)
            fprintf(stderr, "Publish error: %s\n", mosquitto_strerror(ret));
        else {
            printf("Sent: %s\n", payload);
            ret = mosquitto_loop(mosq, 1000, 1);
            if (ret != MOSQ_ERR_SUCCESS) {
                fprintf(stderr, "Network error: %s\n", mosquitto_strerror(ret));
                break;
            }
        }
    }
}

```

```

        sleep(2);
    }

    mosquitto_destroy(mosq);
    mosquitto_lib_cleanup();
    return 0;
}

```

12.4.4.3 Нативная компиляция на Lichee

```

gcc -o mqtt_c_publisher mqtt_c_publisher.c -lmosquitto
./mqtt_c_publisher

```

Исходный файл `mqtt_c_publisher.c` копируется на Lichee через `scp`, компилируется нативно командой выше и запускается прямо на одноплатнике.

12.5 Задание

Ознакомьтесь с ключевыми понятиями и выполните следующие подзадачи.

1. **Базовая задача.** Установите и запустите MQTT-брокер Mosquitto на Lichee RV Dock. Настройте брокер на приём подключений со всех сетевых интерфейсов (`listener 1883 0.0.0.0`). Протестируйте публикацию и подписку между ПК и Lichee через утилиты `mosquitto_pub` и `mosquitto_sub`.
2. **Основная задача.** Реализуйте распределённую систему мониторинга загрузки одноплатника:
 - На Lichee: напишите Python-скрипт-издатель, публикующий загрузку CPU и использование RAM (в процентах) в топик `lichee/stats` раз в 2 секунды. Данные передавайте в формате JSON: `{"cpu": 12.3, "ram": 45.7}`.
 - На ПК: напишите Python-скрипт-подписчик, подключающийся к брокеру на Lichee (по IP-адресу), принимающий данные из топика `lichee/stats` и выводящий их в консоль в читаемом виде: CPU: 12.3% RAM: 45.7%.
 - Продемонстрируйте одновременную работу издателя (на Lichee) и подписчика (на ПК).
3. **Дополнительная часть (С-клиент).** Напишите аналог MQTT-издателя на языке C, публикующий те же системные метрики. Выполните нативную компиляцию программы и проверьте её работу на Lichee.

4. Ответить на контрольные вопросы (письменно, в отчёте).
5. Продемонстрировать работу преподавателю.

12.6 Контрольные вопросы

1. В чём преимущества архитектуры «издатель-подписчик» по сравнению с клиент-серверной для IoT-систем?
2. Какие уровни QoS существуют в MQTT и в каких сценариях каждый из них целесообразно использовать?
3. Как обеспечивается безопасность в MQTT? Какие механизмы аутентификации и шифрования поддерживаются?
4. Чем отличается MQTT от других протоколов IoT, таких как CoAP или HTTP/2?
5. Как организовать иерархию топиков для системы умного дома?
6. Какие проблемы могут возникнуть при использовании MQTT в сетях с нестабильным соединением и как их решить?
7. Как масштабировать MQTT-инфраструктуру при увеличении количества устройств?
8. В чём преимущества использования MQTT поверх WebSocket для веб-приложений?
9. Как реализовать retained- и will-сообщения в MQTT и для чего они используются?
10. Какие библиотеки для работы с MQTT существуют для различных платформ?

12.7 Требования к отчёту

Отчёт должен содержать:

- цель работы;
- схему взаимодействия издателя, брокера и подписчика;
- команды настройки и проверки Mosquitto;
- исходные тексты клиентов;
- примеры опубликованных и полученных сообщений;
- ответы на контрольные вопросы;
- краткий вывод.

Отправьте отчёт в согласованном формате до установленного срока.

12.8 Полезные ссылки

- Официальный сайт MQTT
- Документация Mosquitto

- Eclipse Paho Python
- Документация libmosquitto

13 Лабораторная №13. Итоговое проектное задание по вариантам

13.1 Цель работы

Самостоятельно спроектировать и реализовать распределённую систему, объединяющую одноплатный компьютер Lichee RV Dock, микроконтроллер Arduino Uno и графическое приложение на ПК. Транспорт между компонентами — протокол MQTT (брокер Mosquitto на Lichee).

13.2 Оборудование и исходные данные

- оборудование, указанное в полученном варианте;
- Lichee RV Dock и Arduino Uno для вариантов с аппаратной связкой;
- компьютер с необходимыми средствами разработки;
- изолированная учебная сеть с MQTT-брокером Mosquitto на Lichee;
- результаты и примеры лабораторных №7–12.

13.3 Порядок выполнения

1. Каждый студент получает **один из десяти вариантов** задания
2. Разрабатывает и отлаживает все компоненты на оборудовании стенда
3. Демонстрирует преподавателю полностью работающую систему
4. Предоставляет исходный код всех компонентов (Arduino .ino, Python-скрипты Lichee, GUI-приложение ПК, Makefile/инструкцию по сборке для C-частей)
5. Проходит устную защиту: объяснение архитектуры, протоколов обмена, выбранных решений

Внимание

Полученный вариант выполняется полностью: обязательны все перечисленные в нём компоненты, функции и ключевые проверки. Пункты, явно помеченные как дополнительные или опциональные, не заменяют основную часть варианта.

Опасность повреждения

Во всех вариантах с физическим соединением Arduino Uno и Lichee RV Dock линии SPI должны проходить через преобразователь логических уровней 5↔3,3 В: сторона HV питается от Arduino 5 В, сторона LV — от Lichee 3,3 В. Arduino, Lichee, преобразователь и периферия должны иметь общий GND; прямое попадание сигнала 5 В на GPIO Lichee недопустимо.

Охват лабораторных: №7 (SPI), №8 (I2C OLED), №10 (Arduino), №11 (Arduino + BME280 + SPI), №12 (MQTT).

13.4 Вариант 1. Метеостанция — сбор и визуализация данных с датчика BME280

13.4.1 Описание

Собрать распределённую систему, в которой данные с датчика BME280 (температура, влажность, давление) передаются от Arduino к одноплатнику Lichee, далее через MQTT на ПК и отображаются в реальном времени в графическом интерфейсе.

13.4.2 Цепочка

BME280 — I2C → Arduino Uno — SPI(slave) → Lichee RV Dock — MQTT → ПК (GUI)

13.4.3 Требования

1. **Arduino + датчик (лаб. №11).** Прошивка Arduino читает BME280 по I2C (библиотека `GyverBME280`) и передаёт три значения `float` (температура, °C; влажность, %; давление, гПа) по SPI в режиме ведомого устройства на Lichee. Протокол — серия однобайтовых транзакций.
2. **Lichee — SPI-приём + MQTT-публикация (лаб. №7, №12).** Python-скрипт на Lichee принимает данные от Arduino через `/dev/spidev1.0`, распаковывает `struct.unpack("<fff", ...)` и публикует их в MQTT-топик `sensors/bme280` в формате JSON:

```
{"temperature": 25.1, "humidity": 48.3, "pressure": 1004.5, "timestamp":  
  ↪ "2026-05-04T12:34:56"}
```
3. **I2C OLED (лаб. №8).** Дополнительно — дублировать данные на OLED-экран SSD-1306 через I2C на Lichee (три строки: температура, влажность, давление).

Проверка

Дополнительная демонстрация. OLED в варианте 1 служит дополнительным каналом локальной индикации. Подключайте его к I2C Lichee с питанием 3,3 В; отсутствие OLED не должно нарушать SPI-приём и MQTT-публикацию.

4. ПК — GUI-приложение. Python-приложение, подключающееся к MQTT-брокеру на Lichee и отображающее:

- Текущие значения трёх параметров — крупным шрифтом в отдельных блоках
- Три графика (matplotlib, встроенный в GUI) с историей за последние 60 секунд (ось X — время, ось Y — значение)
- Статус подключения: индикатор (зелёный/красный кружок) — получает ли система данные. Статус менять по тайм-ауту: если данные не поступали более 5 секунд — «нет данных»

13.4.4 Ключевые точки проверки

- Физически подключённый BME280 выдаёт реалистичные значения
- При остановке MQTT-публикации на Lichee GUI не позднее чем через 5 секунд показывает «нет данных» (красный индикатор)
- При отключении Arduino от Lichee — аналогично
- Графики обновляются плавно, история не теряется при свёртывании окна

13.5 Вариант 2. Пульт управления светодиодами через MQTT и SPI

13.5.1 Описание

Через графический интерфейс на ПК управлять светодиодами на мезонинной плате Arduino, передавая команды по цепочке MQTT → Lichee → SPI → Arduino.

13.5.2 Цепочка

ПК (GUI) —MQTT→ Lichee RV Dock —SPI(master)→ Arduino Uno —GPIO→
↔ LED-мезонин

13.5.3 Требования

1. **Arduino — приём команд по SPI (лаб. №10, №11).** Прошивка работает в режиме SPI-slave и управляет четырьмя светодиодами на

пинах 3, 5, 6 и 9 мезонинной платы. Пин 10 зарезервирован аппаратным SPI как вход SS и для управления светодиодом не используется. Команды приходят от Lichee-мастера. Протокол команд студент проектирует самостоятельно, например байт команды и байт данных или четыре бита для четырёх светодиодов.

Поддерживаемые операции для каждого светодиода:

- включить / выключить (индивидуально)
- «бегущий огонёк» вперёд (пины 3→5→6→9→3)
- «бегущий огонёк» назад (пины 9→6→5→3→9)
- остановить анимацию

Скорость бегущего огонька задаётся параметром (задержка в миллисекундах).

2. **Lichee — MQTT-подписчик + SPI-мастер (лаб. №12).** Python-скрипт подписывается на MQTT-топик `led/command`, получает JSON-команды и транслирует их в SPI-транзакции к Arduino.

Формат JSON-команды (примеры):

```
{"pin": 3, "action": "on"}
{"pin": 5, "action": "off"}
{"action": "animation", "direction": "forward", "delay_ms": 150}
{"action": "stop"}
```

3. **ПК — GUI-приложение.** Окно с элементами управления:

- четыре тумблера для индивидуальных светодиодов с подписями «LED 3», «LED 5», «LED 6», «LED 9»;
- Кнопка «Бегущий вперёд» и кнопка «Бегущий назад»
- Слайдер задержки анимации (50–500 мс)
- Кнопка «Стоп»
- При изменении любого элемента GUI публикуется соответствующая MQTT-команда

13.5.4 Ключевые точки проверки

- Каждый тумблер индивидуально включает/выключает свой светодиод
- Бегущий огонёк работает в обоих направлениях
- Смена задержки на слайдере меняет скорость анимации
- «Стоп» останавливает анимацию и оставляет текущее состояние светодиодов

13.6 Вариант 3. Панель системного мониторинга одноплатника

13.6.1 Описание

Создать дашборд (панель мониторинга), отображающий в реальном времени системные метрики одноплатника Lichee: загрузку CPU, использование RAM, занятость корневого раздела, сетевой трафик. Метрики собираются на Lichee и публикуются по MQTT. GUI на ПК отображает их в виде приборных панелей и графиков.

13.6.2 Цепочка

Lichee RV Dock (/proc, /sys) —MQTT→ ПК (GUI)

13.6.3 Требования

1. **Lichee — сборщик метрик (лаб. №12).** Python-скрипт-издатель, публикующий в MQTT-топик `lichee/stats` JSON-пакет раз в 2 секунды:

```
{
  "cpu_percent": 12.3,
  "ram_percent": 45.7,
  "disk_percent": 32.1,
  "rx_bytes_per_sec": 1024,
  "tx_bytes_per_sec": 512,
  "uptime_seconds": 12345,
  "hostname": "lichee-rv",
  "kernel": "6.16.0"
}
```

Источники данных:

- CPU: `/proc/stat`
 - RAM: `/proc/meminfo`
 - Диск: `os.statvfs('/')`
 - Сеть: `/sys/class/net/<интерфейс>/statistics/rx_bytes, tx_bytes` (дельта между опросами)
 - Uptime: `/proc/uptime`
 - Hostname: `socket.gethostname()`
 - Версия ядра: `os.uname().release`
2. **Дополнительно.** Написать сборщик метрик на языке C с использованием только системных вызовов и файлов `/proc`, `/sys`. Скомпилировать под RISC-V (кросс-компиляция с x86_64). По быстродействию сравнить с Python-версией.

Проверка

Дополнительное сравнение. С-сборщик в варианте 3 выполняется после готовой Python-версии. Используйте одинаковый набор метрик и интервал публикации, отдельно зафиксируйте размер процесса, загрузку CPU и размер исполняемого файла.

3. **ПК — GUI-приложение.** Интерфейс дашборда содержит:

- 4 «приборных панели» (gauge / стрелочный индикатор) для CPU, RAM, диска, сети. Каждая панель показывает текущее значение в процентах и имеет цветовое кодирование (зелёный < 50%, жёлтый 50–80%, красный > 80%)
- Строка статуса в нижней части окна: uptime, hostname, версия ядра
- График истории (опционально): загрузка CPU и RAM за последние 60 секунд
- Кнопка «Обновить сейчас» — публикует команду в lichee/cmd (`{"action": "refresh"}`)

13.6.4 Ключевые точки проверки

- Все четыре панели обновляются и показывают адекватные значения
- При нагрузке на Lichee (запуск stress или dd) значения меняются
- Кнопка «Обновить сейчас» вызывает немедленную публикацию
- Строка статуса показывает реальные uptime и hostname

13.7 Вариант 4. Генератор сигналов с управлением по MQTT — практическая верификация анализатором

13.7.1 Описание

С графического интерфейса на ПК задать параметры цифрового сигнала (меандра). Lichee генерирует сигнал на GPIO-пине. Студент подключает логический анализатор к этому пину, захватывает сигнал в PulseView и подтверждает соответствие заданным параметрам.

13.7.2 Цепочка

ПК (GUI) —MQTT→ Lichee (libgpiod2) —GPIO→ Логический анализатор (fx2lafw)
↔ —USB→ ПК (PulseView)

13.7.3 Требования

1. **Lichee — генератор сигнала (лаб. №9).** Python-скрипт подписывается на MQTT-топик `gpio/control`, принимает JSON-команды и управляет GPIO-пином (PE16, линия 144) через `libgpiod2`.

Формат JSON-команды:

```
{ "action": "start", "gpio_line": 144, "freq_hz": 1000, "duty_percent": 50 }
{ "action": "stop" }
```

После получения "start" скрипт в отдельном потоке генерирует меандр. При "stop" поток останавливается, пин переводится в низкий уровень. При повторном "start" без "stop" параметры обновляются без разрыва сигнала.

2. **ПК — GUI-приложение.** Окно с элементами управления:

- Выпадающий список с доступными GPIO-линиями Lichee (линии 144, 145, 146 — PE16, PE17, PE18)
- Поле ввода частоты (Гц) — от 1 до 1000 Гц, с проверкой корректности ввода. Значение по умолчанию: 100 Гц
- Слайдер или поле ввода скважности (%) — от 1 до 99%. По умолчанию: 50%
- Кнопка «Старт» и кнопка «Стоп»
- Предпросмотр сигнала — на Canvas отрисовывается визуализация меандра (два периода) с подписями периода и длительности уровней

3. **Верификация (лаб. №9).** Подключить логический анализатор (fx2lafw) к заданному GPIO-пину Lichee. Запустить захват в PulseView (частота дискретизации $\geq 4\times$ от частоты сигнала, но не менее 12 МГц). Измерить период и скважность сигнала и сравнить с заданными в GUI значениями.

13.7.4 Ключевые точки проверки

- Сигнал на GPIO появляется при нажатии «Старт» и пропадает при «Стоп»
- Частота и скважность, измеренные в PulseView, соответствуют заданным (погрешность в пределах 5% во всем заданном диапазоне)
- Изменение параметров через GUI во время работы генератора обновляет сигнал без остановки
- Предпросмотр на Canvas корректно отображает два периода меандра с верными метками времени

13.8 Вариант 5. Светодиодный проигрыватель мелодий

13.8.1 Описание

Реализовать визуальный проигрыватель мелодий, в котором на ПК в графическом редакторе задаётся «партитура» — последовательность включений светодиодов во времени. Мелодия передаётся по цепочке MQTT → Lichee → SPI → Arduino и проигрывается на четырёх светодиодах мезонинной платы, подключённых к пинам 3, 5, 6 и 9. Пин 10 зарезервирован как вход SS аппаратного SPI.

13.8.2 Цепочка

ПК (GUI-редактор) —MQTT→ Lichee RV Dock —SPI(master)→ Arduino Uno
↔ —GPIO→ 4 LED

13.8.3 Ключевое проектное решение

Arduino получает только один байт — битовую маску четырёх светодиодов (биты 0–3). Вся логика темпа (BPM) и таймингов размещается на Lichee, что минимизирует Arduino-код и упрощает SPI-протокол до однобайтовых транзакций.

13.8.4 Требования

1. **Arduino — прошивка SPI-slave (лаб. №10, №11).** Минимальный скетч в режиме SPI-slave. Принимает один байт от Lichee-мастера и устанавливает пины в соответствии с битами принятого байта. SPI: режим slave, пины MISO(12)/MOSI(11)/SCK(13)/SS(10). При получении 0x00 — гасит всё.
2. **Lichee — MQTT-подписчик + SPI-транслятор (лаб. №12).** Python-скрипт, подписывающийся на MQTT-топик melody/command. При "play" — сохраняет массив шагов и BPM, вычисляет длительность одного шага и в отдельном потоке отправляет битовые маски по SPI с нужной задержкой. После последнего шага отправляет 0x00 и публикует статус "stopped". SPI: /dev/spidev1.0, mode 0, скорость 100 кГц.
3. **ПК — GUI-редактор мелодии.** Окно приложения содержит:
 - редактор на Canvas с сеткой: четыре строки для LED 3, 5, 6 и 9, столбцы — шаги, не более 64; щелчок по ячейке переключает состояние;
 - Слайдер темпа (BPM): 60–240, по умолчанию 120
 - Кнопки Play, Stop

- Выпадающий список «Загрузить пример»: 3 встроенных мелодии
- Кнопка «Сохранить» / «Загрузить» — JSON-файл
- Индикация проигрывания: вертикальная красная линия движется по текущему шагу

MQTT-команды в топик `melody/command`:

```
{ "action": "play", "bpm": 120, "melody": [[1,0,0,1], [0,0,1,0], ...] }
{ "action": "stop" }
```

13.8.5 Ключевые точки проверки

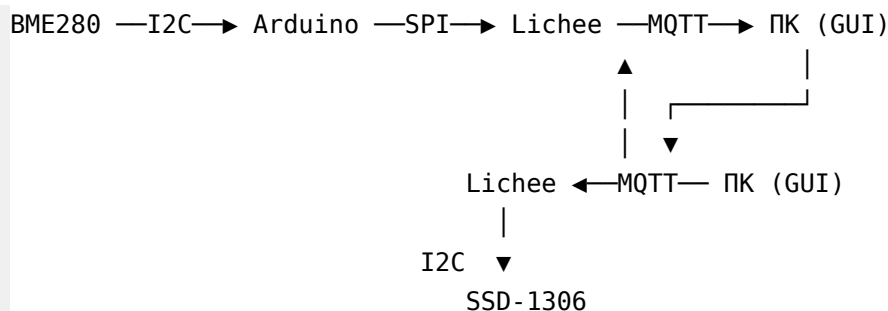
- При Play светодиоды загораются в точном соответствии с рисунком в редакторе
- Красная линия в GUI движется синхронно с физическими светодиодами
- При Stop всё гаснет
- Изменение BPM на слайдере ускоряет/замедляет проигрывание
- Сохранение и загрузка JSON-файла работают корректно

13.9 Вариант 6. Система мониторинга с оповещением на I2C OLED

13.9.1 Описание

Расширение варианта №1 (Метеостанция). Данные с датчика BME280 идут по цепочке Arduino → SPI → Lichee → MQTT → GUI на ПК. Дополнительно оператор задаёт в GUI пороговые значения для каждого параметра. При превышении порога GUI сигнализирует тревогу. Оператор жмёт кнопку «Оповестить на OLED» — по обратному каналу MQTT → Lichee на I2C-экран SSD-1306 выводится тревожное сообщение с текущим значением.

13.9.2 Цепочка



13.9.3 Требования

1. **Arduino + датчик (лаб. №11).** Прошивка идентична варианту №1: читает BME280 по I2C (GyverBME280), передаёт три float по SPI-slave на Lichee.
2. **Lichee — SPI-приём + MQTT-публикация (лаб. №7, №12).** Принимает данные по SPI, публикует JSON в топик `sensors/bme280`.
3. **Lichee — подписчик на тревоги (лаб. №8, №12).** Отдельный Python-скрипт, подписывающийся на топик `alert/oled`. При получении сообщения выводит текст на SSD-1306 через I2C (luma-oled). Через 5 секунд автоматически очищает экран.
4. **ПК — GUI-приложение.** Все элементы варианта №1 плюс:
 - Поля ввода порогов: `T_max`, `H_max`, `P_min`, `P_max` со значениями по умолчанию
 - Индикация тревоги: при превышении порога строка параметра подсвечивается красным и мигает
 - Кнопка «Оповестить на OLED»: активна только когда есть активная тревога
 - Сброс тревоги: когда значение возвращается в пределы порога + гистерезис

13.9.4 Ключевые точки проверки

- При превышении порога GUI подсвечивает соответствующую строку красным
- Кнопка «Оповестить на OLED» активна только при тревоге; при нажатии сообщение появляется на физическом экране SSD-1306
- Через 5 секунд тревожное сообщение удаляется и экран очищается
- При возврате значения в норму тревога сбрасывается с гистерезисом

13.10 Вариант 7. Генератор загрузочных образов — консольный сборщик SD-карты

13.10.1 Описание

Создать консольное приложение на Python, которое из готовых компонентов автоматически собирает загрузочную SD-карту для Lichee RV Dock с заданной конфигурацией. Готовые ядра, Device Tree Blob, rootfs и загрузчик предоставляются преподавателем.

13.10.2 Цепочка

ПК (Python-скрипт) —fdisk/dd/mkfs/cp→ SD-карта

13.10.3 Требования

Скрипт `build_image.py` с интерфейсом командной строки:

Опасность повреждения

Команды `fdisk`, `dd` и `mkfs` необратимо уничтожают данные на выбранном устройстве. Сначала выполните `--dry-run`, который обязан только вывести план без запуска команд записи; затем повторно проверьте имя блочного устройства, отсутствие смонтированных разделов и запросите явное подтверждение перед реальной записью.

```
python3 build_image.py --device /dev/sdX --config spi [--dry-run]
```

Конфигурация	Ядро	Device Tree	Поддерживаемые интерфейсы
basic	Image	sun20i-d1-lichee-rv-dock.dtb	WiFi
spi	Image-spi	sun20i-d1-lichee-rv-dock-spi.dtb	WiFi + SPI
i2c	Image-i2c	sun20i-d1-lichee-rv-dock-i2c.dtb	WiFi + I2C
full	Image-spi	.dtb с SPI + I2C	WiFi + SPI + I2C

Действия программы (по порядку):

1. Проверка: `--device` существует, является блочным устройством и не примонтирован
2. Разбивка: `fdisk` создаёт таблицу разделов DOS (первый раздел 100 МБ FAT32, второй — остаток ext4)
3. Запись загрузчика: `dd if=u-boot-sunxi-with-spl.bin of=<device> bs=1k seek=8`
4. Форматирование: `mkfs.vfat -F 32 <device>1, mkfs.ext4 <device>2`
5. Копирование файлов на BOOT и распаковка rootfs
6. `sync` и уведомление о завершении

Вывод программы — человекочитаемый лог каждого шага:

```
[INFO] Checking device /dev/sdb ... OK (not mounted)
[INFO] Selected config: spi
[INFO] Step 1/6: Partitioning ...
[INFO] Step 2/6: Writing U-Boot (seek=8) ...
...
[INFO] Done. SD card is ready at /dev/sdb
```

13.10.4 Ключевые точки проверки

- Карта, собранная с конфигурацией `basic`, загружается, работает WiFi
- С конфигурацией `spi` — загружается, WiFi и SPI работают (проверить `ls /dev/spidev*`)
- С конфигурацией `i2c` — загружается, WiFi и I2C работают (проверить `ls /dev/i2c-*`)
- `--dry-run` выводит план, но не трогает SD-карту
- При попытке записать на примонтированное устройство — ошибка

13.11 Вариант 8. Система мониторинга со звуковой сигнализацией

13.11.1 Описание

Расширение варианта №1 (Метеостанция). При превышении порогового значения GUI автоматически воспроизводит звуковую сирену. Характер звука настраивается для каждого измеряемого параметра индивидуально. Оператор может квитировать тревогу кнопкой.

13.11.2 Цепочка

BME280 — I2C → Arduino — SPI → Lichee — MQTT → ПК (GUI + динамик)

13.11.3 Требования

1. **Arduino + датчик (лаб. №11).** Прошивка идентична варианту №1.
2. **Lichee — SPI-приём + MQTT-публикация (лаб. №7, №12).** Идентично варианту №1.
3. **ПК — GUI-приложение.** Все элементы варианта №1 плюс:
 - Поля ввода порогов (`T_max`, `H_max`, `P_min`, `P_max`)
 - Настройки сирены для каждого параметра: тип звука (прерывистый/быстрый/медленный/непрерывный/WAV-файл), частота тона (Гц)

- Автоматическое воспроизведение sireны при превышении порога
- Кнопка «Отключить sireну» (квитирование)
- Лог тревог (таблица: Время, Параметр, Значение, Порог) + экспорт в CSV

Генерация звука — через стандартную библиотеку Python wave (без дополнительных зависимостей), воспроизведение через aplay.

13.11.4 Ключевые точки проверки

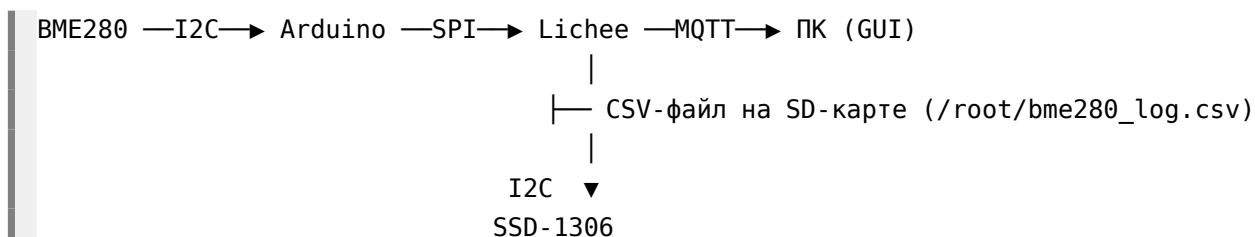
- При превышении порога GUI подсвечивает параметр красным и издаёт звуковую sireну
- Характер звука разный для разных параметров
- Кнопка «Отключить sireну» прекращает звук
- Лог тревог пополняется при каждом превышении; экспорт в CSV — корректный

13.12 Вариант 9. Самописец данных с хранением на SD-карте

13.12.1 Описание

Собрать систему, в которой данные с датчика BME280 накапливаются на SD-карте одноплатника и одновременно доступны для live-мониторинга через MQTT. GUI на ПК отображает текущие значения и позволяет загрузить историю измерений.

13.12.2 Цепочка



13.12.3 Требования

1. **Arduino + датчик (лаб. №11).** Прошивка идентична варианту №1: читает BME280 по I2C, передаёт три float по SPI-slave на Lichee.
2. **Lichee — SPI-приём + логгер + MQTT (лаб. №7, №8, №12).** Python-скрипт:
 - Принимает данные от Arduino через /dev/spidev1.0

- Дописывает строку в CSV-файл `/root/bme280_log.csv`:
 2026-06-24T12:34:56,25.1,48.3,1004.5
- Параллельно публикует текущие данные в MQTT-топик `sensors/bme280` (JSON)
- Дублирует текущие значения на OLED SSD-1306

3. ПК — GUI-приложение. Окно с элементами:

- Текущие значения (температура, влажность, давление)
- График реального времени (данные из MQTT)
- Кнопка «Загрузить историю» — по SCP или через HTTP качает CSV-файл с Lichee и отображает историю на отдельном графике
- Строка статуса: общее количество записей в логe, дата первой и последней записи

Внимание

Для загрузки истории выберите один способ и опишите его в отчёте: SCP с аутентификацией или HTTP с доступом только в учебной сети. Не встраивайте пароль в исходный код и не публикуйте каталог `/root` целиком через HTTP.

13.12.4 Ключевые точки проверки

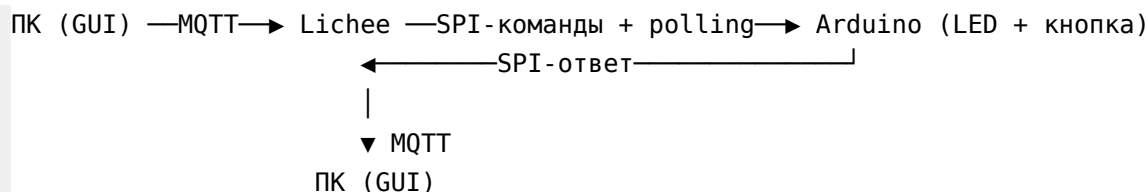
- Данные пишутся в CSV непрерывно, даже если GUI не запущен (проверить: запустить логгер, отключить ПК на минуту, снова подключиться — история сохранена)
- При перезагрузке Lichee файл лога сохраняется (SD-карта)
- График истории загружается корректно, временные метки соответствуют реальным
- Текущие значения на OLED обновляются синхронно с MQTT-потокom

13.13 Вариант 10. Игра на реакцию

13.13.1 Описание

Реализовать игру на измерение времени реакции человека. ПК отправляет команду начала раунда. Lichee ждёт случайную задержку, затем зажигает светодиод на Arduino и одновременно публикует метку времени в MQTT. Игрок должен как можно быстрее нажать кнопку на Arduino, после чего вычисляется время реакции и отображается на GUI ПК.

13.13.2 Цепочка



13.13.3 Требования

Опасность повреждения

Нажатие кнопки обнаруживается локальным прерыванием Arduino, но передаётся на Lichee только как байт ответа при периодическом опросе SPI мастером. Не соединяйте кнопку или выход Arduino отдельной линией с GPIO-прерывания Lichee: прямая линия с уровнем 5 В опасна для Lichee.

1. Arduino — SPI-slave + локальное прерывание (лаб. №10, №11).

Прошивка:

- Режим SPI-slave: принимает однобайтовую команду от Lichee
- Команда 0x01 — зажечь LED на пине 5
- Команда 0x00 — погасить LED
- Команда 0x02 — опросить состояние кнопки без изменения LED
- Кнопка подключена к пину 2 (прерывание FALLING): при нажатии отправляет байт 0xFF в SPI-буфер (мастер при следующей транзакции считает его)

2. Lichee — ведущий игры (лаб. №12). Python-скрипт, подписывающийся на MQTT-топик game/start:

- При получении {"action": "round"}:
 - Начинает опрашивать Arduino командой 0x02 и ждёт случайную задержку 1-5 секунд; нажатие кнопки в этот интервал фиксирует как фальстарт
 - Отправляет 0x01 по SPI (зажечь LED на Arduino)
 - Фиксирует время `t0 = time.time()`
 - Публикует {"event": "go", "timestamp": t0} в game/events
 - Продолжает циклически отправлять команду 0x02 короткими SPI-транзакциями с частотой около 100 Гц и проверять байт ответа Arduino
 - При получении 0xFF от Arduino фиксирует время реакции `dt = time.time() - t0` и публикует в game/result
 - Отправляет 0x00 (погасить LED)

3. ПК — GUI-приложение. Окно с элементами:

- Кнопка «Новый раунд» — отправляет {"action": "round"} в game/start
- Крупное отображение времени реакции (мс) после завершения раунда
- Таблица последних 10 попыток (№, время реакции, результат)
- Лучший результат (минимальное время реакции за сессию)
- Индикатор состояния: «Ждите...», «ЖМИ!», «Результат»

13.13.4 Ключевые точки проверки

- Задержка перед зажиганием LED случайна и разная в каждом раунде
 - Время реакции измеряется в миллисекундах (типичные значения 150–400 мс)
 - Таблица результатов обновляется после каждого раунда
 - При нажатии кнопки до зажигания LED (фальстарт) фиксируется как ошибка и не засчитывается
 - GUI корректно отображает все состояния игры
-

13.14 Контрольные вопросы

1. Какую роль в итоговом проекте играет MQTT-брокер Mosquitto и почему он выбран в качестве транспортного протокола?
2. Почему в заданиях на связку Arduino→Lichee используется SPI, а не UART или I2C? Какие преимущества это даёт?
3. Каковы основные архитектурные паттерны, повторяющиеся во всех вариантах итогового задания?
4. Какие меры следует предусмотреть для обеспечения надёжности распределённой системы при возможных отказах компонентов?
5. Как организовать тестирование и отладку распределённой системы, когда все компоненты работают на разном оборудовании?

13.15 Порядок сдачи

1. Студент получает один из десяти вариантов задания
2. Разрабатывает и отлаживает все компоненты на оборудовании стенда
3. Демонстрирует преподавателю полностью работающую систему
4. Предоставляет исходный код всех компонентов (Arduino .ino, Python-скрипты Lichee, GUI-приложение ПК, Makefile/инструкцию по сборке для C-частей)

5. Проходит устную защиту: объяснение архитектуры, протоколов обмена, выбранных решений
6. Отправляет отчёт в согласованном формате до установленного срока.

13.16 Требования к отчёту

Отчёт должен содержать:

- цель и формулировку полученного варианта;
- архитектуру системы и схему аппаратных соединений;
- описание протоколов и форматов сообщений;
- исходные тексты компонентов и инструкции по сборке и запуску;
- результаты ключевых проверок варианта;
- ответы на контрольные вопросы;
- краткий вывод.